DinHot: A Holographic Optical Trap Studio Comparing Hologram Methods on a Device Model

Felipe Santibáñez-Leal
CAOS open-research programme, Santiago, Chile
fsantibanez@gmail.com

Technical report · v0.02.002, 31 October 2026 · Licence CC BY 4.0
CAOS open-research programme · CEFOP_DinHot, holographic optical trap studio
Version DOI 10.5281/zenodo.23850386 · concept DOI (always latest) 10.5281/zenodo.23788731
Not peer reviewed. Source code, data and computational records:
github.com/fsantibanezleal/CEFOP_DinHot

Abstract

A phase-only spatial light modulator (SLM) in the pupil of a microscope objective can form hundreds to thousands of optical traps at once, but only if the phase pattern it displays is computed well: the pattern fixes how much light reaches the traps, how equal they are and how fast it can be recomputed. The largest published neutral-atom array holds 6,100 atoms with a trap-depth spread of 11.4 percent. DinHot is an open studio that runs 24 methods (19 published ones and 5 proposed here) on the same task, with the same seeds and metrics, on a scalar Fourier model with a device model (quantization, pixel crosstalk, aperture, flicker). The report gives the model, twelve data contracts, every method with equations, pseudocode, parameters and acceptance test, 14 synthetic cases, a benchmark of 1326 rows and two learned models. Over 9 cases of the bake, among them a device-limited and an aberrated one, phase-fixed weighted Gerchberg–Saxton leaves a median relative standard deviation of 0.38 percent (per-seed medians 0.014 to 0.29) against 17 percent (16 to 19) for plain Gerchberg–Saxton, means over 3 seeds, but its published freeze index stalls on a crowded array of this grid. Three of the proposed methods meet their criteria within a measured scope (a device-aware weighted update, a flicker-bounded phase-continuous sequence and an efficiency–uniformity front), one meets a narrowed criterion (an aberration-robust hologram) and one is refuted by a control (a learned warm start). A network trained on three-plane volumes beats its baseline at equal wall-clock on all 162 held-out volumes (median accuracy ratio 1.140), whereas its two-dimensional counterpart ties with Gerchberg–Saxton (ratio 0.9994). The model is a scalar Fourier model of a simulated device, not a calibrated instrument.

Keywords: holographic optical tweezers, spatial light modulator, weighted Gerchberg–Saxton, pixel crosstalk, neutral-atom arrays, hologram benchmarking, learned holography.

1 Introduction

Holographic optical tweezers shape one laser beam into many traps with a phase-only spatial light modulator (SLM) placed in the pupil of a lens. The largest published neutral-atom tweezer array holds 6,100 atoms in about 12,000 tweezers, with a trap-depth spread of 11.4 percent across its sites [1]. The spread matters because sites with unequal depth differ in loading probability, in gate fidelity and in imaging, and the light that forms them is hard to keep uniform as the array grows; the same article names the light field as one of the limits of the scale. A small calibrated array reached a much smaller spread [2], which suggests that the spread is not a fixed limit but depends on how the hologram is computed and on how the device realises it; the two arrays differ in size and set-up, so the comparison is not a controlled one and neither citation shows the cause. Quantum-computing programs pull the array size up: continuous operation of a coherent array of three thousand atoms [3] and a logical processor on reconfigurable atom arrays [4] depend on arrays that are loaded, rearranged and refreshed many times, and learned holography has been used to assemble defect-free arrays within a constant time [5]. Camera feedback closes the loop on a real set-up and reaches a non-uniformity of about one percent in arrays of more than a thousand spots [6], at the cost of a camera

path and iterations for every array. The calibration of the SLM itself, the wavefront and the pixel crosstalk, is becoming a model-based step that takes minutes [7].

The same computation serves other users with different limits. Single-molecule biophysics laboratories use optical tweezers as the method of choice for single-molecule manipulation [8], and parallel traps help them only when the traps exert equal force; neuroscience groups stimulate ensembles of neurons with holographic patterns, where heating and the intensity that falls far from the zero order bound the light [9]; laser micro-processing splits one beam into many with a spatial light modulator, with about half of the light in the first-order beams [10]; and near-eye display research has developed camera-in-the-loop training of holograms that overlaps with the methods here [11]. What the users share is a decision that is taken before any experiment: which method to use to compute the phase, with which device model, and at what cost in time and light.

The hologram methods that answer this decision span half a century: the superposition of prisms and lenses, random superposition and random masks, the Gerchberg–Saxton (GS) iteration and its adaptive and weighted forms [12], direct search [15], conjugate-gradient minimization [19], phase-fixed weighted GS for large arrays [16], mixed-region amplitude freedom for shaped potentials [18], gradient descent through differentiable forward models [20], optimal-transport initialization [32, 33] and learned holography [29]. A search of the literature found no controlled published comparison of weighted GS, phase-fixed weighted GS, conjugate gradient, MRAF and a learned method on the same point-trap array, and no public tool that runs them side by side with a device model and the physics of what is trapped.

DinHot provides that comparison. It began at CEFOP, Universidad de Concepción, as DinHotSys, a C++ desktop tool for dynamic holographic optical tweezers dated 2010; it was rewritten in Python and FastAPI and is rebuilt here as a static browser application whose numbers are computed by the engine or baked offline and replayed. The studio runs 24 methods (19 published ones and 5 proposed here) on 14 cases, with the same seeds, metrics and device model, in Python as the reference and in a TypeScript port that runs live in the browser.

Contributions. The report makes the following contributions.

- A scalar Fourier model of the SLM, the lens and the device (quantization, pixel crosstalk, pixel aperture and flicker) with closed-form adjoints, so that every gradient method differentiates through the device without automatic differentiation (Sections 2 and 6).
- Twenty-four hologram methods implemented behind one interface, nineteen of them replicated from the literature and five proposed here, each with a stated acceptance criterion and a test that measures it (Section 6, Table 19).
- Twelve data contracts with schemas, examples, producer and consumer tests and a declared behaviour for missing data (Section 4).
- A benchmark protocol of 1326 rows over 7 geometries, the ideal device and the device of case C08, every budget of the protocol and up to 20 seeds, and the measured results of the 14 cases (Sections 7 and 8).
- Two learned models trained offline and evaluated on held-out sets written before training: one that beats its baseline at equal wall-clock, and one that is refuted by a control (Section 8.4).
- Measured findings that differ from the sources they replicate, each kept as a result: the source value of the freeze index of phase-fixed weighted GS stalls on a crowded array of this grid, and the reconstruction of the adaptive form needs a scale convention (Sections 6.7 and 6.5).

Structure. Section 2 states the problem and the models; Section 3 collects the notation; Section 4 documents the cases and the contracts; Section 5 describes the pipeline, the engine and the lanes; Section 6 describes the methods one by one; Sections 7 to 9 give the protocol, the results and the validation; Sections 10 to 13 discuss, bound and conclude. Every number in the tables and the text is generated from the committed artifacts of the repository or measured by the scripts kept with this report; the browser timings are quoted from the product's record of measured resources, and the executive figures of this introduction are taken from the cited publications and not re-measured.

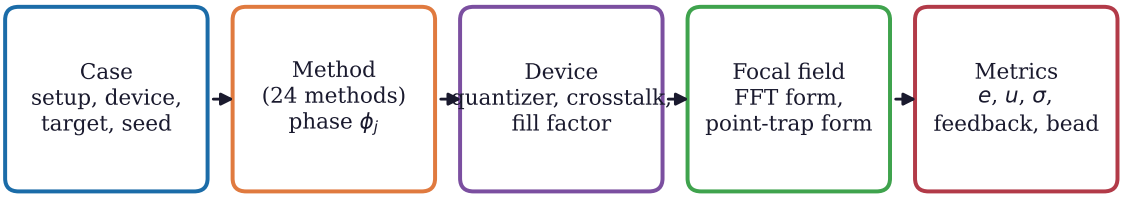


Figure 1: The chain every case runs: a case (optical setup, device, target and seed) is given to a method, which returns a phase; the device model turns the phase into the field the instrument forms; the metrics score the traps, and the feedback and particle simulations act on the scored field.

2 Problem formulation

2.1 Inputs, outputs and variables

The problem is posed per case, and every case runs the chain of Figure 1. A case fixes an optical setup, a device, a target, an aberration, a set of methods and budgets, and a seed. The optical setup is the vacuum wavelength λ, the focal length f of the lens, the numerical aperture NA, the immersion index n_{imm} and the input beam, a Gaussian whose intensity is $\exp(-2r^2/w_{\mathrm{b}}^2)$ with waist $w_{\mathrm{b}} = w_{\mathrm{frac}}\, n_{\mathrm{pix}}\, p/2$, normalised so that $\sum_j A_j^2 = N_{\mathrm{pix}}$. The device is the SLM: $n_{\mathrm{pix}} \times n_{\mathrm{pix}}$ pixels of pitch p, a phase resolution of b bits, a fill factor, a crosstalk kernel and a flicker amplitude. The target is either a list of traps, each with a position (x_m, y_m, z_m), a relative weight w_m and a vortex charge l_m, or a shaped potential given as an amplitude and a signal mask on the focal grid. The aberration is a set of Zernike coefficients in waves, and the seed fixes every random draw of the run. The output of a method is a phase $\phi_j \in [0, 2\pi)$ for every pixel j at position (x_j, y_j) from the optical axis, together with the history of the metrics along the iterations. The quantities that the problem estimates are the metrics of the hologram on the device (Section 2.2) and the dynamics of what is trapped (Section 2.3).

Table 1 lists the symbols and their units. Lengths on the focal plane are carried in micrometres and in Fourier units, one unit being the width $\lambda f/(n_{\mathrm{pix}} p)$ of a bin of the unpadded transform, so that a trap on an integer multiple of a bin has a closed form in both models. A trap is refused if its position lies at or beyond the unaliased half field of Eq. (3), which is the rule that case C11 exists to prove.

2.2 Forward model, objective and constraints

For a trap m at (x_m, y_m, z_m) with vortex charge l_m, the phase delay from SLM pixel j is

$$\Delta_j^m = \frac{\pi z_m}{\lambda f^2}(x_j^2 + y_j^2) + \frac{2\pi}{\lambda f}(x_j x_m + y_j y_m) + l_m \operatorname{atan2}(y_j, x_j), \tag{1}$$

and the field and the intensity of trap m are

$$V_m = \frac{1}{N_{\mathrm{pix}}} \sum_j A_j e^{\,i(\phi_j - \Delta_j^m)}, \qquad I_m = |V_m|^2, \tag{2}$$

following Di Leonardo et al. [12], so that I_m is the fraction of the input power in the peak of trap m. The second form of the model is the centred, zero-padded, orthonormal fast Fourier transform of the SLM field; at a trap on a bin with $z = 0$ and no vortex the two forms agree exactly, $V_m = E[\mathrm{bin}]\,\mathrm{pad}/n_{\mathrm{pix}}$, which the test suite checks. One bin of the padded transform and the half-window that the pixel sampling leaves unaliased are

$$\Delta x = \frac{\lambda f}{\mathrm{pad}\, n_{\mathrm{pix}}\, p}, \qquad X_{1/2} = \frac{\lambda f}{2p}, \tag{3}$$

and the grid spans the replica period $\lambda f/p$ of the pixel sampling, so that replicas fold onto it and are counted as ghosts in the power budget. Depth is handled by the angular spectrum with the evanescent components

removed,

$$H = \exp\Big(2\pi i \frac{\Delta z}{\lambda_m}\sqrt{1-(\lambda_m\xi_x)^2-(\lambda_m\xi_y)^2}\Big), \qquad \xi_x^2+\xi_y^2 < \lambda_m^{-2}, \quad \lambda_m = \frac{\lambda}{n_{\mathrm{imm}}}, \tag{4}$$

which assumes the scalar paraxial model, so that the axial position of a displaced trap is exact only for $|z| \ll f$.

With I_m the trap intensities (divided by the target weights where they differ) the problem is scored by three metrics,

$$e = \sum_m I_m, \qquad u = 1 - \frac{\max I - \min I}{\max I + \min I}, \qquad \sigma = 100\,\frac{\mathrm{std}(I)}{\mathrm{mean}(I)}\ \text{percent}, \tag{5}$$

the efficiency, the uniformity ($u = 1$ is perfect) and the relative standard deviation, the last with the population standard deviation. Polin et al. [15] use the same ratio without the leading one, a non-uniformity; the two are never mixed here. A power budget separates the traps, the zero order, the ghost traps (everything else on the grid, including folded replicas) and the remainder, which Parseval's relation makes zero for the plain Fourier form. The problem is a multi-criteria one: maximize e and u together (and minimize σ), over the phase ϕ, subject to the constraints that the phase is realisable by the device, that the traps lie inside the unaliased field, and that the input beam has the stated, not necessarily flat, amplitude. The last constraint bounds the efficiency. For one trap the bound follows from Eq. (2), since $|V_m| \le \overline{A}$; for the metric of the engine the record of the forward model states the ceiling for a Gaussian input beam as

$$e \le (\overline{A})^2, \qquad \overline{A} = \frac{1}{N_{\mathrm{pix}}}\sum_j A_j, \tag{6}$$

which is 0.869 at the test setup. The bound holds for one trap: $|V_m| \le \overline{A}$, with equality for the phase that puts all the terms of Eq. (2) in phase, and $\overline{A}^2 \le 1$ by the Cauchy–Schwarz inequality, with equality for a flat beam. For many traps on distinct Fourier units the traps are orthogonal channels of a unit-norm pupil field, and the only bound that holds for every beam and every phase is $e \le 1$ (Bessel's inequality); $\overline{A}^2$ is not a bound there. A numerical maximization of the sum of the trap intensities over all pixel phases for the Gaussian beam of the test setup (`data/ceiling_check.py`: Adam, 600 iterations, one seed, the closed-form adjoint of the engine) gives $e = 0.932$ for ten traps one Fourier unit apart, above the reference level 0.869, while ten traps eight bins apart reach 0.870, the level itself, and a 10×10 lattice at the same spacing reaches 0.841 in the same budget, below it. These are measurements on one setup and one seed and not a proof, and they depend on the spacing; traps on adjacent bins of the padded grid are not orthogonal channels, and the same check gives 1.85 for them, above one, so that Bessel's bound does not apply to them. The value $\overline{A}^2$ is used in this report only as the reference level of the single-trap limit, and the acceptance criteria that state a fraction of it are thresholds against that reference, not against a proven ceiling. For a shaped potential the metrics are the signal-region ones: the fraction of the power in the signal region, and the root-mean-square fractional error η of Eq. (25) over the measure region.

2.3 The device and what is trapped

A real SLM quantizes the phase, couples neighbouring pixels, covers only part of each pixel and fluctuates. The device model applies, in order, wrapping, quantization to 2^b levels, a nonlinear response $2\pi(\phi_q/2\pi)^\gamma$, Gaussian flicker of stated rms and wrapping, and blurs the upsampled phase with the crosstalk kernel of Eq. (34), whose fitted parameters $\sigma_K = 1.24$ and $q = 1.80$ on 3×3 sub-pixels are those of Schroff et al. [23] (an error below two percent in that work). A square pixel of fill factor fill multiplies the focal field by a product of cardinal sines, so that the intensity envelope and the first-order efficiency of a stepped grating with N levels are

$$\mathcal{E}(x) = \mathrm{sinc}^2\Big(\frac{\sqrt{\mathrm{fill}}}{2}\,\frac{x}{X_{1/2}}\Big), \qquad \eta_1 = \mathrm{sinc}^2\Big(\frac{1}{N}\Big), \tag{7}$$

so that the intensity falls to about 0.4 at the edge of the unaliased window; the envelope is derived in the engine and tested against a brute-force transform, and the stepped-grating efficiency is a published result [44] (Figure 2 shows both, and the kernel). Flicker is a computed parameter: the one measured device that was read is a twisted-nematic display with a large fluctuation [43], so the flicker of case C08 is a stated parameter and not a calibrated one. Zernike aberrations are expanded in the Zernike polynomials of Noll [25], with the single-index ordering of the engine (Section 6.14), and enter as an additive wavefront on the SLM phase.

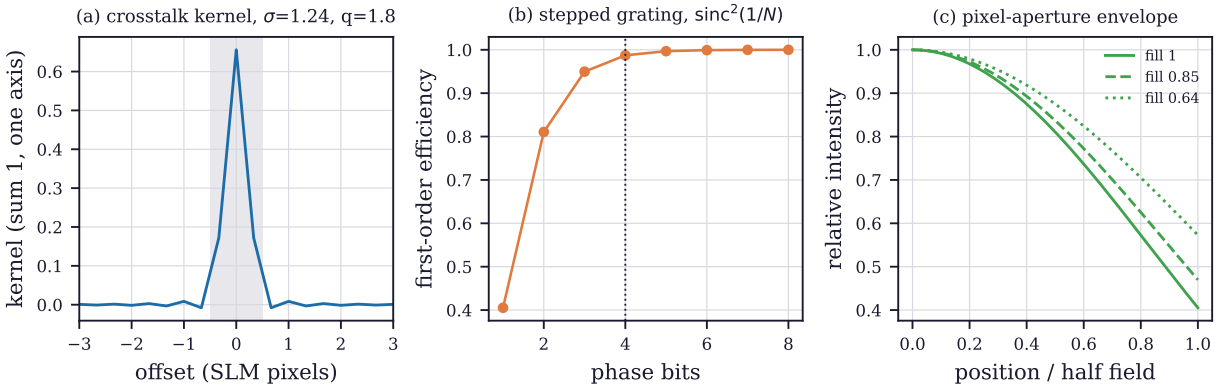


Figure 2: The device model at the parameters of case C08: (a) the crosstalk kernel along one axis, the inverse transform of Eq. (34) on the sub-pixel grid, with the width of one SLM pixel shaded; (b) the first-order efficiency of a stepped grating against the number of phase bits, with the C08 resolution dotted; (c) the pixel-aperture envelope of Eq. (7) across the unaliased half field, for fill factors of one, the C08 value and 0.64.

What is trapped follows from the intensity. For an atom in a red-detuned beam the dipole potential and the radial and axial trap frequencies of a focused Gaussian are, after Grimm, Weidemüller and Ovchinnikov [35],

$$U_{\mathrm{dip}} = \frac{3\pi c^2}{2\omega_0^3}\frac{\Gamma}{\Delta} I, \qquad \omega_r = \sqrt{\frac{4\hat{U}}{m w_0^2}}, \qquad \omega_z = \sqrt{\frac{2\hat{U}}{m z_R^2}}, \tag{8}$$

with $\hat{U}$ the trap depth, w_0 the waist and $z_R = \pi w_0^2/\lambda$ the Rayleigh range of the focus. For a Rayleigh bead the gradient force gives a stiffness proportional to the intensity curvature [39],

$$\kappa_i = -\frac{\mathrm{Re}(\alpha)}{2 n_m c \epsilon_0}\frac{\partial^2 I}{\partial x_i^2}, \qquad f_c = \frac{\kappa}{2\pi\gamma}, \qquad P(f) = \frac{D}{\pi^2(f^2 + f_c^2)} \quad (f > 0), \tag{9}$$

where α is the polarizability, γ the Stokes drag, $D = k_B T/\gamma$ the diffusion coefficient and $P(f)$ the thermal Lorentzian spectrum of a trapped bead, from which the stiffness is calibrated [40]. The convention is the one-sided spectrum on $f > 0$, which is the form the engine fits to the simulated trace; the two-sided spectrum on $-\infty < f < \infty$ has half that value, $D/[2\pi^2(f^2 + f_c^2)]$. For a sphere of radius r and index n_p in a medium of index n_m the polarizability is $\alpha = 4\pi\epsilon_0 n_m^2 r^3 (m^2 - 1)/(m^2 + 2)$ with $m = n_p/n_m$, and $I = \frac{1}{2} n_m c \epsilon_0 |E|^2$ is the intensity in the medium, so the gradient force is $\mathbf{F} = \frac{2\pi n_m r^3}{c}\frac{m^2-1}{m^2+2}\nabla I$, derived here from the polarizability above (Harada and Asakura [38] treat the radiation forces on a dielectric sphere in the Rayleigh regime; their printed equation was not checked); the dipole force of Bouloumis and Nic Chormaic [39] (their Eq. 9) is its vacuum case, $n_m = 1$. A bead with $n_p < n_m$ has a negative polarizability and is pushed out of the focus.

2.4 Assumptions and where each comes from

The model is scalar and paraxial: the engine computes Fraunhofer and angular-spectrum propagation, and a high numerical aperture is flagged as outside its validity (case C07) because the vectorial equations of the focus were not read at the source. The cases are synthetic: the optical setups, devices and targets are stated parameters taken from published experiments, labelled as such, and no measured sample is used. Absolute numbers therefore belong to the stated model and not to a measured instrument. The device parameters of the crosstalk kernel come from a published fit, while the fill-factor, four-bit and flicker effects have no published source in this build and are computed only; the pixel that the kernel width refers to (SLM or computational pixel, a factor of three) is the engine's reading and is not settled by the source (Section 6.13). The emulated camera has errors that are stated parameters. The input beam is a Gaussian of stated waist unless a test uses a flat beam. The randomness is the portable generator of the engine, so that every run is a pure function of its parameters and its seed. Each of these assumptions bounds a claim in Section 11.

3 Notation

Table 1 fixes the symbols that are used in more than one section. Three conventions apply throughout. First, indices: j runs over SLM pixels and m over traps, the iteration is a superscript k or a subscript i where the

source of the method uses it, and the focal bins carry the coordinate $\mathbf{u}$ of the padded transform. Second, normalisation: every intensity I_m is a fraction of the input power, so that the efficiency e lies between zero and one (the single-trap reference level of Eq. (6) is not a bound for many traps); the relative standard deviation is carried in percent and never as a fraction, which the units gate of Section 9 checks on every shipped artifact; and phases are in radians, with the Zernike coefficients in waves and the unit-rms basis so that the rms of the wavefront is the root sum of squares of the coefficients. Third, units: positions are in micrometres on the focal plane and in metres in the engine's internal interface, the pixel pitch in micrometres, the wavelength in nanometres, the focal length in millimetres, times in seconds and frequencies in hertz; a position in metres where micrometres are expected would be about a hundred thousand times too small, which the same gate catches. Every table and every figure names each method by its name, and the order of the methods in a table is that of the ladder and not a ranking.

The objects that carry the notation in the code are the setup of the optical system, the target (traps or a shaped field with its signal mask), the budget (an iteration cap and an optional wall-clock cap), the result (the phase, the history, the trap fields and a metadata record) and the method specification (identifier, name, family, lane, supported targets and dimensions). The interface is documented in Section 5.2. Where a symbol is overloaded by the sources, the report keeps the source's symbol in the method section where it is used and states the mapping: for example the exponent α of the assignment cost of Eq. (40) and the weight α of Eq. (18) are different quantities, and the mixing m of Eq. (24) is not the trap index. The overloads that remain are these. The polarizability α of Eq. (9) is a third α; the index ratio $m = n_p/n_m$ of the polarizability and the atomic mass m of the trap frequencies of Eq. (8) are two more m. The width w_{b} of the input beam is not the trap weight w_m, and the waist and Rayleigh range of the focus of an atom, w_0 and z_R, are not the SLM-limited range $z_{R,\mathrm{SLM}}$ of the volume network. The focal length f of the lens is not the frequency f of the bead spectrum nor the weight f_u of the direct-search cost. The relative standard deviation σ of Eq. (5) is in percent, whereas the kernel width σ_K and the residual σ of the direct-search cost are not; γ is the Stokes drag of the bead, the best scale of the direct-search cost and the coefficient of the conjugate-gradient direction in the three places where each is defined; and η is the error of a shaped potential, the first-order efficiency η_1 of a stepped grating and the efficiency level that the optimizers request, each defined where it is used.

4 Data and data contracts

4.1 Data sources and splits

DinHot reads no external dataset. Its inputs are the 14 case records of the case library, written to a source document by the pipeline, and the published anchors that the acceptance criteria quote. The owner of the case library is the product itself (the repository named in the header block, licence Apache-2.0); each record states its parameters in the units of Section 3, and every record is synthetic, so no sensitive or measured sample is involved. The records span 9 to 1024 traps and four categories of cases, listed in Section 7. The disordered array of case C13 reproduces the geometry of Kim et al. [16] with the coordinates drawn by DinHot, because the published ones are not available. The quasi-periodic set of the benchmark is likewise DinHot's own, since the 119 traps of Polin et al. [15] are not published as coordinates.

The derived data are the committed artifacts of the build. The directory of derived data holds 35 files of 2559 KiB in all, the largest trace being 228 KiB against the lane gate of 256 KiB, and the released graph of the learned model is 4.67 MB. The artifacts are written by the pipeline and read by the web, the API and the tests; they are committed so that a reader can re-check every number without running the bake, and the pipeline tests run in a sandbox that fails the test run if a committed artifact changes.

The splits matter only for the two learned models. The training data are generated by a seeded generator in the data root and never committed. The held-out sets are written apart before training, with seeds other than the training ones, and their hash is recorded in the model card: 162 volumes for the three-plane network and 66 geometries for the warm start. A model is judged on the held-out set only, and it is released only if it meets its criterion; otherwise the baseline answers behind the same contract. The acceptance tests of the other methods use no split: a test fixes a task and a seed, and the benchmark repeats the tasks over up to 20 seeds, with the tuning of a method run on other seeds (offset by a thousand) at 30 iterations, so that no tuned row is scored on the seeds that tuned it.

Each data contract of the catalog is described below in the order of the catalog: purpose, fields (type, unit, range, in the table of the contract; the nested fields of the larger contracts are in Appendix A), producer and consumers, version and the behaviour when data are missing. A schema file is the single source of its

contract: the Python producers and consumers validate against it in their tests, and the TypeScript types are generated from it. A change bumps the contract's version by the semantic-version rule (a removed or renamed field, a narrowed type or a new required field is a major step, a new optional field a minor step, a description change a patch) and updates both sides in one change. Table 2 lists the twelve contracts.

4.2 Case record (params-record)

The params-record contract (version 1.0.0) is the ingestion boundary: the record of one case, with the optical setup, the device, the target (traps or a shaped potential), the aberration, the method set and budgets, the seed, the feedback, dynamics and rearrangement blocks and the expected ranges of the metrics. Its fields are in Table 3, and the nested ones in Appendix A. The producer is the module that validates records against the schema with a standard-library validator, and the consumers are the bake and the tests. The contract states rules a schema cannot: a trap at or beyond the half field is refused with the field's bounds, which case C11 exists to prove. The missing-data behaviour is partial: a rejected record is dropped with its reason and the bake goes on with the accepted records, and the readiness document lists the rejected case ids; if the source document of cases is absent, the bake uses the registry's cases and readiness says missing.

4.3 Case index (case-index)

The case-index contract (version 1.0.0) is the inventory of the build: one entry per case, with bilingual titles and categories, the default case, and the status (baked or refused) with the reason for a refused one. Its fields are in Table 4. The producer is the manifest module of the pipeline and the consumer is the frontend, which reads the index first, so that without it there is no site. The behaviour when it is missing is therefore the strictest of the catalog: the site build stops at the copy of the data, the API answers 503 with the state missing, and readiness says missing. When the index is stale, because its engine version is older than the product's, readiness says stale with both versions and the views show it. A refused case has an entry and no manifest, and the workbench shows its reason. A change to any field of this contract bumps its version, now 1.0.0, by the semantic-version rule of the catalog, so that a reader holding a document can tell which fields it may carry.

4.4 Case manifest (case-manifest)

The case-manifest contract (version 1.0.0) describes one baked case: its parameters and seed, the engine version, the lane label and the gate that produced it, the score per method (the metrics scored on the case's device), the models behind the numbers, the expected ranges and the bilingual band, and the published anchor where one exists. The fields are in Table 5, with the nested ones in Appendix A. The manifest carries no clock: the run time that decides the lane is stored as a lane label and as the deterministic budgets, never as milliseconds, so that two bakes of the same inputs write identical bytes, which a test compares byte by byte. If a manifest that the index lists is absent, the contract is partial: readiness lists its path, the case selector marks the case as not available yet and the other cases work, and the API answers 404 for it. A stale manifest is handled as for the index.

4.5 Trace (trace)

The trace contract (version 1.0.0) is the compact replay artifact of one case, at most 256 KiB. It holds, per method, the metric series against iteration (at most 200 points, first and last kept), the final per-trap intensities and metrics; for the case's reference method, the quantized phase mask (256 levels, base64), the focal intensity in decibels relative to the mean trap on a crop of at most 128 bins, with blocks max-pooled so that a peak is never lost, and the axial cut for three-dimensional cases; and, for the dynamic cases, the bead or atom trajectory (at most 4000 points), the power spectrum (at most 200 log-spaced bins) and the rearrangement frames. Table 6 lists the top-level fields. A trace that would exceed the byte gate is squeezed through a fixed list of five settings, and one that still does not fit raises an error, so the bake fails and the lane gate never sees an oversized artifact. When an artifact that a manifest names is absent, the case's replay views show not available yet with the reason and the live lane still runs from the manifest's parameters.

4.6 Benchmark results (benchmark-results)

The benchmark-results contract (version 1.0.0) holds the table of the benchmark protocol: one row per method, geometry, device, budget kind, budget and parameter set, with the median and the inter-quartile range over the seeds of the efficiency, the uniformity, the relative standard deviation, the wall-clock time and the iterations, the number of seeds of the cell, the stored Pareto fronts and the cells that did not run, with the reason (Table 7). The producer is the benchmark stage and the consumer is the benchmark views of the frontend. A cell that cannot run is listed in the skipped list and is absent from the rows, never silently: readiness lists the cells as method and geometry, and the views mark each one as not run. If the document is absent, the views say not available yet with the reason and never show a number from an older bake. A change to any field of this contract bumps its version, now 1.0.0, by the semantic-version rule of the catalog, so that a reader holding a document can tell which fields it may carry.

4.7 Model cards (cgh-net and warmstart-model)

Two model slots share the model-card schema (Table 8; the warm-start card is in Table 9). The cgh-net contract (version 1.0.1) maps the target amplitudes of three focal planes at -8, 0 and $+8$ SLM-limited Rayleigh ranges, unit-sum intensities, for a 128×128 SLM with no padding, to one phase in radians; the baseline behind it is the volumetric Gaussian GS at 30 iterations. The card names the ONNX graph, its size, its sha256 and its opset, the grid, the input and output tensors with their dtype and shape, the training record and the held-out set. The behaviour when the slot has no valid released model (the card or the graph is absent or fails its schema or its checksum) is the state training with the baseline provenance: the baseline answers, and the views label the result with the model that answered. A marker file means that the released model answers while a newer one trains.

4.8 Warm-start model (warmstart-model)

The warmstart-model contract (version 1.0.1) maps a padded list of up to 1024 traps (position in focal bins and in Rayleigh ranges, plus the weight) and a mask to one phase per trap in radians, with the random trap phases of the portable generator as its baseline. It shares the card schema of the previous contract and its missing-data behaviour, with the baseline being the random start. In this build the slot is refuted and no model file is committed: the record of the refusal holds the held-out metrics and the control, and the contract test asserts that no warm-start model file exists, so the state of the slot is the baseline answering. The contract is nevertheless documented because the schema, the example and the swap test make the refusal a tested path and not an omission. A change to any field of this contract bumps its version, now 1.0.1, by the semantic-version rule of the catalog, so that a reader holding a document can tell which fields it may carry.

4.9 Held-out metrics (model-metrics)

The model-metrics contract (version 1.0.1) holds the held-out evaluation of the model slots against their baselines: the number of held-out cases, the accuracy ratio against the baseline at equal wall-clock, the time of the model and the hash of the held-out file, for the slots that have a released model (Table 10). The producer is the evaluate stage; the consumer is the benchmark views of the frontend. A slot with no released model has no entry and its views say so, because a metric for a model that is not released would be a number about a model nobody can use. The comparison it records is always against the baseline of the slot at equal wall-clock, never against a fixed number. If the document is absent, the views say not available yet and never show a number from an older bake; if its engine version is older than the product's, readiness says stale.

4.10 Training evidence (model-evidence)

The model-evidence contract (version 1.0.0) is the evidence behind the metrics: the training records of the released model, the records of the rejected candidates (the single-plane network and the warm start) and the evaluation detail with the ratio by kind of target (Table 11). The producer is the training stage and the consumers are the tests of the two learned methods, which read the records to assert the criteria: the committed metrics of the volume network, and the measured negative of the warm start together with its control. A slot with no training record has no evidence to show, and the benchmark page says that the slot answers with its baseline, as for the metrics document. A rejected candidate is kept on purpose: a

negative result is a result, and the record lets a reader re-check the refusal. A change to any field of this contract bumps its version, now 1.0.0, by the semantic-version rule of the catalog, so that a reader holding a document can tell which fields it may carry.

4.11 Engine reference values (engine-reference)

The engine-reference contract (version 1.0.0) holds the reference values that the Python engine writes and the TypeScript parity tests read: for each quantity, the inputs, the expected outputs and the tolerance, absolute and relative (Table 12). It is a test input and never a run-time document, so its missing-data behaviour is that readiness says missing when the folder of fixtures holds no file and the parity tests fail naming the absent file. The fixtures are the means by which the live engine is held to the reference: 46 reference documents are committed, with a loosest absolute tolerance of 10^{-9}, and 92 browser-side tests compare the port with them (Section 9). The documents are written by the Python engine from fixed inputs and seeds, so that a change of the engine that moves a result beyond the stated tolerance fails a test in the repository instead of surfacing in the browser. The version of the contract is that of the schema, and a change in a tolerance is a change in the contract.

4.12 Readiness (readiness)

The readiness contract (version 1.0.0) says what is real in a build, row by row, in one of six declared states: available, partial, stale, provider, training and missing (Table 13). The document is computed from the catalog and the repository, and the front end and the API show it; editing the catalog makes it stale until it is recomputed, which a guard fails on. Of the 11 rows it judges, 7 are fully available in this build, and the build declares the state of the others: `params-record` is partial (1 of 14 records rejected by contract 1); `benchmark-results` is partial (61 of 168 benchmark cells were skipped); `warmstart-model` is answered by its baseline, with no released model; `api` is missing (The API lane is dormant; the site reads the committed documents). If the document is missing, the site build stops, and if it cannot be loaded at run time the views keep working and each says readiness unknown. A provider row, a row served by a stand-in while the real producer is not built, is labelled on every surface and never counted as done; in this build every row has a real producer. A change to any field of this contract bumps its version, now 1.0.0, by the semantic-version rule of the catalog, so that a reader holding a document can tell which fields it may carry.

4.13 Read-only API (api)

The api contract (version 1.0.0) is the dormant read-only layer over the committed documents: GET only, with the index, a case manifest, an artifact by path, the benchmark results, the model metrics and the readiness document (Table 14). It is owned by the application folder, tested against the OpenAPI document of the catalog, and consumed by the typed client of the frontend. While no document is baked, or while a route's document is absent, the API answers 503 with the state missing and the reason, never an empty body; an unknown case or artifact is 404. The static readiness says missing for this row while the API lane is dormant, and the dynamic readiness route says available. The API is the only boundary whose producer is a service and not a file, which is why its contract is the OpenAPI document and not a schema. A change to any field of this contract bumps its version, now 1.0.0, by the semantic-version rule of the catalog, so that a reader holding a document can tell which fields it may carry.

5 System architecture

DinHot is an instance of a product archetype in which the science runs offline, its results are committed, and a static web workbench replays them and re-runs what is light enough, live. Figure 3 shows the data flow: the case records enter through the first contract, the named stages compute and score every method, the gates check the result, the second contract writes the artifacts, and three lanes serve them. The base (the layout, the contracts, the stage names, the lanes and the guards) is carried by the product template and its shared shell; the core is DinHot's own: the engine, the methods, the cases and the views.

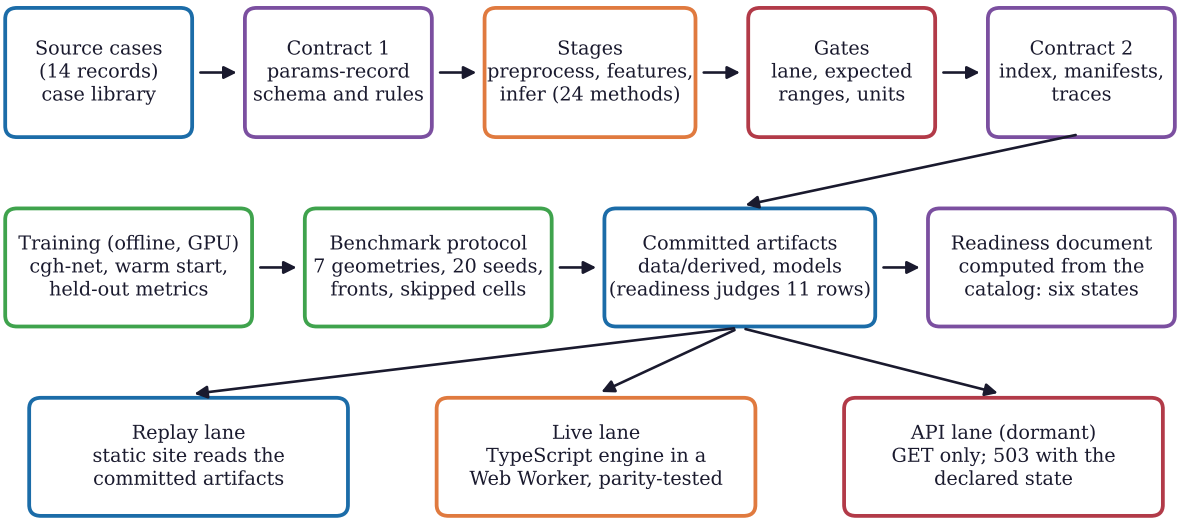


Figure 3: Data flow of the build: the case records pass the first contract, the stages run every valid method with the same seed and score it, the gates check lane, expected ranges and units, and the second contract writes the artifacts that the replay, live and dormant API lanes serve; training and the benchmark are offline and write committed artifacts too.

5.1 The staged pipeline

The pipeline orchestrates named stages whose names and signatures are frozen by the base, and whose bodies are DinHot's. Every stage is deterministic, typed, seeded and independently testable. Table 15 lists them with their inputs, outputs and artifacts. The per-case bake runs, in order: the contract-1 record, the methods through the registry, the expected ranges, the lane measurement, the maps and drivers, the trace within the byte gate, the units gate and the manifest. It never raises for a method or a gate problem, so that one bad case does not hide the others: the failures are collected and one error names case, method or driver and metric at the end. The cases bake in parallel, one case per process, at most twelve processes with one linear-algebra thread each. The canonical bake is an explicit release operation and never runs in continuous integration; tests and quick runs write to a sandbox so that they cannot change the committed evidence.

5.2 The engine and the method interface

The engine lives in a folder of the pipeline package and is documented card by card in the frameworks of the repository; it is not a package of its own. A method is a module with a specification and a run function; the registry discovers it, and the same interface serves the bake, the benchmark and the TypeScript port. The specification names the method's identifier, family, lane (live, both or baked), supported target kinds and dimensions. The run function takes the optical setup, the target, an optional device, a budget, the random generator and an optional starting phase, plus the keyword parameters of Table 16, and returns a result with the phase, a history with one entry per iteration (iteration, efficiency, uniformity, relative standard deviation and time), the fields at the traps and a metadata record. Of the 24 methods, 16 are live or both and 8 are baked. A method that does not accept a target refuses it with an error that names the supported kind, so a missing combination is a declared state and not a silent fall-back.

A method reports its own history, but the score that the manifest and the benchmark carry is recomputed by the scorer from the final phase, through the same forward model for every method and through the case's device and aberration. A method therefore cannot grade itself, and the methods are compared on one footing. The forward model, the device model, the metrics, the trap physics, the particle dynamics, the emulated camera and the portable random generator are the physics components that every method works inside; the components are described by the equations of Section 2.

5.3 Determinism and the trace

A run is a pure function of its parameters and its seed. There is no global random generator anywhere: every random draw goes through a portable generator of the xoshiro128 family seeded by SplitMix32 [42], whose uniforms the TypeScript port reproduces bit for bit and whose normals it reproduces to rounding. The draw order is part of the contract and is fixed in each module's documentation (the starts, the camera gains and noise, the device flicker, the particle noise). Every method of a case is given the same seed, so all of them start from the same random numbers and the same budget. Same inputs give byte-identical artifacts: the manifest carries no clock, and the only measured quantity behind it, the run time that decides the lane, is stored as a lane label and the deterministic budgets.

The trace is decimated and not the raw state (Section 4.5): histories to at most 200 points, the reference method's phase in 256 levels and its focal intensity in decibels with a floor, an axial cut for three-dimensional cases, trajectories to at most 4000 points and rearrangement to at most 120 frames. Training data sets and checkpoints stay in the data root and never in the repository; only the compact artifacts, the released models and their cards are committed and shipped.

5.4 The gates

Three pipeline gates stand between a method's output and a committed artifact, and a fourth, measured on the built site, belongs to the web. The lane gate decides, by measurement, whether a case may be re-run live in the browser. The unit of judgement is the case's reference method; the case is live when that method's lane is live or both, one run at the case's own grid and budget takes at most the run budget of 1500 ms, and the trace is within 256 KiB. Every method of the case also gets a lane of its own, and a method labelled live that does not fit is baked, never silently live. The cost judged is one run after a warm-up as the median of three, because timing the whole case once, validation and cold start included, made the lane label depend on bake order.

The expected-range gate states, for every case, a numeric range for named metrics (the metrics of the reference method, the case metrics measured by a driver, and the metrics of each method), and the bake fails on a value outside it, naming the case, the method and the metric. An expectation written only as prose can be wrong for months without anyone noticing; the ranges are measured on three seeds, each range the wider of ten percent around the median and the seed spread, and a range stays provisional where no published band sets it. The units gate checks, in the bake and again over the shipped artifacts, that every observable is in its physical domain and in the units of the question it answers: efficiency, uniformity and the fractions between zero and one; the relative standard deviation in percent between zero and a thousand; trap intensities relative to the mean trap; positions in micrometres inside the unaliased field; times in seconds, stiffness in piconewtons per micrometre and frequencies in hertz. The relative standard deviation and the uniformity are recomputed from the per-trap intensities and must agree, so a fraction written where a percent belongs fails.

5.5 The lanes and the live engine

The offline lane bakes every case with every valid method, checks each against its expected ranges and runs the benchmark and the units gate; training runs locally only. The replay lane serves the committed artifacts, copied into the site and fetched with the version in the address; the copy script stops the build when the released graph is absent or its size or checksum is not the card's. The live lane is a TypeScript port of the forward models, the device model and the live methods, running in a Web Worker so that the interface never blocks, with the fast Fourier transform in double precision (the fft.js library). An optional WebGPU backend runs the Fourier-form iterations of plain GS, weighted GS and phase-fixed weighted GS for ideal-device point traps on bins with no depth offset and no vortex, and every absent, ineligible or failing GPU run takes the CPU path and records the reason; an agreement check inside every GPU run compares the trap fields with the CPU result.

At the live grid (256^2 padded to 512^2) a weighted GS iteration took about 24 ms in Node 24, and in the browser's worker 30 iterations on C02 took 0.49 to 0.80 s, inside the lane budget. On an integrated Intel gen-12lp adapter, 30 iterations of phase-fixed weighted GS took 0.31 to 0.32 s with the GPU backend against 1.5 to 1.6 s on the CPU path in the same browser. The Python engine writes 46 reference documents and 92 parity tests compare the TypeScript engine with them, each quantity within the tolerance its document states (at most 10^{-9} absolute). The live lane is held to the Python engine by tests over fixtures that the

Python engine writes: the ported methods are exactly those whose Python lane is live or both, and every other id raises a not-live error and is replayed from the bake. The methods that are baked and replayed are labelled as such in the interface, and the workbench labels every map with its lane. The ONNX runtime for the web runs the released network of Section 6.17 on one thread, since the static host sends no cross-origin isolation headers.

5.6 The API lane and the deployment

The API lane is dormant: a read-only layer over the committed documents that answers 503 with the declared state while nothing is baked or a route's document is absent. It exists so that the product keeps the same contracts if a server is added, and it is tested against its OpenAPI document. The site itself is static: every computation runs in the browser or was baked offline and is replayed, and the deployment is the upload of the built site to a static host after the main branch passes the guards. The measured gate walks the built site at five sizes, in both themes and both languages, and a person reads its captures; the gate measures and does not judge content. The browser measurements behind the live design (the timings above, the memory of the point-trap plan and the GPU buffers) are quoted from the product's record of measured resources, and each states who measured it and on what.

6 Methods

The methods are described in the order of the design document's ladder: the published methods, from the superposition of prisms and lenses to the volumetric form of Gerchberg–Saxton, and then the methods proposed here, from the device-aware weighted GS to the efficiency–uniformity Pareto front; each subsection names its method. Every method is a module with a specification and a run function behind the interface of Section 5.2: it takes the optical setup, a target of traps or of a shaped field, an optional device, a budget, the portable random generator and an optional starting phase, and returns the phase, a history of the metrics, the trap fields and a metadata record. The forward model of Eqs. (1) and (2) and the metrics of Eq. (5) are shared by all of them, and the scorer recomputes the scores from the returned phase, so that no method grades itself. Each subsection gives the role of the method and its source, the equations, the parameters with their values and how they were chosen, the computational complexity, the lane, the acceptance criterion with the gate that measures it and the measurement of this report, and the implementation module. Every method is a module of the engine of version 0.02.002, the version of the product this report describes, and the module is named at the end of its subsection. The three closed-form methods (the superposition, the random superposition and the random mask) are given by their equations; every other method that runs as a procedure has its pseudocode, transcribed from the module (Algorithms 1 to 18), the plain, adaptive and weighted forms of Gerchberg–Saxton sharing one algorithm with the coefficient rule as a parameter. The measured text of each method is generated by the acceptance script and not typed here. The keyword defaults are read from the signature of each run function and are listed in Table 16; the acceptance criteria are in Table 19.

6.1 Superposition of prisms and lenses

The superposition of prisms and lenses is the oldest and cheapest hologram for a set of point traps, and it is the zero point of the ladder against which every other method is measured. A single trap at (x_m, y_m, z_m) with vortex charge l_m is formed by the delay Δ_j^m of Eq. (1): a prism for the lateral offset, a lens for the axial offset and a spiral for the charge. The hologram of M traps is the phase of the sum of the single-trap fields,

$$\phi_j = \arg \sum_{m=1}^{M} \sqrt{w_m}\, e^{\,i\Delta_j^m}, \tag{10}$$

with w_m the target weight of trap m (all equal to one in the source table of Di Leonardo et al. [12], who call this row "S"; the same form appears in Bianchi and Di Leonardo [13]). The sum is the adjoint of the point-trap projection of Eq. (2) applied to amplitudes $\sqrt{w_m}$ with all trap phases equal to zero, so Eq. (10) is the first half of one Gerchberg–Saxton step started from a flat focal phase. Because only the argument of the sum is kept, the amplitude information of the sum is thrown away, and the light that the discarded amplitude would have carried goes to the zero order and to the interference orders of the lattice.

Assumptions and parameters. The method accepts point targets only, in two or three dimensions, and refuses a shaped field with an error that names the supported kind. It has no tunable parameter, uses no random number, ignores the device and the starting phase, and does not consume its iteration budget, so its output is identical for every seed. The module evaluates the sum with the point-trap back-projection of Eq. (2) and then recomputes the trap fields; the result carries a one-entry history, so that the common interface of Section 6 holds.

Complexity. The cost is $N_{\mathrm{pix}} \times M$ complex terms and no iteration, the entry of the row "S" of Table 1 of Di Leonardo et al. [12]. The method runs in the browser worker, and its TypeScript port agrees with the Python fixtures to the tolerance of the parity tests of Section 9. The benchmark protocol runs it as a one-pass method.

Acceptance criterion and gate. The criterion fixes the zero point: on the 10×10 task with a flat beam the efficiency is within a stated band of the published value and the uniformity stays below a stated bound (Table 19). The gate runs the method on the 10×10 task of Section 7 and asserts the two bounds; companion tests check that the output is deterministic and independent of the seed, that a single-trap hologram is a grating with unit field, and that a shaped target is refused. The measurement taken by the script of this report is: efficiency 0.281, uniformity 0.0003 (deterministic). The published row is an efficiency of 0.29 and a uniformity of 0.01 on a larger grid [12], so the engine reproduces the published low efficiency and the near-absence of uniformity that fix the zero point; the uniformity of the engine is below the published 0.01, a difference at the second decimal place, because the grid has fewer pixels.

Reading the bake. On the committed cases the method gives its worst scores on the symmetric lattices and its best on low-symmetry sets: Table 23 lists a relative standard deviation of several hundred percent on the lattices of cases C02, C03 and C12, and a spread of a few tens of percent on the ring-and-line case C06 and the disordered array C13, where there is no regular interference to concentrate the light. The reading is the one Di Leonardo et al. give: the superposition sends energy to ghost traps on symmetric layouts. It is therefore a baseline and not a method for arrays, and it is kept in the studio because its failure is the reference that makes the gain of every other method measurable.

Why the light is lost. Write $S_j = \sum_k \sqrt{w_k}\, e^{i\Delta_j^k}$ for the sum inside Eq. (10). The field that the kept phase $\phi_j = \arg S_j$ gives at trap m is Eq. (2) evaluated at that phase,

$$V_m = \frac{1}{N_{\mathrm{pix}}} \sum_j A_j \, \frac{S_j}{|S_j|} \, e^{-i\Delta_j^m}. \tag{11}$$

The term $k = m$ of S_j contributes a positive real amount to V_m, the average over the pixels of $A_j\sqrt{w_m}/|S_j|$, and every term with $k \neq m$ contributes a cross term whose phase varies with the pixel as $\Delta_j^k - \Delta_j^m$. On a lattice these cross terms are gratings in their own right, and they add coherently at orders of the lattice that are not traps, which is the mechanism by which the method sends energy to ghost traps for symmetric layouts. Because the sum is divided by its own modulus, the amplitude information of the single-trap fields is not used to balance the traps, so the method has no handle on the intensity of any one of them.

Implementation. The module `m01_superposition` of the package `pipeline.model.holo.methods.baseline` holds the specification and the run function; the shared helper that refuses unsupported target kinds lives in the module `_common` of the same package. The tests are in `tests/methods/baseline/`.

6.2 Random superposition

Random superposition repairs the main defect of the plain superposition with one change: each trap receives a random phase before the sum. The random phases break the regular interference between the prisms, and the efficiency rises from a fraction of the input power to about three quarters of it. The hologram is

$$\phi_j = \arg \sum_{m=1}^{M} \sqrt{w_m}\, e^{\,i(\Delta_j^m + \theta_m)}, \qquad \theta_m \sim \mathcal{U}[0, 2\pi), \tag{12}$$

the row "SR" of Table 1 of Di Leonardo et al. [12], drawn once per trap. Every random number comes from the portable generator of the engine (Section 5), through one call that returns M phases, so the result is a pure function of the seed and the TypeScript port reproduces the generator bit for bit. The function that draws the phases and back-projects the amplitudes $\sqrt{w_m}e^{i\theta_m}$ is not only the method itself: it is the default starting phase of the iterative methods of the ladder, namely Gerchberg–Saxton and its adaptive and weighted forms, the phase-fixed weighted form, the direct search, the closed-loop method, the multi-plane

method and the proposed methods that start from a weighted Gerchberg–Saxton iterate. Its quality therefore bounds the quality of the starts of most of the report.

Assumptions. As for the plain superposition, the target must be a set of points and the device is ignored. The efficiency of Eq. (12) is high because the phases of different traps add with random signs, which makes the amplitude of the sum close to the square root of the number of traps times the single-trap amplitude, so that the phase of the sum retains most of the light; the same randomness is the reason why the trap intensities are unequal. For M traps with independent phases the trap amplitudes scatter by an amount that does not shrink with the number of pixels, which the measurement below confirms on two grid sizes.

Complexity and parameters. The cost is $N_{\mathrm{pix}} \times M$ and no iteration; there is no parameter. On a graphics processor Bianchi and Di Leonardo report a cost per trap that DinHot does not rely on [13].

Acceptance criterion and gate. The criterion requires an efficiency above that of the plain superposition on the 10×10 task and a uniformity below a stated bound; the published uniformity of 0.01 is not the bar, because on this task the intensity spread of random superposition is intrinsic to the method (Table 19). The gate runs six seeds on the live grid and asserts the efficiency band per seed and the medians; a second test asserts that the efficiency exceeds that of the plain superposition by a stated margin. The measurement of this report is: efficiency 0.750 [0.738, 0.752], uniformity 0.252 [0.106, 0.272]. The efficiency clause holds, the published uniformity is not reproduced, and the acceptance bound is set at the measured ordering. The spread is about the same whatever the number of pixels (grids of two sizes were tried), so the published relative standard deviation is not reached on this task; the ordering of the methods, not the value, transfers.

Reading the bake. Table 23 shows that the quality of a random start depends on the geometry: the low-symmetry sets (the ring-and-line case C06 and the disordered array C13) score a much lower spread than the lattices (C02, C03, C12), and the efficiency of every case stays in a narrow band (Table 24). No intensity control is available, so the method is a baseline and a start, never a recommended hologram.

Why the efficiency is high. Over the random phases the cross terms of the sum in Eq. (12) average to zero, so that for $S_j = \sum_m \sqrt{w_m}\, e^{i(\Delta_j^m + \theta_m)}$

$$\mathbb{E}_\theta \, |S_j|^2 = \sum_m w_m, \tag{13}$$

independent of the pixel and of the geometry. The modulus of the sum is therefore nearly constant over the SLM, the division by it in the phase-only projection costs little amplitude, and most of the light is kept at the traps; with the regular sum of the plain superposition the modulus fluctuates strongly between pixels, and the projection discards the light. The same randomness is the reason the trap intensities scatter, because each trap's field is a random sum of the contributions of the others.

Implementation. The module `m02_random_superposition` and the start function of `_common`; the default start of each iterative method calls the same function unless an initial phase is supplied.

6.3 Random mask

The random mask encodes equal brightness by construction and pays for it in efficiency. The SLM pixels are partitioned into disjoint random subdomains I_k, one per trap, of equal share N_{pix}/M (a share proportional to the weight when the weights differ), and each pixel shows the phase of its own trap alone:

$$\phi_j = \Delta_j^{m(j)}, \qquad I_l \cap I_m = \emptyset, \qquad \bigcup_k I_k = \mathrm{SLM}. \tag{14}$$

The assignment rule is that of Montes-Usategui, Pleguezuelos, Andilla and Martin-Badosa [14], and the benchmark numbers are those of Table 1 of Di Leonardo et al. [12]. Each subdomain acts as a separate sparse grating that sends its light to its own trap, so that no trap competes with another for amplitude; the price is that a given trap receives only the light that falls on its share of the pixels, about $1/M$ of the input power, and that the remaining light of each sub-grating is diffracted into the other orders of the focal plane.

Construction. The partition is a random permutation of the pixels cut at the cumulative weight edges, so that the shares are exact and not binomial; the independent per-pixel draw of Di Leonardo et al. is the same method with a binomial share. A trap whose weight is so small that its share rounds to zero is skipped. The delay of each pixel is built from the grating, lens and vortex terms of the delay of Eq. (1), evaluated only on the pixels the trap owns.

Physical bound. A second display relation gives the bound that the SDD records as the oracle of the

method: with M equal traps the efficiency of the random mask is about

$$e \approx \frac{1}{M}, \qquad u \text{ depends on the number of pixels per trap } \frac{N_{\mathrm{pix}}}{M}, \tag{15}$$

and the uniformity is controlled by the pixels per trap because the interference of the sub-gratings of other traps behaves as speckle whose relative amplitude falls with the share.

Acceptance criterion and gate. The ladder row states an efficiency near $1/M$ and, on the grid of the source, a uniformity within the published spread (Table 19). The gate has two parts: five seeds on the live grid assert that the efficiency lies within a factor of two of $1/M$ and that the uniformity is in a band, and two seeds on the published grid of 768×768 pixels assert that the published uniformity is reproduced within a stated tolerance and that the efficiency is in the published range. The measurement of this report is: 256 squared: efficiency 0.010, uniformity 0.057 [0.031, 0.072]; 768 squared: efficiency 0.008, uniformity 0.566 [0.543, 0.590]. The published numbers are reproduced on the grid on which they were measured; on the live grid there are nine times fewer pixels per trap, and the uniformity falls, which is the pixels-per-trap dependence of Eq. (15). The larger grid validates the implementation of the encoding; the live grid shows the behaviour a reader of the studio sees.

Reading the bake. In the committed cases the efficiency is about the inverse of the trap count whatever the geometry: the nine-trap case C01 reaches an efficiency near one tenth, the hundred-trap cases near one hundredth (Table 24). The method is only well behaved on large SLMs, and it is the counter-example to the superposition: equal traps at a low, known efficiency.

Why the light that remains is equal. Each trap's own sub-grating is exact on its pixels, so the field it forms at its own trap is the same for every trap of equal share, and the only trap-to-trap difference is the interference of the other sub-gratings at that location. Those contributions are sums of many terms with unrelated phases, and their relative size falls as the number of pixels per trap rises, which is why the published grid, with nine times more pixels per trap than the live grid, gives the higher uniformity. A trap with a larger weight gets a proportionally larger share of pixels, so a requested intensity ratio is honoured to within the binomial or exact share, and the exact partition used here removes the random fluctuation of the share that the independent per-pixel draw would add.

Complexity and implementation. The cost is linear in the number of pixels for the permutation plus linear per trap to build its delay on its own pixels. The module is `m03_random_mask`; it returns, besides the phase, the integer map of the trap that owns each pixel, which the figures of the studio use to show the partition.

6.4 Gerchberg–Saxton, point-trap and Fourier forms

Gerchberg–Saxton (GS) alternates between the SLM plane and the trap plane, imposes the known amplitude in each and keeps the phase that comes back. It is the workhorse that the weighted variants modify, the reference of the learned and optimization methods, and, with an empty noise region, the baseline that mixed-region amplitude freedom improves for shaped targets. With the trap fields V_m of Eq. (2), target amplitudes $a_m = \sqrt{w_m}$ and the delay of Eq. (1), iteration k is

$$\phi_j^k = \arg \sum_{m=1}^{M} a_m \, \frac{V_m^{k-1}}{|V_m^{k-1}|} \, e^{+i\Delta_j^m}, \tag{16}$$

where the phasor $V/|V|$ is set to one where the field vanishes, a convention of this engine that Di Leonardo et al. [12] do not state. The start is the random superposition of random superposition unless an initial phase is given.

Two forms of one map. The point form is the direct sum of Eq. (2). The Fourier form places the constraint on the bins of the zero-padded transform,

$$\phi \leftarrow \arg \mathcal{F}^{-1}\Big[\sum_m c_m \, \delta(\mathbf{u} - \mathbf{u}_m) \Big], \qquad c_m = a_m \frac{V_m}{|V_m|}, \tag{17}$$

and requires every trap to lie on a bin, at $z = 0$ and without a vortex. For traps on bins the two forms are the same linear map up to a scale, so they agree to rounding; an automatic mode takes the Fourier form when the conditions hold and the point form otherwise, and records the choice. For shaped targets the field form iterates the same projection with the mixed-region rule of Section 6.8 at unit mixing, that is, with a zero noise region, after resetting the SLM amplitude to the beam and rescaling the target to the input power.

Algorithm 1 The Gerchberg–Saxton family in the trap form, with the coefficient rule as a parameter: plain GS, the adaptive form and weighted GS (the engine loop of the module `_gs`); the point form back-projects with Eq. (1) and the Fourier form with the focal deltas of Eq. (17).

Require: traps with target amplitudes $a_m = \sqrt{w_m}$, rule $\in$ {GS, GAA, GSW}, weight α (GAA), iteration budget K, start ϕ^0

$\phi \leftarrow \phi^0$ (the random superposition when no start is given); $V \leftarrow$ trap fields of ϕ
$w_m \leftarrow a_m$ for GSW, $w_m \leftarrow 1$ otherwise
for $k = 1, \ldots, K$ **do**
  if rule is GS **then**
    $c_m \leftarrow a_m \, V_m / |V_m|$
  else if rule is GAA **then**
    $r_m \leftarrow |V_m| / a_m$; $\tilde{r}_m \leftarrow r_m / \langle r \rangle$; $c_m \leftarrow a_m \, (V_m/|V_m|) \, [(1-\alpha) + \alpha/\tilde{r}_m]$
  else
    if $k > 1$ **then** $r_m \leftarrow |V_m|/a_m$; $w_m \leftarrow w_m \, \langle r \rangle / r_m$ **end if**
    $c_m \leftarrow w_m \, V_m / |V_m|$
  end if
  $\phi \leftarrow \arg \sum_m c_m \, e^{+i\Delta^m}$ (point form), or $\arg \mathcal{F}^{-1}[\sum_m c_m \delta(\mathbf{u} - \mathbf{u}_m)]$ (Fourier form)
  $V \leftarrow$ trap fields of ϕ; record the history entry of iteration k
end for
return ϕ

Parameters. The form is chosen automatically for traps and the Fourier form for shaped fields (Table 16); the start of a shaped target is a Fresnel-lens phase of 20 waves at the aperture edge, because a random start leaves speckle (the original of Gerchberg and Saxton starts from uniform random phases, pp. 238–239 of Optik 35, 237, 1972, so the Fresnel-lens start is DinHot's choice and not the source's); a trap is accepted as on a bin within a tolerance of two thousandths of a bin, because case coordinates are rounded to a ten-thousandth of a micrometre.

Pseudocode. Algorithm 1 lists the iteration of plain GS, of the adaptive form (Section 6.5) and of weighted GS (Section 6.6) as the engine runs them: one loop, three coefficient rules; the vortex method of Section 6.15 is the weighted rule in the point form with charged traps. The weighted rule updates the weights only from the second iteration.

Complexity. The point form costs $K \times M \times N_{\mathrm{pix}}$ for K iterations [12]; the Fourier form costs $O(P \log P)$ per iteration with $P = n_{\mathrm{fft}}^2$, which is the lane in which the browser runs the method.

Acceptance criterion and gate. The ladder row has two parts (Table 19): the two forms agree to rounding, and 30 iterations on the 10×10 task with a flat beam reach an efficiency of at least 0.9 and a uniformity near 0.6. A Gaussian input beam lowers the single-trap reference level to $(\overline{A})^2$, the square of the mean amplitude (Section 2.2), so the bar of 0.9 holds for a flat beam; the Gaussian test therefore asserts 0.9 of that reference level, a threshold relative to the single-trap limit and not to a proven ceiling of the array. The measurement of this report is: flat beam: efficiency 0.939 [0.933, 0.939], uniformity 0.604 [0.557, 0.626]; Gaussian beam: efficiency 0.815 against its ceiling 0.869. The published values are an efficiency of 0.94 and a uniformity of 0.60 after 30 iterations [12], which the flat-beam run reproduces.

Role in the ladder and in the bake. Gerchberg–Saxton is the method against which most others are measured, and the bake uses it at the same budget as every other iterative method, so that its row of Table 23 is the reference column. The spread it leaves is nearly the same on every lattice case of the bake and differs mostly on the low-symmetry sets, because the iteration imposes the target amplitudes in the trap plane and keeps only the phase that comes back, so that it has no mechanism that drives the achieved intensities to the target ones: the spread is set by the algorithm and the start more than by the geometry. The default form of the Python engine for traps is the automatic choice, while the browser port uses the point form unless told otherwise; the two are identical on bins, and the port does not choose the Fourier form on its own for this method. The input and output conventions are those of the common interface: a finite starting phase of the shape of the SLM may replace the random start, which is how the weighted variants and the refinement stages of the hybrid methods continue from a previous phase.

Limits. The uniformity stalls near 0.6 because the iteration does not control the trap intensities; the adaptive, weighted and phase-fixed variants do. The method stagnates in local minima, sends light to ghost traps, and leaves speckle in a shaped target, which is the reason for the mixed-region method. Table 23 shows a nearly constant spread over the lattices of the bake (a median of 17 percent over the cases it ran

on), which is the family trend. The module is `m04_gs`, on the engine of `_gs` (the rule, the trap loop and the field loop).

6.5 Generalized adaptive additive GS

The generalized adaptive additive form (GAA) is the first published remedy for the poor uniformity of GS, and the rung between GS and the weighted form. It adds one feedback: the field of each trap is divided by a power of its own amplitude before it is sent back, so that a weak trap is boosted and a strong one is damped. Di Leonardo et al. [12] maximize $(1-\alpha)\sum_m |V_m| + \alpha \sum_m \log |V_m|$; the stationarity condition, iterated, gives

$$\phi_j = \arg \sum_m e^{+i\Delta_j^m} \, \frac{V_m}{|V_m|} \Big[(1-\alpha) + \frac{\alpha}{|V_m|} \Big], \tag{18}$$

and for $\alpha = 0$ the iteration is GS (a test pins this). With target weights the phasor is scaled by a_m and $|V_m|$ is replaced by $r_m = |V_m|/a_m$.

A stated deviation. The bracket of Eq. (18), as printed, depends on the absolute scale of $|V_m|$, which is a fraction of the input power and of the order of a tenth on the 10×10 task; the term $\alpha/|V_m|$ then dominates and the iteration oscillates, with a uniformity below that of GS. DinHot expresses $|V_m|$ in units of its mean over the traps,

$$c_m = a_m \frac{V_m}{|V_m|} \Big[(1-\alpha) + \frac{\alpha}{\tilde{r}_m} \Big], \qquad \tilde{r}_m = \frac{r_m}{\langle r \rangle}, \tag{19}$$

which makes the iteration independent of the input power. The bracket of Eq. (18) is the update of Di Leonardo et al. [12] as printed (their Eq. 16, with ξ for the weight written α here, and $\xi = 0.5$ their value); with V_m normalized by the number of pixels as in their Eq. 3, $|V_m|$ is of order 0.1 for 100 traps. The source states no rescaling of $|V_m|$ and reports, for $\xi = 0.5$, a uniformity of 0.79 at the efficiency of GS; with the printed bracket the iteration measured here oscillates and falls below GS in uniformity, a difference the source does not explain. The division by the mean is therefore a modelling choice of this product, and the bracket as printed is kept as an option and is measured to fail.

Parameters. The weight α has the default 0.5, the value of Di Leonardo et al. (an efficiency of 0.93 and a uniformity of 0.79 at 30 iterations in the source); the scale convention has the default `mean`; the form is chosen as for plain GS. The benchmark tunes α over a grid of four values stored in the protocol (Section 7).

With α at its two limits the method is easy to read: at $\alpha = 0$ it is GS, and as α grows toward one the logarithmic term dominates, so that the update acts on the ratio of the amplitude to its own mean and the feedback on a weak trap is strongest, at the price of a larger risk of oscillation when the amplitudes change from one iteration to the next. The value of the source sits between the two, and the measurement of this report is at that value.

Complexity. The cost is the same as GS, $K \times M \times N_{\mathrm{pix}}$ for the point form, with no extra state: the method has no integral memory of earlier corrections, which is what the weights of weighted GS add.

Acceptance criterion and gate. The criterion is a uniformity above that of GS at equal iterations on the 10×10 task (Table 19). The gate runs five seeds of 30 iterations for both methods and asserts, per seed, a higher uniformity for GAA, a median gain of at least a stated amount, and an efficiency within five percent of GS. The measurement of this report is: uniformity 0.688 [0.638, 0.709] against 0.615 [0.550, 0.637] for GS at equal iterations; efficiency 0.811 against 0.816. GAA is above GS in every seed; the efficiency stays within a small fraction of GS; with the printed bracket the uniformity falls below GS, which is why the test fixes the reading of the formula.

Why the bracket helps. Differentiating the objective $(1-\alpha)\sum_m |V_m| + \alpha \sum_m \log |V_m|$ with respect to the phase and keeping the stationarity condition gives the weight $(1-\alpha) + \alpha/|V_m|$ on the phasor of trap m. The first term is the plain GS weight; the second is inversely proportional to the amplitude already achieved, so a trap that is weak receives a larger coefficient in the back-projection, and a trap that is strong is relatively damped. The logarithm in the objective is what makes the correction multiplicative in the amplitude, and it is also the reason for the scale problem: $\log |V_m|$ is defined up to the unit of $|V_m|$, and a change of that unit moves the relative weight of the two terms, which the printed bracket does not fix. Expressing $|V_m|$ in units of its mean over the traps, as in Eq. (19), removes that ambiguity, and it leaves the update invariant to the overall power of the input beam.

Limits. The scale convention is DinHot's, not the source's. GAA stays below the weighted methods weighted GS and phase-fixed weighted GS in uniformity: on the lattices of the bake its spread is higher than that

of weighted GS (Table 23), as its missing integral memory predicts. The module is `m05_gaa`, on the same engine as plain GS.

6.6 Weighted Gerchberg–Saxton

Weighted GS (GSW) puts an integral feedback on the trap weights, so that dark traps are boosted and bright traps damped at every iteration. It is the method of Di Leonardo et al. [12] that reaches high efficiency and high uniformity together on the 10×10 task, the best row of the ladder among the classic methods for point arrays, and the weighted base of the phase-fixed, closed-loop, Zernike and vortex methods and of the starts of several optimizers. The maximum condition of the weighted objective is $\phi_j = \arg[\sum_m e^{i\Delta_j^m} w_m V_m/|V_m|]$, and the iteration, started from the random superposition with $w_m^0 = 1$, is

$$w_m^1 = w_m^0, \qquad w_m^k = w_m^{k-1} \frac{\langle |V^{k-1}| \rangle}{|V_m^{k-1}|} \quad (k \geq 2), \qquad \phi_j^k = \arg \sum_m e^{+i\Delta_j^m} w_m^k \frac{V_m^{k-1}}{|V_m^{k-1}|}, \tag{20}$$

where V^{k-1} is the field of the hologram of the previous iteration and V^0 that of the start. The first iteration therefore projects the field of the start hologram with the initial weights and is plain GS, and the first weight update uses the field of the second iteration. This is a convention of DinHot's implementation, not a statement of the source: it follows the weighted GS of the open library slmsuite (no update at its iteration 0), to which the engine is equal to four digits from the same start on case C03 (1024 traps, 30 iterations, 4 seeds; the engine test `test_m06_against_slmsuite`), and it is one iteration later than a recursion that updates the weights from the first field as well. The numerator of the weight update is the average of the trap amplitudes over the traps, $\langle |V| \rangle = \frac{1}{M} \sum_m |V_m|$, as in the flowchart of Bianchi and Di Leonardo [13], whose weight-update box reads $w_m \leftarrow w_m \langle |V_m| \rangle / |V_m|$; at convergence $|V_m| = \langle |V_m| \rangle$ and the weights no longer change. With target weights, $a_m = \sqrt{w_m^{\text{target}}}$, the initial weight is $w_m^0 = a_m$, and $|V_m|$ is replaced by $r_m = |V_m|/a_m$; the update is then the multiplication of the weight by the ratio of the mean of r to r_m, which is identical to Eq. (20) for equal weights. The history of the method reports the uniformity and the relative standard deviation of $I_m / w_m^{\text{target}}$, so that a requested intensity ratio is the quantity equalized.

Why it works. The weights are a running product of the corrections applied so far, so a trap that stays dark keeps being boosted, as an integral controller removes a constant error; GAA has only the proportional term. The update has the fixed point $r_m = \langle r \rangle$ for every trap, which is uniform intensity. The memory also explains the limit of the method on a crowded array, where later corrections are applied to a phase that keeps moving: this is the problem that freezing the phase solves in phase-fixed weighted GS.

Taking logarithms of the weight update shows the integral action explicitly,

$$\ln w_m^k = \ln w_m^0 + \sum_{i=1}^{k-1} \Big(\ln \langle r^i \rangle - \ln r_m^i \Big), \qquad r_m^i = \frac{|V_m^i|}{a_m}, \tag{21}$$

so that the log-weight of a trap is the running sum of the log-errors of that trap, exactly the structure of an integral controller acting on the quantity $\ln r_m$, whose reference is the mean over the traps. A trap that is persistently below the mean accumulates a positive sum and its weight grows until its amplitude catches up; the accumulated weights are never reset, which is both the strength of the method (it removes a constant bias) and its weakness when the phase keeps changing under the weights.

Parameters. Only the form (default `auto`) and the initial weights, set by the target weights; the benchmark tunes nothing for weighted GS.

Complexity. $K \times M \times N_{\text{pix}}$ for the point form and $O(P \log P)$ per iteration for the Fourier form. The source reports a graphics-processor cost per trap and iteration (0.44 ms on a GeForce GTX 260, its Fig. 5) [13]; the studio relies on its own measurement in the browser (Section 5.5).

Acceptance criterion and gate. The ladder row requires an efficiency and a uniformity both at least 0.9 within a median of 15 iterations on the 10×10 task with a flat beam (the source reports about 10), and the $3 \times 3 \times 3$ lattice above 0.9 and 0.98 (Table 19). The gate asserts for every seed that the first iteration with both quantities at 0.9 is at most 15, that the median uniformity at iteration 10 is at least 0.85, that the Gaussian beam reaches 0.9 of its ceiling and 0.9 in uniformity within 20 iterations, that the lattice reaches its two thresholds within 100 iterations, and that a requested 2:1 weight gives an intensity ratio of 2 within a tenth. The measurement of this report is: flat beam: efficiency and uniformity both at 0.9 after 12, 15 and 12 iterations (median uniformity 0.857 at 10); Gaussian beam: 12, 15 and 12; 3 x 3 x 3 lattice (0.9 and 0.98): 35, 20 and 27; budgets 15, 20 and 100. Convergence is slightly slower on the live grid than in the

source because the grid has fewer pixels, and the bound is set accordingly; the lattice does not converge monotonically.

Limits. The efficiency of a Gaussian beam lies below the flat-beam value, by the scale of the single-trap reference level of Eq. (6). On a device with crosstalk and few phase levels the weights equalize the ideal model and not the device: on the device-limited case C08 the bake leaves a relative standard deviation of the order of thirteen to fifteen percent for weighted GS and phase-fixed weighted GS, which is the gap that the device-aware methods of this report address (Table 23). The module is `m06_gsw`, on the engine of `_gs`.

6.7 Phase-fixed weighted GS

Phase-fixed weighted GS is the practical method for large arrays. The solution replicated is that of Kim, Keesling, Omran, Levine, Bernien, Greiner, Lukin and Englund [16]: after an initial number N of weighted-GS iterations the focal-plane phase is frozen, so that later iterations adjust amplitudes only and the weights stay effective. The same procedure is used in the 256-atom programmable quantum simulator of Ebadi et al. [17] and, through the references of Manetsch et al. [1], at the scale of thousands of sites. The source reports a modulation efficiency of 91.2 percent and a non-uniformity below 0.5 percent on 50×30 foci, and a camera correction from 22 percent to 1.59 percent in three steps against 6.47 percent for conventional weighted GS.

Derivation. At iteration i let $B_i(\mathbf{u}_m) = |V_m|/a_m$ be the focal amplitude and $T_m = a_m$ the target. With $\langle B_i \rangle_N$ the mean over the traps, the weights and the SLM field follow

$$g_i(\mathbf{u}_m) = \frac{\langle B_i \rangle_N}{B_i(\mathbf{u}_m)}\, g_{i-1}(\mathbf{u}_m), \quad g_0 = 1, \qquad \mathcal{F}\big[A_{i+1} e^{i\phi_{i+1}}\big] = g_i\, T\, e^{i\psi_i}, \tag{22}$$

where the focal phase follows the field, $\psi_i = \arg V_m$, for the first N iterations and is then frozen at its value ψ_N,

$$\psi_i = \psi_N \quad (i \geq N). \tag{23}$$

In the engine the first iteration projects the field of the start hologram with $g = 1$ and the first weight update uses the field of the second iteration, so that Eq. (22) holds from $i \geq 2$ with $g_1 = g_0$. In the recursion of Kim et al. [16] the weight g_i, built from the focal amplitude B_i of the i-th iteration, produces the hologram of the $(i+1)$-th, with $g_0 = 1$ (preprint arXiv:1903.09286, page 2). The text read does not say whether $g_0 = 1$ is applied to the field of the start hologram, as the engine does, or whether the update begins at the first measured field, which would be one iteration earlier than the engine; the preprint text available to this report does not settle the indexing of the first iteration. The engine follows the phase-fixed weighted GS of the open library slmsuite (no update at its iteration 0); the convention is DinHot's choice and not a verified statement of the source. The weights g keep updating after the freeze and carry the memory of all earlier corrections. The reason given by the source is that replacing the SLM-plane amplitude by the laser amplitude introduces a change $\delta\psi_i$ of the focal phase, which makes g_i less effective; fixing ψ removes the change. The sources do not define B_i (it is taken as the field amplitude, as in weighted GS) nor say whether the N-th iteration already uses the frozen phase; here it computes the phase, and a one-iteration shift turns the parity test of the port red, which is how the choice is pinned. The source states the weight update and the freezing of the focal phase in its section on the calculation of the hologram and in its Fig. 1 (Kim et al. [16]); both are displayed without an equation number in the preprint, so they are cited here by section and figure.

Pseudocode. Algorithm 2 lists the procedure as implemented in the Fourier form.

Parameters and how they were chosen. The freeze index has the default 12, the value of Kim et al., pinned by a test; the acceptance criterion is run with $N = 60$, which was measured on the 50×30 task of DinHot's grid, and the default is not grid dependent (whether it should become grid dependent is not settled). The measured finding is that the literal setting does not transfer to a smaller SLM: with the source value the spread stalls on the crowded array, and larger values of N remove the stall (Section 8). A replication of a published setting that does not transfer is itself a result and is kept. Since the corrections of the update rule, the method matches the weighted GS with the phase freeze of the open library slmsuite to a small fraction of a percent in the final spread from the same start; this holds with the convention above, in which the first iteration is plain GS; the engine before that convention reweighted the field of the start hologram.

Complexity. $O(P \log P)$ per iteration in the Fourier form with $P = n_{\mathrm{fft}}^2$ and $O(N_{\mathrm{pix}} M)$ in the point form; the method is two-dimensional only because the freeze is defined for a plane. The WebGPU backend covers this method with the traps on bins and the ideal device (Section 5.5).

Acceptance criterion and gate. The ladder row requires fewer iterations than weighted GS to reach a relative standard deviation of 0.5 percent on a 50×30 array with the phase fixed after the measured

Algorithm 2 Phase-fixed weighted GS in the Fourier form. The weights g are updated at every iteration but the first, which projects the field of the start hologram with $g = 1$; the focal phase ψ is recomputed until iteration N and then kept.

Require: traps $\{\mathbf{u}_m\}$ on FFT bins, target amplitudes a_m, freeze index N, iteration budget K, start ϕ^0
$g_m \leftarrow 1$ for all m; $\phi \leftarrow \phi^0$ (random superposition when no start is given)
for $k = 1, \dots, K$ **do**
  $V \leftarrow \mathcal{F}[Ae^{i\phi}]$ sampled at the trap bins; $b_m \leftarrow |V_m|/a_m$
  if $k > 1$ **then**
    $g_m \leftarrow g_m \langle b \rangle / b_m$
  end if
  if $k < N$ **then**
    $\psi_m \leftarrow \arg V_m$
  else if $k = N$ **then**
    $\psi_m \leftarrow \arg V_m$; store ψ
  end if
  $c_m \leftarrow g_m\, a_m\, e^{i\psi_m}$ (with the stored ψ for $k > N$)
  $\phi \leftarrow \arg \mathcal{F}^{-1}\big[\sum_m c_m \delta(\mathbf{u} - \mathbf{u}_m)\big]$
end for
return ϕ

index (Table 19). The gate runs two seeds of 150 iterations in the Fourier form and asserts that phase-fixed weighted GS reaches the target and that weighted GS either never does or needs more iterations, with a final efficiency above 0.6; a second test pins the stall with the source value. The measurement of this report is: 50 x 30 array to 0.5 percent in 64 and 64 iterations against 136 and 106 for weighted GS (phase fixed after 60; efficiency 0.83 and 0.83); with the source value of 12 it stalls at 3.58 percent. A sweep of the freeze index is also run by the script of this report (the key `n_fix_sweep_min_rel_std_150` of the acceptance file; Table 20).

Limits. The index depends on the grid; on a device with crosstalk and four phase levels the method keeps the spread of the ideal-model optimum, which is the gap that the first proposed method removes (Section 6.20). The module is `m07_phase_fixed_wgs`.

6.8 Mixed-region amplitude freedom

Mixed-region amplitude freedom (MRAF) is the method for smooth potentials, a disc, a ring or a bar, in which GS leaves speckle. The solution replicated is that of Pasienski and DeMarco [18]: the amplitude is left free outside the signal region, so that the power the shape does not need drains into a noise region and the shape comes out accurate. The same rule in power form, with $\sqrt{p}$ and $\sqrt{1-p}$ in place of m and $1-m$, is Eq. (3) of the arXiv version of Holland et al. [36], who use $p = 0.7$ for a periodic square lattice of traps. With signal region SR, noise region NR and mixing parameter m the focal constraint is

$$G^{(n)} = \Big\{ m\,T\big|_{\mathrm{SR}} + (1-m)\,\big|E^{(n)}_{\mathrm{out}}\big|\,\Big|_{\mathrm{NR}} \Big\}\, e^{\,i \arg E^{(n)}_{\mathrm{out}}}, \qquad \phi^{(n+1)} = \arg \mathcal{F}^{-1} G^{(n)}, \tag{24}$$

with the SLM amplitude reset to the beam. In the signal region the amplitude is forced to mT, and in the noise region it is kept and scaled by $1-m$; $m = 1$ is plain GS with a zero noise region, which a test pins. The target T is rescaled to the input power. The error of the history is the root-mean-square fractional error of the source,

$$\eta = \sqrt{\Big\langle \Big(\frac{s\,I - T^2}{T^2} \Big)^2 \Big\rangle}, \qquad s = \frac{\sum T^2}{\sum I}, \tag{25}$$

over the measure region. Three design decisions are DinHot's and not the source's. First, the measure region is the shape eroded by one resolution cell, because a pixel closer to the edge than the cell cannot reach its target for any hologram; without it the error floor is dominated by the edge, and a shape thinner than the erosion is refused. Second, the default start is a Fresnel-lens phase of 20 waves at the aperture edge, because a random start leaves speckle that the iteration removes slowly. Third, the offset variants of the method are not implemented, because their primary papers were not opened.

Pseudocode. Algorithm 3 lists the iteration; with $m = 1$ it is plain GS with a zero noise region.

Algorithm 3 Mixed-region amplitude freedom (the field loop of the engine). The target is rescaled to the input power, the noise region keeps the amplitude it reaches, scaled by $1 - m$.

Require: target amplitude T and signal mask SR on the focal grid, mixing $m \in (0, 1]$, start waves W, iteration budget K, start ϕ^0

$T \leftarrow T\sqrt{N_{\text{pix}} / \sum_{\text{SR}} T^2}$ on SR, 0 elsewhere; measure region $\leftarrow$ SR eroded by one resolution cell

$\phi \leftarrow \phi^0$, or the Fresnel-lens phase of W waves at the aperture edge when no start is given; $E \leftarrow \mathcal{F}[Ae^{i\phi}]$

for $k = 1, \ldots, K$ **do**

  $G \leftarrow m\,T$ on SR, $(1 - m)\,|E|$ elsewhere; $G \leftarrow G\,e^{i \arg E}$

  $\phi \leftarrow \arg \mathcal{F}^{-1}[G]$; $E \leftarrow \mathcal{F}[Ae^{i\phi}]$ (the SLM amplitude is reset to the beam)

  record the efficiency in SR and the error η of Eq. (25) over the measure region

end for

return ϕ

Parameters. The mixing has the default 0.3, which lies in the range the source reports as typical; on DinHot's grid a larger value trades a higher efficiency for a larger error, a trade measured when the default was set. The benchmark tunes the mixing over a grid of four values.

Complexity. Two Fourier transforms per iteration, $O(P \log P)$; the source reports convergence in under a hundred iterations.

Acceptance criterion and gate. On a disc, a ring and a bar, the signal-region error is at least five times below GS at the cost of efficiency (Table 19). The gate runs the three shapes on the live grid with two starts of different focal curvature (the default start is deterministic, so the starts are varied instead of seeds) and asserts a ratio of errors of at least five and a lower efficiency for both. The measurement of this report is: signal-region error 12.4 (disc), 7.8 (ring) and 10.4 (bar) times below GS at the worse of two starts; efficiency 0.181 to 0.184 against 0.963 to 0.992. A thin line is resolution limited: for a line a few bins wide the ratio falls below five because the error floor is the shape edge and not the method, so the criterion is met for shapes wider than the resolution cell, and the thin-line limit is a recorded result.

Why the noise region helps. In plain GS the whole focal plane outside the traps is constrained to zero amplitude, which forces the field to satisfy a constraint it cannot meet with a phase-only hologram, and the unsatisfiable part reappears as speckle inside the signal region. Freeing the amplitude outside the signal region lets the power that the shape does not need leave through the noise region; the mixing m sets how much of the signal amplitude is imposed against how much of the achieved field is kept in the noise region, so that a smaller m gives the noise region more freedom, a lower efficiency in the signal region and a more accurate shape. The efficiency of the method is therefore the fraction of the power that the iteration leaves in the signal region at its fixed point, which is the quantity that the history reports. A shape thinner than the resolution cell cannot be improved by any choice of m, which is the limit that the thin-line test pins.

Limits. The efficiency is low by design. The model has no device: a hologram optimized on the ideal device and scored on the crosstalk device of case C04 shows a signal error that differs by a large factor from the acceptance value, and the single factor responsible is not established, since the case differs from the test in disc radius, margin and device at once; no single-factor comparison was run. The module is `m08_mraf` and the field loop of `_gs`.

6.9 Direct search

Direct search is the optimization method of holographic optical tweezers: visit every SLM pixel in random order, try every phase level and keep the best against an explicit cost that trades efficiency for uniformity with one weight. It is the reference method for perfect uniformity in Di Leonardo et al. [12] and the explicit-cost ancestor of the gradient methods. The cost is Eq. (11) of Polin et al. [15], which Di Leonardo et al. [12] write as the gain $\langle I \rangle - f\sigma$ (their Eq. 17). With I_m the trap intensities as fractions of the input power, $I_m^D = w_m / \operatorname{mean}(w)$ the design intensities and M the trap count,

$$C = -\langle I \rangle + f_u\,\sigma, \qquad \sigma = \sqrt{\frac{1}{M}\sum_m \left(I_m - \gamma I_m^D\right)^2}, \qquad \gamma = \frac{\sum_m I_m I_m^D}{\sum_m (I_m^D)^2}, \tag{26}$$

where f_u weights uniformity against efficiency (the symbol f is the focal length elsewhere), σ is the root-mean-square residual of the design in units of the intensity and not the percentage of Eq. (5), and γ is the best common scale of the design and not the drag of Eq. (9). One sweep visits every pixel once in a seeded

Algorithm 4 Direct search with the cost of Eq. (26). One sweep visits every pixel once in a seeded random order; the cost never increases.

Require: traps with design intensities I_m^D, levels P, weight f_u, number of sweeps K, cap on the trial phases
$\phi \leftarrow$ the random superposition rounded to the P levels $2\pi p/P$; $V \leftarrow$ trap fields of ϕ
for sweep $= 1, \ldots, K$ **do**
  $C_{\mathrm{cur}} \leftarrow C(|V|^2)$ by Eq. (26); draw a random order of the N_{pix} pixels from the seeded generator
  for each pixel j in that order **do**
    stop when the cap on the trial phases would be exceeded
    for $p = 0, \ldots, P-1$: $V^{(p)} \leftarrow V + \frac{A_j}{N_{\mathrm{pix}}}\big(e^{2\pi i p/P} - e^{i\phi_j}\big)e^{-i\Delta_j^m}$ and $C^{(p)} \leftarrow C(|V^{(p)}|^2)$
    $p^* \leftarrow \arg\min_p C^{(p)}$
    if $C^{(p^*)} < C_{\mathrm{cur}}$ **then**
      $V \leftarrow V^{(p^*)}$; $\phi_j \leftarrow 2\pi p^*/P$; $C_{\mathrm{cur}} \leftarrow C^{(p^*)}$
    end if
  end for
  recompute V in full from ϕ (drops the rounding drift of the increments); record the history entry
end for
return ϕ

random order; at each pixel all P trial phases $2\pi k/P$ are evaluated and the best is kept if it lowers the cost, so the cost never increases. The trap fields are updated incrementally: a change $\delta z = e^{i\phi'} - e^{i\phi}$ at pixel j changes each field by

$$\delta V_m = \frac{A_j}{N_{\mathrm{pix}}}\,\delta z\, e^{-i\Delta_j^m}, \tag{27}$$

an update that costs $O(M)$ and holds for any trap depth and any vortex charge, because it uses the point-trap form. After each sweep the fields are recomputed in full to drop the rounding drift of the increments. The start is the random superposition rounded to the P levels, and an initial phase overrides it. The source reports convergence in about N trials and a worst case of $N\,P$.

Pseudocode. Algorithm 4 lists a run.

Parameters. The number of levels has the default 8 and the weight f_u the default 0.5, the values of the direct-search row of Di Leonardo et al.; the budget is the number of sweeps taken from the iteration budget, and a cap on the trial phases serves the time-boxed rows of the benchmark. The simulated-annealing variant of the source is not implemented, because the source gives no quantitative result for it.

Complexity and lane. One trial costs $O(M)$ and a sweep evaluates $N_{\mathrm{pix}}P$ trials, so a sweep costs $O(N_{\mathrm{pix}}PM)$; the method is sequential by construction, so it is baked and replayed and not run live. A test confirms that the incremental update agrees with a full evaluation to 10^{-10} and that the cost never rises.

Acceptance criterion and gate. The criterion requires a uniformity of at least 0.98 on the 10×10 task and a spread below five percent on a quasi-periodic set (Table 19). The quasi-periodic set is DinHot's own, a product of two Fibonacci chains with a stated gap, with two sites removed to give 119 traps, because the coordinates of the 119 traps of Polin et al. are not published. The gate asserts the uniformity and an efficiency above 0.3 (so that the uniformity is not bought by switching traps off). The measurement of this report is: uniformity 0.9994 and 0.9992 (efficiency 0.66 and 0.65); 119 quasi-periodic traps at 0.023 percent rms. The published reference is a uniformity of 1.00 and an efficiency of 0.68 for direct search on the 10×10 task [12], and a root-mean-square brightness variation above fifty percent for the adaptive-additive form on the quasi-periodic set [15], which the product does not re-run.

Why the search can stop. At every pixel the best of the P trial phases is kept only if it lowers the cost, and the current phase is among the candidates, so each accepted step is a non-increasing step of C and the sequence of sweep costs is monotone and bounded below by the minimum of Eq. (26). The search therefore converges to a coordinate-wise local minimum, a phase pattern in which no single pixel can be improved by any of the P levels, and not to a global minimum; this is the local minimum that Polin et al. report. The scale γ in the definition of σ makes the uniformity term insensitive to the overall efficiency, so that the weight f trades the two quantities and does not merely rescale one of them. The result is sensitive to the order of the visits, which the seeded permutation fixes, and to the number of levels, since a lower number of levels leaves each pixel a coarser choice.

Limits. The method is sequential, slow and stuck in local minima; the ghost traps absorb efficiency, so the efficiency in the bake is lower than that of weighted GS. The cost model is the ideal device: scored on a

Algorithm 5 Fletcher–Reeves conjugate gradient with a strong-Wolfe line search on the powered intensity cost, in stages of equal share of the iterations (the loop shared with the device-aware conjugate gradient, which differs by the forward model).

Require: target (traps or shaped field), power schedule $(t_1, \dots, t_S)$, requested efficiency η, iteration budget K, start ϕ^0

$\phi \leftarrow \phi^0$ (uniform random phases when no start is given)
for each stage $s = 1, \dots, S$ with $K_s \approx K/S$ iterations (the last stage takes the remainder) **do**
    build the cost C_{t_s} with target level η; $f \leftarrow C_{t_s}(\phi)$; $g \leftarrow \nabla C_{t_s}(\phi)$ by Eq. (28) (two Fourier transforms); $d \leftarrow -g$
    for $i = 1, \dots, K_s$ **do**
        if $d \cdot g \geq 0$ **then**
            $d \leftarrow -g$
        end if
        strong-Wolfe search along d ($c_1 = 10^{-4}$, $c_2 = 0.1$, cubic interpolation, first trial moving the largest pixel by 0.1 rad, later trials $2(f - f_{\text{old}})/(g \cdot d)$)
        if no step is found **then**
            restart with $d \leftarrow -g$ and search again; **stop** the run if that fails too
        end if
        $\phi \leftarrow \phi + a\,d$; $g' \leftarrow \nabla C_{t_s}(\phi)$; $\gamma \leftarrow (g' \cdot g')/(g \cdot g)$; $d \leftarrow -g' + \gamma d$; $g \leftarrow g'$
    end for
end for
return ϕ wrapped to $[0, 2\pi)$

device with crosstalk and four phase levels the uniformity drops, which is the reason for the device-aware methods. The module is `m09_direct_search`.

6.10 Conjugate-gradient minimization

Conjugate-gradient minimization optimizes all pixel phases at once against an intensity cost with a closed-form gradient. The solution replicated is that of Harte, Bruce, Keeling and Cassettari [19], the reference for smooth potentials with suppressed optical vortices and the parent of the device-aware variant of Section 6.13 and of the gradient family. The SLM field is $E_{\text{in}} = A_j e^{i\phi_j}$, the focal field $\tilde{\psi}$ is its zero-padded Fourier transform, and the cost over the signal region (the noise region is free) is

$$C_t = \sum_{nm} \left(T_{nm} - |\tilde{\psi}_{nm}|^2\right)^t, \qquad \frac{\partial C_t}{\partial \phi_{rs}} = 2t\ \mathrm{Re}\left(i\,\psi^*_{rs} X_{rs}\right), \tag{28}$$

$$X_{rs} = \sum_{nm} (T_{nm} - I_{nm})^{t-1}\ \tilde{\psi}^*_{nm}\, e^{+2\pi i(rn+sm)/N}. \tag{29}$$

In the code the cost is normalized by $\sum (sT)^t$, which has the same minimizers, with the target intensity carrying the requested efficiency η: for traps $T_m = \eta\, w_m / \sum w$, and for fields $T = \eta\, N_{\text{pix}} a^2 / \sum_{\text{SR}} a^2$, so that $\sum T$ is the requested fraction of the total power. The gradient costs two Fourier transforms, one forward and one adjoint, which makes the method cheap per step and lets the TypeScript port run without automatic differentiation. The basic cost $t = 2$ leaves optical vortices in the signal region; the powered cost $t = 4$ suppresses them, and the sequential $t = 4$ then $t = 2$ reaches a small error on a ring and power-law target in the source after thousands of iterations.

Optimizer. The Fletcher–Reeves nonlinear conjugate gradient builds directions $d_i = -g_i + \gamma_i d_{i-1}$ with $\gamma_i = g_i \cdot g_i / (g_{i-1} \cdot g_{i-1})$ (Harte Eqs. (3) to (5)), with a strong-Wolfe line search by cubic interpolation. The source does not state the line search; this is DinHot's choice, and a second Wolfe constant below one half keeps the Fletcher–Reeves directions descent directions. If a direction is not a descent direction or the search fails, the iteration restarts from steepest descent and the run stops if that fails too, so that the cost never increases. The start is n^2_{pix} uniform random phases from the portable generator.

Pseudocode. Algorithm 5 lists the optimizer.

Parameters. The power t has the default stages of the field case (the powered cost first, then the basic one, with an equal share of the iterations) and the basic cost for traps; the design-efficiency argument is left unset by default and the code then chooses a design level for traps and a lower one for fields (Table 16);

the Wolfe constants and the cap on line-search evaluations are numerical guards. The cost refuses an odd or non-integer power; the benchmark grid holds only the even schedules $t = 2$, $t = 4$, $(4, 2)$ and $(4, 2)$ with a design efficiency of 0.8, so no entry of the grid is refused and no benchmark row of conjugate-gradient minimization is skipped.

Acceptance criterion and gate. The analytic gradient agrees with finite differences, and a square lattice of flat-top spots reaches an error below one percent (Table 19). The gate checks the gradient for $t = 2$ and $t = 4$ on a shaped target and on traps by central differences in six directions, runs the lattice on a 64-pixel SLM padded twice with 1500 iterations and two seeds, and checks that the cost decreases monotonically. The measurement of this report is: analytic gradient within 2.5e-06 of finite differences; square lattice error 0.55 and 0.51 percent. Harte et al. [19] report 0.58 percent at 4000 iterations for the same figure.

Why the gradient costs two transforms. The cost of Eq. (28) depends on the phase only through the intensity $|\tilde{\psi}|^2$ of the focal field, which is a linear function of $e^{i\phi}$. The derivative with respect to a pixel phase therefore needs the focal field itself, one forward transform, and the sum X of Eq. (29), which is an inverse transform of the weighted residual, one adjoint transform. No derivative of the transform is evaluated numerically, and the port needs no automatic differentiation. The line search multiplies the number of cost evaluations, each a forward transform, which is why the cost of an iteration is dominated by it, and why the bake budget of a few tens of iterations leaves the method far from its limit on the trap arrays. The sequential powers for fields, the powered cost first and the basic cost after, use the second stage to remove the amplitude error that the first stage leaves while keeping the vortex-free phase that the first stage found.

Limits. The strong-Wolfe search is DinHot's choice. The model is the ideal device. The cost grows with the pixel count, so full-size runs are baked; and the bake budget of 30 iterations from a random start is far from the lattice test, so the trap-array rows of the bake are far from converged and are not a statement about the limit of the method. The module is `m10_cg` on the optimization helpers of `optim/_common`.

6.11 Wirtinger gradient descent with Adam

The hologram is treated as a differentiable program: any linear forward model (the point-trap form, the Fourier form, a device) can be optimized by a first-order method if the gradient with respect to the phase is available. The solution replicated is Wirtinger holography of Chakravarthula, Peng, Kollin, Fuchs and Heide [20], with the Adam optimizer of Kingma and Ba [21] (named in the module; the original was not re-read). It is the bridge between the iterative methods and the learned ones. The loss is minimized over the phase with a differentiable propagation P and $H = c\,e^{i\phi}$; the gradient of the real loss with respect to the complex field is the Wirtinger derivative, pulled back through P by the chain rule,

$$\frac{\partial L}{\partial \phi_j} = 2\,\mathrm{Re}\big(i\,f_j\,\overline{(P^H G)_j}\big), \qquad G = \frac{\partial L}{\partial z^*} = L'(I)\,z, \tag{30}$$

so that any linear forward model works. The loss is the quadratic intensity loss $C_2 = \sum (I_m - T_m)^2 / \sum T_m^2$ of the optimization module, with the design efficiency of conjugate-gradient minimization; the penalty term of the source has weight zero, because a phase penalty has no role for a phase-only hologram of traps. Adam updates, with bias-corrected moments,

$$m \leftarrow \beta_1 m + (1-\beta_1) g, \quad v \leftarrow \beta_2 v + (1-\beta_2) g^2, \quad x \leftarrow x - \mathrm{lr}\,\frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}. \tag{31}$$

The source uses L-BFGS for quadratic losses and Adam with a small rate for learned losses; the start is n_{pix}^2 uniform random phases.

Pseudocode. Algorithm 6 lists the loop.

Parameters and how they were chosen. The learning rate has the default 1, tuned on the trap loss: the rate of the source moves a pixel by a small angle in 30 iterations, and the tuning record of the design document (tuning seeds, then separate seeds) shows the spread falling as the rate rises up to a value near the default, where a larger rate ties within one percent and a still larger one diverges; the smaller step is kept. The moment constants are the defaults of Kingma and Ba. The benchmark's tuning grid for the rate is 0.5, 1, 2 and 4, around the default of 1. The tuned row equals the default row on three of the seven geometries, and on the others the rate chosen at 30 iterations is within a factor of about 1.4; at 100 iterations, however, the tuned rows of G1, G5 and G6 are four to eight times worse than the default (a relative standard deviation of 8.5, 9.8 and 9.0 percent against 1.2, 2.2 and 1.2), so a rate tuned for a short budget can hurt a long one. A mutation run with the rate of the source fails the acceptance test, which shows that the tuned rate is what makes the row pass.

Algorithm 6 Wirtinger gradient descent with Adam on the quadratic intensity loss; the history records the state before each step, so the cost may rise.

Require: target, learning rate lr, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, iteration budget K, start ϕ^0

$\phi \leftarrow \phi^0$ (uniform random phases when no start is given); $m \leftarrow 0$; $v \leftarrow 0$

for $i = 1, \ldots, K$ **do**

  forward model: $z \leftarrow$ field of ϕ; $L \leftarrow C_2$; adjoint pass: $g \leftarrow 2\,\mathrm{Re}\left(i\, f\, \overline{P^H G}\right)$ with $G = L'(I)\, z$ (Eq. (30)); record the state

  $m \leftarrow \beta_1 m + (1-\beta_1) g$; $v \leftarrow \beta_2 v + (1-\beta_2) g^2$; $\hat{m} \leftarrow m/(1-\beta_1^i)$; $\hat{v} \leftarrow v/(1-\beta_2^i)$

  $\phi \leftarrow \phi - \mathrm{lr}\, \hat{m}/(\sqrt{\hat{v}} + \epsilon)$

end for

return ϕ wrapped to $[0, 2\pi)$

Complexity. One forward and one adjoint evaluation per iteration, two Fourier transforms in the Fourier form, so that the cost per iteration is the same as for the conjugate gradient without its line search; the method runs both live at small sizes and baked at full size. Adam is not a descent method, so the history records the state before each step and the cost may rise. With a device the traps must lie on bins with no depth offset and no vortex.

Acceptance criterion and gate. The criterion is a lower trap-intensity error than GS at equal iterations on the 10×10 task (Table 19); the gate runs three seeds of 30 iterations and requires a lower spread per seed and in median than the in-test copy of the GS rule, which was cross-checked against plain GS and gives the same numbers to four digits. Companion tests check the gradient on a set with depth offsets, a vortex and unequal weights. The measurement of this report is: relative standard deviation 2.53, 2.46 and 2.26 percent against 17.78, 17.98 and 18.82 for GS at 30 iterations. Because the derivative is pulled back by the chain rule, the same code differentiates the point-trap form, the Fourier form and the device form; the only requirement on a forward model is that its adjoint is available, and the engine provides it for each. The tuning record is therefore a statement about one loss on one family of grids: a rate that is right for the 10×10 task is not guaranteed to be right for a crowded array or a shaped field. The method stops at a few percent in 30 iterations, since it is first order; the tuned rate is specific to this loss and these grids; and on a shaped target scored on a device the rate of the trap loss is untuned. The module is `m11_wirtinger_adam`.

6.12 Closed-loop weighted GS on an emulated camera

Camera-in-the-loop correction measures the trap intensities and rescales the target until the traps are equal. Two published procedures are replicated, on an emulation. Nogrette et al. [22] reduce a 19 percent root-mean-square trap-depth spread to 1.4 percent in 20 iterations at gain $G = 0.7$, and Kim et al. [16] reduce a 22 percent spread to 1.59 percent in three phase-fixed steps. The camera is an emulation of per-trap fixed gain error, additive noise, blur and roll-off, so a result shows that the loop works against those stated errors and not against an instrument; a measured hardware loop is a documented frontier of the product. One feedback iteration measures the trap intensities of the current hologram, rescales the target and runs a number of inner iterations from the current phase; the open-loop start is a number of iterations of weighted GS from the random start.

Update rules. With $\rho_m = (I_m/w_m)/\langle I/w \rangle$ the measured intensity relative to its target, the Nogrette variant runs plain GS inside and rescales the current target intensity,

$$w_m^T \leftarrow w_m^T \, \frac{1}{1 - G(1-\rho_m)}, \tag{32}$$

implemented on amplitudes with a floor on ρ. Nogrette et al. [22] compute, after each measurement, a new target intensity $I_i' = \bar{I}/[1 - G(1 - I_i/\bar{I})]$ from the measured trap intensities I_i alone (their Eq. 3, gain $G \simeq 0.7$) and run the Gerchberg–Saxton iteration again from the previous phase. The engine applies the same factor as a multiplier of the running target intensity, Eq. (32); this cumulative form agrees with the source at the first correction from a uniform target, and from the second correction on it keeps the product of all earlier factors, an integral action that stops changing only at $\rho = 1$, where the source recomputes the target from the latest measurement alone. Read literally, the printed rule does not close the loop: a uniform measurement returns the target to the base design, so uniformity is not its fixed point, and implemented as the variant `nogrette-printed` it ends, after 20 corrections with $G = 0.7$ on seeds 1, 2 and 3, at a spread of 15.4, 16.8 and 18.2 percent, where the cumulative form ends at 0.84, 1.07 and 2.18 percent. The source reports a fall from 19 to 1.4 percent in 20 iterations, which only the cumulative reading reproduces, so

Algorithm 7 Closed-loop weighted GS on the emulated camera, the two variants (the cumulative reading of the Nogrette rule and the square-root rule of Kim); the history records the spread of the measured intensities.

Require: traps on FFT bins, camera, variant, gain G, inner iterations n_{in}, start iterations n_0, feedback budget K

$\phi \leftarrow n_0$ iterations of weighted GS from the random superposition; $a \leftarrow a^{\mathrm{base}}$; measure the camera readings meas of ϕ
for $k = 1, \ldots, K$ **do**
  $\rho_m \leftarrow \max\{(\mathrm{meas}_m/w_m)/\langle \mathrm{meas}/w\rangle,\ 10^{-3}\}$
  if variant is Nogrette **then**
    $a_m \leftarrow a_m\sqrt{1/(1-G(1-\rho_m))}$; n_{in} iterations of plain GS on the target amplitudes a from the current phase
  else
    $a_m \leftarrow a_m/\sqrt{\rho_m}$; $g_m \leftarrow 1$; $\psi \leftarrow$ none
    for $j = 1, \ldots, n_{\mathrm{in}}$ **do**
      $V \leftarrow$ trap fields of ϕ; **if** ψ is none **then** $\psi \leftarrow \arg V$ **else** $b_m \leftarrow |V_m|/a_m$; $g_m \leftarrow g_m\langle b\rangle/b_m$ **end if**
      $\phi \leftarrow \arg \mathcal{F}^{-1}[\sum_m g_m a_m e^{i\psi_m}\delta(\mathbf{u}-\mathbf{u}_m)]$
    end for
  end if
  measure the camera readings of ϕ; record the spread of meas$/w$
end for
return ϕ

DinHot reads the measured intensity of Eq. 3 as relative to the current target; the results of this method are those of the cumulative form. The Kim variant uses the square-root weights of the source,

$$T_m^{(j)} = \sqrt{\frac{\langle I\rangle}{I_m}}\, T_m^{(j-1)}, \tag{33}$$

at full gain, and the first inner iteration displays $\arg \mathcal{F}^{-1}[Te^{i\psi}]$ with the focal phase frozen at that of the current hologram and no weight correction on top: a second correction overshoots and the loop diverges, which was measured. Later inner iterations add the phase-fixed weights of phase-fixed weighted GS, reset at every step.

Pseudocode. Algorithm 7 lists a run; the variant that reads the update as printed, without memory, replaces $a_m \leftarrow a_m\sqrt{\cdot}$ by $a_m \leftarrow a_m^{\mathrm{base}}\sqrt{\cdot}$.

Parameters. The variant, the gain (default 0.7) and the numbers of inner and start iterations are listed in Table 16. The gain is that of Nogrette et al. The inner iteration count has two measured defaults: with more than one inner GS iteration the loop settles at several percent because the algorithm's own spread is reintroduced between measurements, and for the Kim variant three inner iterations reach a fraction of a percent where one does not. The default camera has a gain error of 0.19 and no roll-off, which is Nogrette's spread.

The emulated camera. At the bin nearest to each trap the camera returns the intensity map, optionally blurred by a Gaussian and multiplied by the pixel-aperture roll-off, times a fixed gain per trap identity $\exp(sz - s^2/2)$, with s the stated gain error and z the first normal of a stream seeded from the seed and the trap identity, so that the gain of a trap does not depend on the order or composition of the trap set; additive Gaussian noise is clipped at zero. Every error is a parameter with a stated meaning. The emulator was first proposed as a method and dropped from the ladder: an emulator that reproduces published feedback curves is a validation tool for the closed-loop method, not a method that answers the problem differently.

Acceptance criterion and gate. From the published open-loop spreads the loop reaches the published curves within a factor of 1.5 (Table 19). The gate runs two seeds and asserts a final measured spread within the factor and below one eighth of the open-loop spread, a plain-GS start, the Kim variant in three steps, convergence with a noisy blurred camera, and determinism. The measurement of this report is: emulated camera loop: open-loop 20.9 and 20.4 percent to 0.84 and 1.07 percent in 20 iterations; 24.3 and 23.6 percent to 1.05 and 0.74 percent in three phase-fixed steps. The loop reaches the published level for the Nogrette variant and goes below it for the Kim variant, because the emulated camera has no noise. Two findings are kept as results: more than one inner GS iteration degrades the Nogrette loop, and a second weight correction in the first Kim step diverges. Only the end points of the published curves are compared, and the

Algorithm 8 Conjugate gradient through the device model. The gradient is the adjoint of the crosstalk operator; a staircase line search replaces the strong-Wolfe search where the quantizer makes the cost a staircase.

Require: target T, device $\mathcal{D}$ (crosstalk kernel K, upsampling, aperture envelope, quantizer), power t, iteration budget K, start ϕ^0

$\phi \leftarrow \phi^0$; evaluate $C_t(\phi)$ and $g \leftarrow \nabla C_t(\phi)$ through the adjoint of Eq. (35); $d \leftarrow -g$
for $k = 1, \dots, K$ **do**
    find a step α by the line search along d (strong Wolfe, or the staircase search on a quantizing device)
    $\phi \leftarrow \phi + \alpha d$; $g' \leftarrow \nabla C_t(\phi)$
    $\gamma \leftarrow (g' \cdot g')/(g \cdot g)$; $d \leftarrow -g' + \gamma d$; $g \leftarrow g'$
    if $d \cdot g \geq 0$ **then**
        $d \leftarrow -g$ (restart from steepest descent)
    end if
end for
return ϕ wrapped to $[0, 2\pi)$

conventional-WGS clause of the research record was not run.

Limits and complexity. The method accepts traps on bins only (Fourier form, no depth offset, no vortex). One feedback iteration is the inner iterations plus one camera evaluation. The case feedback driver is a separate loop that applies the Nogrette factor to the case's weight-aware method and is the subject of Section 8. The module is `m12_closed_loop` on the camera of `core/camera`.

6.13 Device-aware conjugate gradient with pixel crosstalk

A real SLM couples neighbouring pixels, so a hologram that is optimal for the ideal model is not optimal on the device. The solution replicated is that of Schroff, La Rooij, Haller and Kuhr [23]: each SLM pixel is upsampled to a block of computational pixels, and the phase is convolved with a fitted kernel before the Fourier transform. A method that optimizes through that blur reaches a lower error on the device than the same optimizer that ignores it. With each SLM pixel represented by 3×3 computational pixels and the phase kept in $[0, 2\pi)$, the forward model of the source is

$$\tilde{\phi} = K * \phi_{\uparrow 3}, \qquad K = \mathcal{F}^{-1}\Big\{ \exp\Big[-\frac{|\kappa_x|^q + |\kappa_y|^q}{\sigma^q} \Big] \Big\}, \tag{34}$$

with the fitted values $\sigma = 1.24$ and $q = 1.80$. The convolution is circular, so the phase wraps at the border of the SLM. The device that the optimizer sees is the crosstalk, the pixel-aperture envelope and the quantizer, with the quantizer treated straight-through (quantized in the forward pass, identity in the backward pass); flicker is dropped from the model. The cost is the powered intensity cost C_t of Eq. (28) evaluated on the focal field of the device, and it is minimized by the same Fletcher–Reeves conjugate gradient as conjugate-gradient minimization. The source uses a Polak–Ribière search with automatic differentiation; DinHot's gradient is closed form, because the adjoint of the crosstalk operator is the convolution with the reversed kernel followed by the sum over each 3×3 block,

$$\frac{\partial C}{\partial \phi} = \big(K^\star * \cdot\big)^{\downarrow 3}\Big[\frac{\partial C}{\partial \tilde{\phi}}\Big], \qquad K^\star(\mathbf{r}) = K(-\mathbf{r}), \tag{35}$$

so the TypeScript port needs no automatic differentiation. A pruned zero-padded transform, which keeps only the nonzero rows and the wanted frequencies, makes the forward and the adjoint cheaper than the plain upsampled transform pair by a factor that the design record measures.

Pseudocode. Algorithm 8 lists the procedure.

Unit of the kernel variable. The kernel of Schroff et al. [23] is $K = \mathcal{F}^{-1}\{\exp[-(|\kappa_x|^q + |\kappa_y|^q)/\sigma^q]\}$ with the fitted $\sigma = 1.24$ px^{-1} and $q = 1.80$, applied to the phase up-scaled to 3×3 computational pixels per SLM pixel; it is their Eq. 4, in the Results subsection on pixel crosstalk modelling, and the article defines the variable, in the equation of the image-plane field, as the spatial frequency of the image plane, $\kappa_x = x/(\lambda f)$. The open question is the pixel that the unit px^{-1} refers to: the SLM pixel, or the computational pixel on which the kernel is convolved, which is one third of it. DinHot reads κ in cycles per SLM pixel, so that the computational grid spans plus and minus three halves of a cycle per SLM pixel; at the Nyquist frequency of the SLM, one half, the transfer along one axis is then $\exp[-(0.5/1.24)^{1.8}] = 0.82$. If the width were in cycles

per computational pixel, it would be three times larger per SLM pixel, 3.72, and the transfer at the same frequency $\exp[-(0.5/3.72)^{1.8}] = 0.97$, a much weaker low-pass; a reading in radians per pixel would be a much stronger one. The source does not settle the reading in the text that was read, and the factor of three changes the width of the blur that the device-aware methods model, and with it the size of their gain. The claim of the method is therefore relative to this device model and not to a measured instrument.

Parameters. The device is the one passed, and the published kernel is available as a ready device with upsampling three. The power, the design efficiency and the line search are those of conjugate-gradient minimization (Table 16). Where the device quantizes, the cost is a staircase and the strong-Wolfe search failed after a few iterations whatever the budget; a staircase search used by this method only lets it run its budget. The scored spread on that device does not improve systematically with the budget (three seeds before and after the change give a spread that moves in both directions), which is a measured negative. From a weighted-GS start on the full C08 device the method scores lower than conjugate-gradient minimization on the ideal model, and a variant for the device without the quantizer and the flicker scores lower still, so on that case the straight-through quantizer costs more than it gives; the start matters, because from a random start the same run is far worse.

Complexity. One iteration costs about the 3×3 upsampled transform pair against one pair for conjugate-gradient minimization, and the source reports a run time several times longer than without the model. The method is baked and replayed, because the gradient on the upsampled grid is too heavy for the live budget at full size; in the bake the cases C04 (shaped disc) and C08 (device-limited array) run it. Traps with an axial offset or a vortex are refused on a device, and the trap fields are read at the nearest bin. With no device the method is conjugate-gradient minimization bit for bit, which a test checks.

Acceptance criterion and gate. On the crosstalk device the method has a lower error than conjugate-gradient minimization without the model, on a large disc (Table 19). The gate runs a disc of 24 bins radius on a 64-pixel SLM padded twice, with 100 iterations and one seed, scores both holograms through the device and requires the device-aware error to be below that of conjugate-gradient minimization; companion tests check the gradient through the crosstalk operator against finite differences, the refusal of a vortex trap, and determinism. The measurement of this report is: error 0.241 (efficiency 0.535) against 1.382 (0.507) for conjugate-gradient minimization without the device model, large disc on the crosstalk device. Both errors are far above the published ones (1.9 percent with the model against 2.7 percent without, at an efficiency of 28 against 34 percent [23]), because the acceptance budget is 100 iterations and a single seed; the test claims the ordering, not the published level. In the bake the method has the lowest mean signal error on case C04 among the methods run, and on the trap array of C08 its 30 iterations are not converged.

Implementation. The module is `m13_crosstalk_cg`, on the device model of `core/device` and the cost classes of `optim/_common`.

6.14 Zernike correction and sensorless modal recovery

The practice of removing the aberration of the optical system from a tweezer array is additive: the aberration phase is added to the final SLM phase, so a correction is the opposite wavefront added to the hologram. The additive model with a Zernike set is the one of the large-array experiments of Nogrette et al. [22] and Ebadi et al. [17], and a global Zernike set is exactly what the aberration-robust hologram of Section 6.23 improves on. The sensorless search, which needs no wavefront sensor and uses only the camera, is DinHot's implementation of a merit-function modal search; no primary source for the parabola search was opened. The displayed hologram is the base hologram minus the correction, and the focal field includes the wavefront of the system,

$$\phi_{\text{shown}} = \phi_{\text{base}} - 2\pi \sum_j \hat{c}_j Z_j, \qquad E = \mathcal{F}\Big[A\, e^{\,i(\phi_{\text{shown}} + 2\pi \sum_j c_j Z_j)}\Big], \tag{36}$$

where the unknown coefficients c_j are in waves and the basis is orthonormal over the disc inscribed in the SLM, with unit root-mean-square value, so that the rms wavefront error is the root sum of squares of the coefficients. Indexing follows Noll [25]; the ordering beyond mode 11 follows a rule of the engine that was not checked against the paper, and the index of the first modes (the table of Noll) was not read on the primary text.

The merit and the search. The merit is the summed measured trap intensity $M(\hat{c}) = \sum_m I_m$ from the emulated camera of Section 6.12. The Strehl ratio of a small residual aberration is $S \approx \exp(-\sigma_\phi^2)$ with $\sigma_\phi = 2\pi\sigma$ the wavefront standard deviation in radians, the exponential form that Mahajan [37] compares with the Maréchal expansion $1 - \sigma_\phi^2$; hence $M \propto \exp[-(2\pi\sigma)^2]$ with σ the residual wavefront rms in waves, so $\ln M$ is a parabola in each coefficient. One sweep visits the searched modes; for each it evaluates M at

Algorithm 9 Sensorless modal recovery of the Zernike coefficients on the emulated camera: a parabola fit of $\ln M$ in each searched mode, repeated over sweeps.

Require: traps, camera, searched modes $\mathcal{J}$ (Noll 4 to 11), trial step δ, sweep budget K

$\phi_{\text{base}} \leftarrow$ weighted GS of the traps with the base iterations (point form); $\hat{c} \leftarrow 0$
for sweep $k = 1, \ldots, K$ **do**
  biggest $\leftarrow 0$
  for each mode $j \in \mathcal{J}$ **do**
    $y_{\pm}, y_0 \leftarrow \ln M$ at $\hat{c}_j \mp \delta$, $\hat{c}_j$, $\hat{c}_j \pm \delta$, where $M = \sum_m I_m$ is the summed camera reading of the hologram $\phi_{\text{base}} - 2\pi \sum_i \hat{c}_i Z_i$ in the aberrated system (three camera frames)
    $q \leftarrow y_- - 2y_0 + y_+$
    if $q < 0$ **then**
      step $\leftarrow$ clip $\big(\delta\,(y_- - y_+)/(2q),\ \pm 2\delta\big)$
    else
      step $\leftarrow$ the point of $\{-\delta, 0, +\delta\}$ with the largest y
    end if
    $\hat{c}_j \leftarrow \hat{c}_j +$ step; biggest $\leftarrow \max(\text{biggest}, |\text{step}|)$
  end for
  record the ideal trap fields of the corrected system and the coefficient error; **if** biggest $< 10^{-4}$ waves **then stop**
end for
return $\phi = \phi_{\text{base}} - 2\pi \sum_{j \in \mathcal{J}} \hat{c}_j Z_j$ wrapped to $[0, 2\pi)$

$\hat{c}_j - \delta$, $\hat{c}_j$ and $\hat{c}_j + \delta$, fits the parabola to $\ln M$ and moves to its vertex,

$$\hat{c}_j \leftarrow \hat{c}_j + \delta\,\frac{y_- - y_+}{2\,(y_- - 2y_0 + y_+)}, \qquad y = \ln M, \quad \text{step clipped to } \pm 2\delta, \tag{37}$$

or to the best of the three points when they are not concave. The modes are not exactly orthogonal under the Gaussian beam and the off-axis traps, so the sweep repeats and stops when the largest step of a sweep falls below a small tolerance or the sweep cap is reached. Tip, tilt and piston are excluded, because they move the traps or leave them unchanged. The camera cost is three frames per searched mode, 24 per sweep for the eight modes, and the history of the sweep adds one evaluation of the ideal field of the corrected system, which is not a camera frame.

Pseudocode. Algorithm 9 lists the recovery.

Parameters and how they were chosen. The mode of operation is recovery by default, and the modes are Noll 4 to 11 (defocus, astigmatism, coma, trefoil and primary spherical), the set of the ladder row. The trial step δ has the default 0.06 waves, about the size of the injected modes of the acceptance set and not tuned against a source; the base hologram is the weighted GS of weighted GS with 30 iterations, the count at which Di Leonardo et al. report the weighted row [12]. The default camera is ideal (no gain error, no noise, no roll-off) unless one is passed.

Complexity. Each sweep costs 24 camera evaluations and one ideal-field evaluation, each a focal field by a fast Fourier transform, $O(P \log P)$ with $P = n_{\text{fft}}^2$. The method runs in the browser, and in the bake the aberrated case C09 runs it with the case's aberration as the injected set.

Acceptance criterion and gate. The search recovers an injected set of Noll modes 4 to 11 within five percent of their amplitude (Table 19). The gate draws three seeds with random signs and amplitudes in a stated range of waves, runs six sweeps on the 10×10 task, and requires every searched mode within the bound, the unsearched coefficients exactly zero and a final efficiency at least 0.98 of the aberration-free base; companion tests check that a known aberration is subtracted exactly, that partial knowledge corrects partially, and that recovery with a noisy camera stays within ten percent of the injected rms. The measurement of this report is: worst per-mode relative error 0.002 [0.001, 0.002] over Noll modes 4 to 11. Before correction the efficiency is a small fraction of the aberration-free base; after recovery it returns to it.

Limits. The merit assumes a roughly Gaussian response in each coefficient, which strong aberrations break; the camera is an emulation, so the recovery is shown against stated errors and not against an instrument; and a global set cannot correct aberrations that vary across the field, the gap from which the aberration-robust hologram starts. The module is `m14_zernike` on the polynomials of `core/zernike`.

6.15 Laguerre–Gaussian vortex traps

The established way to give a trap orbital angular momentum is one more term in the delay of the point-trap model, so that vortex traps, rings of light with a dark centre, mix freely with plain traps in one hologram. Di Leonardo et al. [12] note that the delay of the point-trap model generalizes to carry orbital angular momentum, and vortices were demonstrated in a trap array by Polin et al. [15]. The delay of a charged trap is Eq. (1) with the azimuthal term $l_m \operatorname{atan2}(y_j, x_j)$. The field V_m of a charged trap is the overlap of the focal field with the vortex mode of charge l_m; it is this projection that is equalized, and the intensity at the centre of the trap is dark, which a test checks. The hologram is the weighted GS of weighted GS with the charges carried by the back-projection,

$$\phi_j = \arg \sum_m e^{+i\Delta_j^m}\, w_m \frac{V_m}{|V_m|}, \qquad \Delta_j^m \text{ with } l_m \neq 0, \tag{38}$$

so that the powers of the modes equalize. Only the point form exists, because charged traps are off the bins of the Fourier form, and there is no graphics-processor path.

The acceptance quantity. On a ring of radius r around the trap, sampled at 48 points, the focal phase ψ_k is read, the total winding is the sum of the wrapped phase steps, which should be $2\pi l$ times a fixed sign, and the maximum error is the largest deviation of ψ_k from the best-fit line,

$$\varepsilon_{\max} = \max_k \left|\psi_k - (\pm\, l\, \theta_k + c)\right|, \qquad \sum_k \operatorname{wrap}(\psi_{k+1} - \psi_k) = \pm 2\pi l. \tag{39}$$

The radius is the one of maximum mean intensity among a set of candidate radii, so that the measurement is made on the bright ring; the number of points and the radius rule are DinHot's choices and are not sourced. The convention is that the winding of the field has the sign of the charge. The field at a ring point is the plain point-trap projection there.

Why the equalization is on the mode power. The intensity at the centre of a vortex trap is zero, so the quantity that can be equalized is not the peak of the focal intensity but the overlap of the focal field with the mode of the trap, which is the projection that Eq. (2) computes with the vortex term in the delay. The weights of Eq. (38) therefore equalize the mode powers of the charged traps with those of the plain traps in the same hologram, and the ring of light of a vortex trap carries a power that is comparable to that of a plain trap of the same weight.

Parameters. The charge defaults to the one carried by each trap, and a keyword overrides all traps with one integer; a target with no charged trap and no keyword is refused. The hologram rule is that of weighted GS, in the point form.

Complexity. The point form costs the number of pixels per trap per iteration, because the delay of a charged trap is dense and built one trap at a time. The method runs in the browser, and the port agrees with the Python fixtures to the tolerance stated in Section 9; in the bake the cases C05 and C06 run it. For a target whose traps already carry charges weighted GS chooses the point form and runs the same weighted GS as the vortex method, so the two methods give the same values in the bake.

Acceptance criterion and gate. The ring phase winds by $2\pi l$ with an error below 0.1 rad (Table 19). The gate has an isolated vortex of charges 1, 2, 3 and -2, with the winding exact and the maximum deviation below the bound, and an array of four vortices 24 bins apart in one hologram, with the winding exact, the rms error below 0.1 rad and the maximum below 0.2 rad (the array bound is looser than the ladder, see below), plus tests of the sign convention, of the keyword and of determinism. The measurement of this report is: ring phase winds by 2 pi l (winding error below the 1e-9 rad of the arithmetic) with a maximum error of 0.033 [0.028, 0.060] rad (isolated vortices of charge 1, 2, 3 and -2). In the array one member, an off-axis trap with charge 2, exceeds 0.1 rad in its maximum, so the array test asserts the rms below 0.1 and the maximum below 0.2; the ladder's bound holds for the isolated case.

Limits. Charged traps are point-form only. The vortex phase is a pure azimuthal term, so the model carries the charge but not a radial index. The model is scalar, and the high numerical aperture case carries a validity flag. The module is `m15_vortex`; the ring-profile function is the measurement of the acceptance quantity and is the one the notebooks reuse.

6.16 Hungarian assignment and linear phase interpolation

A loaded array of atoms has holes. Rearrangement moves loaded atoms into a defect-free target pattern: it decides which atom goes to which site, and it moves the traps smoothly with a sequence of holograms so that

Algorithm 10 Hungarian assignment and linear phase interpolation. The solver is the Kuhn–Munkres algorithm with potentials; every frame is the point-trap back-projection of the interpolated amplitude and phase of each tweezer.

Require: target sites t_i, loaded atoms a_j (at least as many as sites), exponent α, step bound $s_{\max}$ (one Fourier unit), freeze index N

$f \leftarrow$ Hungarian assignment minimizing $\sum_i |t_i - a_{f(i)}|^\alpha$; starts $x^i_m \leftarrow a_{f(m)}$, ends $x^f_m \leftarrow t_m$

$S \leftarrow \max\{1, \lceil d_{\max}/s_{\max} \rceil\}$ with $d_{\max}$ the longest path; count the pairs of crossing or overlapping paths

run phase-fixed weighted GS on the start traps and on the end traps; read the amplitudes a^i, a^f and phases ψ^i, ψ^f of the trap fields

$\Delta\psi_m \leftarrow \mathrm{wrap}(\psi^f_m - \psi^i_m)$

for $s = 0, \dots, S$ **do**

  $\tau \leftarrow s/S$; $x_m \leftarrow x^i_m + \tau(x^f_m - x^i_m)$; $c_m \leftarrow \left(a^i_m + \tau(a^f_m - a^i_m)\right) e^{i(\psi^i_m + \tau\Delta\psi_m)}$

  $\phi_s \leftarrow \arg \sum_m c_m \, e^{+i\Delta^m(x_m)}$; read the trap fields of ϕ_s at the positions x_m

end for

return the frames $\phi_0, \dots, \phi_S$ and, per trap, the lowest intensity over the frames divided by its mean

no moving trap dims enough to lose its atom. Two sources are replicated: the Hungarian assignment with the cost d^α of Lee, Kim and Ahn [27], and the linear phase interpolation (LPI) of the sequence of [28], which reports, from a high-speed camera on a separate test set-up, that the weighted-GS spot nearly vanished several times while the spot of the interpolation, for identical initial and final phase, stayed above 70 percent.

Assignment. Each target site t_i is matched to a distinct loaded atom $a_{f(i)}$ so as to minimize

$$C(f) = \sum_i d^\alpha_{i,f(i)}, \qquad d_{i,j} = |t_i - a_j|, \quad \alpha > 1, \tag{40}$$

so that paths do not cross; for a square lattice an exponent above 1.12 keeps the minimum interatomic distance during moves at or above $1/\sqrt{2}$ of the lattice constant [27]. The solver is the Kuhn–Munkres algorithm with potentials, $O(n^2 m)$ for an $n \times m$ cost with $n \le m$ (a rectangular case, whose unmatched atoms are released); it is tested against brute force on 96 random matrices. A routine counts the pairs of straight paths that intersect or overlap along a segment.

Hologram sequence. The initial and final holograms are phase-fixed weighted GS (phase-fixed weighted GS). From each, the position, amplitude and optical phase $\psi_m = \arg V_m$ of every tweezer are read and linearly interpolated over S steps,

$$x_m(s) = x^i_m + \tfrac{s}{S}(x^f_m - x^i_m), \quad a_m(s) = a^i_m + \tfrac{s}{S}(a^f_m - a^i_m), \quad \psi_m(s) = \psi^i_m + \tfrac{s}{S}\,\mathrm{wrap}(\psi^f_m - \psi^i_m), \tag{41}$$

and the transform is inverted at each step, $\phi(s) = \arg \sum_m a_m(s) e^{i\psi_m(s)} e^{+i\Delta^m_j(s)}$, which is the point-trap back-projection, exact at fractional positions. The number of steps is set by the longest path, $S = \lceil d_{\max}/s_{\max} \rceil$. Because the amplitude is interpolated and the phase advances smoothly, no trap passes through zero intensity, unlike independent weighted-GS holograms whose free phases jump between steps. Knottnerus et al. [28] obtain the position and optical phase of each tweezer from the transform of the initial and the final hologram, interpolate them linearly over N steps and invert the transform at each step; N is chosen so that the longest trajectory advances at most one Fourier unit, $\lambda f/L \approx 0.45$ μm, per hologram (their Section 3, in prose). The engine uses the same step, one Fourier unit, so $s_{\max}$ is one bin of the unpadded grid. The source does not state how the phase difference is taken; DinHot interpolates the wrapped difference, the shortest phase path. The equations above write the described procedure in the notation of this report.

Pseudocode. Algorithm 10 lists the sort.

Why an exponent above one avoids crossings. With a linear cost ($\alpha = 1$) the total path length is minimized and two atoms may swap targets at no cost, so the optimal matching can contain paths that cross; a convex cost penalizes the long paths more than the short ones, which pushes the solution toward the matching in which every atom takes the nearest free target and the paths stay disjoint. The same convexity is what the lattice bound quoted above expresses: for a square lattice the minimum interatomic distance is kept during the moves only when the exponent exceeds the bound.

Parameters. The exponent has the default 1.5, the paper value, above the lattice bound; the step bound defaults to one Fourier unit; the freeze index is that of phase-fixed weighted GS (default 12, whose grid dependence is discussed there).

Complexity. The assignment costs $O(n^2 m)$; the sequence costs two phase-fixed weighted GS runs plus

$S+1$ point-trap back-projections of $O(N_{\mathrm{pix}}M)$ each. The method runs in the browser, and the port receives the loaded atoms of the case's rearrangement block, drawn with one random number per site.

Acceptance criterion and gate. There are no crossing moves for the chosen exponent on a sort from a 7×7 lattice to a 3×3 pattern, and a moving trap stays above 70 percent of its mean intensity (Table 19). The gate draws 60 loadings of 21 atoms and asserts zero crossings in every one; a second test runs five loadings of the sequence and requires no crossing, a stated minimum number of moving traps, and a worst minimum-ratio of at least 0.7. The measurement of this report is: 0 path crossings in 60 loadings; moving traps keep 0.767 [0.747, 0.792] of their mean intensity (worst moving trap of each of 5 loadings). A comparison against independent weighted-GS holograms per step was planned and dropped from the criterion, because with nine traps the weighted GS equalizes every step and shows no flicker, unlike the camera observation of the paper; the proposed flicker-bounded sequence of Section 6.21 takes over that comparison.

Limits. LPI does not optimize its intermediate frames, which is the gap that the flicker-bounded sequence addresses, and the transient measure of this module is simplified, since each frame is scored at its own positions and not in the blend model of that method. The assignment needs at least as many atoms as targets. The success rate of the source is a different quantity and is not reproduced here. The module is `m16_rearrangement`.

6.17 Unsupervised convolutional network for three-plane holograms

The learned method replicates, in the setting of its own paper and at the size of this product, DeepCGH of Eybposh et al. [29]: an unsupervised U-Net maps the target amplitudes of several planes to one SLM phase, trained with a loss that propagates the hologram to every plane through a differentiable model. The source reports that the network is at least ten times faster than Gerchberg–Saxton and more accurate on volume targets, with a stated number of iterations for each iterative method and a stated test-set size. A first version of this product on a single plane tied with GS and was rejected; the method was rebuilt in the setting of the paper, a volume of three planes. No source that was read compares a learned method against weighted GS on the uniformity of hundreds of point traps; a supervised variant for tweezers [30] reports a time per hologram well below that of GS on about twenty sites. The problem the method answers is a hologram for a three-plane point target by one network pass, in tens of milliseconds, when the target changes at interaction rate; the baseline it must beat is the volumetric Gerchberg–Saxton of Section 6.19 at the same wall-clock.

Network and loss. The network is a U-Net of 14 channels that predicts one complex focal field per plane; a constant inverse discrete Fourier transform (a matrix product) and the lens phase of each plane carry each field to the SLM, the fields are summed, and the output is the phase of the sum. The exported graph is below the size limit of the browser release, and every operator is in the list the browser runtime supports. Eybposh et al. [29] define the accuracy of a volume as the cosine similarity of the target and reconstructed intensities, $\sum I\hat{I}/\sqrt{\sum I^2 \sum \hat{I}^2}$ over all voxels (their Eq. 3). DinHot scores, and trains with, a per-plane overlap that has the same range and the same maximum but not the same value, so the accuracies of this report are not comparable number for number with those of the source; the same definition is used for the loss, the baseline and the evaluation. The plane field is the centred orthonormal Fourier transform of $A\,e^{i(\phi - Zq)}$ with q the quadratic lens term and Z the defocus of the plane at the aperture edge, and with T_p and I_p the unit-sum plane intensities of the target and of the hologram,

$$\mathrm{AC} = \frac{1}{P}\sum_{p=1}^{P} \frac{2\sum T_p I_p}{\sum T_p^2 + \sum I_p^2}, \qquad \mathcal{L} = 1 - \mathrm{AC}. \tag{42}$$

The depths are -8, 0 and $+8$ SLM-limited Rayleigh ranges $z_{R,\mathrm{SLM}}$, where

$$z_{R,\mathrm{SLM}} = \frac{\lambda f^2}{\pi R^2}, \qquad R = \frac{n_{\mathrm{pix}}\, p}{2}, \tag{43}$$

which makes the defocus phase at the aperture edge exactly Z radians. The torch forward model is verified against the point-trap model of the engine at a trap of that depth.

Inference and fall-back. With the model released, the hybrid runs the network and then the volumetric GS from the network's phase in what is left of the time budget; the refinement is kept only if it raises the volume accuracy of the network's mask, because the volumetric GS is a point-voxel method whose refinement can lower the accuracy of the full plane images. With no released model the baseline answers, the volumetric GS at 30 iterations, and the result carries the provenance `baseline`, the missing-data rule of the catalog row of the model (Section 4). The released graph is specific to its crop: a 128×128 SLM with no padding,

Algorithm 11 Inference of the volume network with the refinement and the fall-back (training is offline and not part of the run).

Require: three-plane target traps within the stated lateral limit, released model (or none), wall-clock budget, refinement flag
build the network input from the traps: unit-energy amplitudes of the three planes (spots of one bin width)
if no model is released **then**
    return volumetric GS from the random superposition for 30 iterations, with the provenance `baseline`
end if
$\phi_{\text{net}} \leftarrow$ network output for the three planes (one pass)
if refinement is on and time remains **then**
    $\phi_{\text{ref}} \leftarrow$ volumetric GS started from ϕ_{net} in the remaining budget
    if $\text{AC}(\phi_{\text{ref}}) > \text{AC}(\phi_{\text{net}})$ by Eq. (42) **then**
        return ϕ_{ref}
    end if
end if
return ϕ_{net}

three planes, traps at integer lateral bins within a stated distance of the axis and no vortex; any other setup is refused with the requirement named.

Pseudocode. Algorithm 11 lists the inference.

Training, parameters and how they were chosen. Training is offline on a graphics processor and never in the continuous-integration run or the browser; it runs 16000 steps with a stated batch size, data seed, initialization seed and learning rate recorded in the training record of the model contract, and the schedule was restarted once with a shorter total. The depths and the lateral limit are DinHot's choices. The refinement is on by default, though the evaluation recommends calling with it off, because the refinement was never kept on the held-out set. The network answers in about 44 ms.

Acceptance criterion and gate. On held-out three-plane volumes the network start followed by the refinement inside the same wall-clock budget reaches a higher accuracy than the volumetric GS alone (Table 19); the code and the test implement the median per-volume ratio of the hybrid to the baseline above one. The gate reads the committed metrics (at least 150 held-out volumes and a median ratio above one), recomputes on a sample of the held-out volumes when the released graph and the held-out file are available, and checks the fall-back, the budget and the error messages. The measurement of this report is: three-plane network refined by volumetric 3D GS: median accuracy ratio 1.140 (quartiles 1.103 and 1.163) on 162 held-out volumes; taken from the committed metrics.json, not remeasured here. The held-out volumes are 162 (a generated set of four kinds plus two benchmark layouts), the budget is the median time of the baseline at 30 iterations, 2.40 s, and the ratio is above one on 100 percent of the volumes; the graph agrees with its training framework on the plane intensities to 8.4×10^{-6} of the maximum, while the phase of a near-zero SLM field is ill conditioned, so the parity claim is made on intensities. The single-plane version, measured against GS, gave a median ratio of 0.9994 on 219 held-out targets and was not released.

Limits. The model is specific to its crop and planes, as DeepCGH also trains one model per resolution and plane set; the advantage is on plane images and not on trap uniformity, since the volumetric GS optimizes trap uniformity and the accuracy measure scores full plane images, ghosts and blur included, which is what the network optimizes. The module is `m17_cgh_net`; the web encoder builds the three-plane input that the model card declares.

6.18 Optimal-transport initialized MRAF

Mixed-region amplitude freedom from a random start leaves speckle that it removes slowly. An optimal-transport map between the input beam and the target intensity gives a smooth phase that already sends the light roughly to the right place, so that MRAF starts close to its answer. The solutions replicated are those of Swan et al. [32], whose table of six targets compares MRAF with and without the initialization, and, for the memory-light dual form, Torchylo et al. [33]. The MRAF stage is the rule of Section 6.8. Under the stationary-phase approximation the SLM phase satisfies the Monge–Ampère equation

$$G^2\big(\nabla\phi(\mathbf{x})\big)\,\det\big(\nabla^2\phi(\mathbf{x})\big) = g^2(\mathbf{x}), \tag{44}$$

Algorithm 12 Optimal-transport seed by the separable Sinkhorn iteration, followed by the mixed-region iteration of Eq. (24).

Require: shaped target (amplitude, mask), mixing m, entropic parameter ε, Sinkhorn iterations K_{OT}, MRAF iterations K, optional start

if no start is given **then**
  $g^2 \leftarrow A^2/\sum A^2$ (input intensity); $G^2 \leftarrow T^2/\sum T^2$ on the mask (target intensity); kernels $K_x = K_y = \exp[-(x-y)^2/(2\sigma^2)]$, $\sigma^2 = \varepsilon n_{\mathrm{fft}}^2$ bins squared
  $v \leftarrow$ indicator of $G^2 > 0$
  for $l = 1, \ldots, K_{\mathrm{OT}}$ **do**
    $u \leftarrow g^2 \oslash (K_y v K_x^{\mathsf{T}})$; $v \leftarrow G^2 \oslash (K_y^{\mathsf{T}} u K_x)$ where $G^2 > 0$, else 0
  end for
  barycentric map $T(\mathbf{x}) \leftarrow$ the Sinkhorn plan's mean target coordinate (the identity where the plan has no mass)
  $\Psi \leftarrow$ least-squares potential with $\nabla\Psi = T$ (Poisson solve on a mirror-extended grid); $\phi \leftarrow (2\pi/n_{\mathrm{fft}})\Psi$ wrapped to $[0, 2\pi)$
end if
scale the target amplitude so that its power equals the input power; $E \leftarrow \mathcal{F}[Ae^{i\phi}]$
for $k = 1, \ldots, K$ **do**
  $G \leftarrow mT$ on the mask, $(1-m)|E|$ elsewhere, times $e^{i \arg E}$; $\phi \leftarrow \arg \mathcal{F}^{-1}[G]$; $E \leftarrow \mathcal{F}[Ae^{i\phi}]$; record the error
end for
return ϕ

with g^2 the input intensity and G^2 the target intensity; the quadratic-cost optimal-transport map γ between them is integrated, $\phi(\mathbf{x}) = \int \gamma(\mathbf{s}) \cdot d\mathbf{s}$, and seeds the iteration.

Entropic transport. The separable Sinkhorn form uses a plan $\Gamma = uKv$ with a separable kernel $K = K_x \otimes K_y$ and the updates

$$u \leftarrow g^2 \oslash (\Lambda v \Lambda), \qquad v \leftarrow G^2 \oslash (\Lambda^{\mathsf{T}} u \Lambda^{\mathsf{T}}), \tag{45}$$

evaluated as matrix products of the one-dimensional kernels, so that the memory is linear in the number of pixels. The updates are Eqs. (7) and (8) of Torchylo et al. [33]; the implementation is the same pair of updates, evaluated as matrix products of the symmetric one-dimensional kernel, with the kernel width of their $\Lambda_{jK} = \exp[-(j-K)^2/(2n^2\varepsilon)]$ in bins, and is checked by the marginals and by recovery tests. The cost is half the squared distance between the SLM pixel coordinate and the focal bin coordinate; the kernel width is the entropic parameter times the squared grid size in bins, because the coordinates are not normalized. The barycentric map $T(\mathbf{x}) = \sum_y \Gamma(\mathbf{x}, y)\, y / \sum_y \Gamma(\mathbf{x}, y)$ (the identity where the plan has no mass) is integrated to a potential with $\nabla\Psi = T$ by a least-squares Poisson solve on a mirror-extended grid, which is exact for a curl-free map and its least-squares projection otherwise, and the stationary-phase relation $k = (n_{\mathrm{fft}}/2\pi)\, d\phi/dj$ gives $\phi = (2\pi/n_{\mathrm{fft}})\Psi$. The projection removes the small curl that the barycentric map of an entropic plan has.

MRAF stage and fixed point. Then the rule of Eq. (24) runs with the mixing m; it is the update of Pasienski and DeMarco [18] (their Eq. 1). The source keeps the power of the target constant across iterations and gives no numerical normalization; DinHot scales the target amplitude so that its power equals the input power, and rescaling to the current signal-region power instead drains the signal region, which was measured. With mixing m the fixed point of the stage is near the efficiency that solves

$$\eta = \frac{m^2}{m^2 + (1-m)^2(1-\eta)}, \tag{46}$$

so that the efficiency of the method is in the MRAF class and not in the class the paper reports.

Pseudocode. Algorithm 12 lists the seed and the iteration.

Parameters. The mixing has the default 0.4, in the range reported for MRAF and larger than the mixing of mixed-region amplitude freedom; the entropic parameter has the default 0.001, the Sinkhorn value of the source for coordinates normalized by the image size; the number of Sinkhorn iterations is DinHot's choice, 200, with the marginal test run on a smooth problem. A supplied initial phase replaces the transport seed, which is what the random-start baseline of the acceptance test uses.

Complexity. The separable Sinkhorn needs memory linear in the number of pixels and matrix products of the one-dimensional kernels, then $O(P \log P)$ per MRAF iteration; the method is baked, and runs at a live size of a 128×128 focal grid and at a baked size of 256×256.

Acceptance criterion and gate. The error is lower than that of MRAF from a random start on the shaped targets (Table 19). The gate runs a disc, a ring and a line at the live size and the first two at the baked size, with 100 MRAF iterations at the default mixing, and requires the initialized error to be below the minimum of two random starts and the efficiency above 0.3; component tests check both Sinkhorn marginals, that the barycentric map of a pure shift is a shift, that the gradient integration recovers a known potential, that the seed moves the beam onto a displaced blob, and determinism; a mutation run that replaces the seed by a random phase fails the tests. The measurement of this report is: signal-region error 0.0204 (disc), 0.0179 (ring) and 0.0221 (line) against 0.250, 0.188 and 0.223 for MRAF from a random start (the better of two); baked size: 0.0259 (disc) and 0.0171 (ring) against 0.266 and 0.228. The error is about an order of magnitude lower than the random start. The published errors are lower, with higher efficiency, than those of this product (the fixed point of Eq. (46) bounds the efficiency here).

Limits. The efficiency does not reach the range of the paper; the model is the ideal device, so scored on the crosstalk device of case C04 the error is close to that of the device-aware method and far above the ideal-device acceptance values. The module is `m18_ot_mraf`.

6.19 Multi-plane Gerchberg–Saxton, the volumetric form

The established method for placing traps at several depths from one hologram is the form of Whyte and Courtial [26]: a three-dimensional Gerchberg–Saxton between a real-space volume (the target) and a k-space field restricted to the Ewald sphere, which is a three-dimensional Fourier pair and not a plane-by-plane propagation. It is the baseline of the learned network of Section 6.17, which is compared with it at equal wall-clock, and it is the alternative to the three-dimensional form of the weighted point-trap model (an axial term in the delay) against which the lattice test of weighted GS measures it. The constraint is the Ewald sphere $k_0^2 = k_x^2 + k_y^2 + k_z^2$ with $k_z > 0$: in the discrete implementation, for each (k_x, k_y) only the k_z element nearest to $\sqrt{k_0^2 - k_x^2 - k_y^2}$ carries the field, and all others are zero (a hemisphere one element thick). One iteration is

$$K[k_x, k_y, k_z^{\mathrm{sph}}] = A_j e^{i\phi_j}, \qquad U = \mathcal{F}_{3D} K, \qquad U' = a_m e^{i \arg U} \text{ at the target voxels, 0 elsewhere,} \tag{47}$$

$$\phi_j = \arg\left(\mathcal{F}_{3D}^{-1} U'\right)[k_x, k_y, k_z^{\mathrm{sph}}], \qquad a_m = \sqrt{w_m}, \tag{48}$$

and the SLM amplitude is reset to the beam. The hologram is the k-space phase on the sphere projected onto the SLM plane.

Mapping to the engine. The mapping is derived in the module and not in the source. SLM pixel j is the lateral frequency $\boldsymbol{\xi} = (x_j, y_j)/(\lambda f)$; the lateral voxel is one bin times the padding, $dx = \lambda f/(n_{\mathrm{pix}} p)$, so that a trap at x_m sits on voxel round(x_m/dx); in z the voxel pitch is a parameter and the carrier $k_0 z$ is removed (baseband), so that the sphere element is

$$i_z = \mathrm{round}\left(n_z\, dz\, \frac{1 - \sqrt{1 - s}}{\lambda_m}\right) \bmod n_z, \qquad s = (\lambda_m \xi)^2, \quad \lambda_m = \frac{\lambda}{n_{\mathrm{imm}}}, \tag{49}$$

which in the paraxial limit is the lens term of the point-trap model. The lateral origin is put on the optical axis by a linear ramp, and the start is the random superposition of the snapped traps.

Pseudocode. Algorithm 13 lists the iteration.

Parameters. The axial voxel pitch defaults to the smallest gap between the distinct plane depths of the target, the finest axial structure that the target contains; the number of axial elements is the next power of two of at least a stated minimum and of eight times the largest sphere index plus one, which sets the length of the three-dimensional transform. There is no per-plane weighting, because the source has none (the weak experimental match that the paper reports is a secondary statement that was not read).

Complexity and limits of the construction. One iteration is two three-dimensional transforms of a complex volume of $n_z \times n_{\mathrm{pix}} \times n_{\mathrm{pix}}$, whose size grows with n_z. The depth extent $n_z\, dz$ is a period: the sphere elements wrap modulo n_z at the corners of the square SLM, producing periodic replicas of the volume. The axial placement error at depth z is about $\pi z/(n_z\, dz)$ radians. The axial resolution is set by the aperture, with a depth of field of about $2\lambda/\mathrm{NA}^2$ and $\mathrm{NA} = n_{\mathrm{pix}} p/(2f)$, so planes closer than that cannot be told apart by any hologram. The method runs live at small size and baked at full size.

Algorithm 13 Volumetric Gerchberg–Saxton with the Ewald-sphere constraint on the grid of the engine.

Require: point traps, axial pitch dz, axial size n_z, iteration budget K, start ϕ^0

snap each trap to its voxel $(\mathrm{round}(x_m/dx), \mathrm{round}(y_m/dx), \mathrm{round}(z_m/dz))$ (refuse two traps on one voxel); compute the sphere index $i_z(j)$ of every SLM pixel by Eq. (49)

$\phi \leftarrow \phi^0$ (the random superposition of the snapped traps when no start is given)

for $k = 1, \ldots, K$ **do**

  $K \leftarrow 0$; $K[i_z(j), j] \leftarrow A_j e^{i\phi_j}$ for every pixel j; $U \leftarrow \mathcal{F}_{3D} K$ (with the ramp that puts the origin on the axis)

  $U' \leftarrow 0$; $U'_m \leftarrow a_m \, U_m / |U_m|$ at the target voxels

  $\phi_j \leftarrow \arg\left(\mathcal{F}_{3D}^{-1} U'\right)[i_z(j), j]$; record the trap fields of the snapped traps

end for

return ϕ

Acceptance criterion and gate. On a three-plane target every plane mean is within 25 percent of the overall mean (Table 19); the plane intensity band is defined by the test, since the design document does not define it. The gate places three planes a depth of field apart with a 5×5 grid of traps on each, the planes shifted laterally, 20 iterations and two seeds, and requires each plane mean within the band and a final efficiency above 0.3; companion tests check that the reported fields equal the point-trap model of the snapped traps, that a single off-plane trap is focused, that the efficiency rises, and that two traps on one voxel are refused. The measurement of this report is: plane means over overall mean 0.938, 0.987 and 1.075 and 0.926, 1.034 and 1.039 on three planes. The within-plane spread is reported and not bounded; it is high, and it is higher still in the bake, so that the method equalizes the planes and not the traps, while the point-trap three-dimensional form of weighted GS reaches a spread of about one percent on the same case. No plane-by-plane variant exists in this build.

Why the planes interact. In a plane-by-plane propagation each plane's constraint acts on a field that is propagated independently, and the hologram is the average of the back-projections, so the planes do not see each other. In the Ewald-sphere form there is one three-dimensional field, and an update at one depth changes the field at the others through the three-dimensional transform, so that the amplitude constraint at the target voxels of every plane is imposed on the same hologram at once; this is the property that the source demonstrates and that the acceptance test measures as plane means within a band.

Implementation. The module is `m19_multiplane`, whose output is checked against an independent evaluation of the point-trap model.

6.20 Device-aware weighted GS

The first proposed method puts the pixel crosstalk and the quantizer of the SLM inside the loop of the weighted GS for trap arrays, with the phase update replaced by a few conjugate-gradient steps on a device-aware loss. The hypothesis is that this makes the traps equal on the device and not only in the ideal model: on the crosstalk device the relative standard deviation of the trap intensities is at least two times below that of phase-fixed weighted GS, at an efficiency at most ten percent lower. The gap is stated against the literature. Schroff et al. [23] put the crosstalk model into conjugate gradient on shaped potentials, with a long time per hologram and no weighted-GS comparison; Persson, Engström and Goksör [24] modify a commonly used hologram algorithm to compensate the pixel crosstalk of the SLM and improve the spot uniformity by more than one hundred percent on a device with large crosstalk; Kim et al. [16] rely on camera feedback and use no device model in the solver; Ebadi et al. [17] pre-compensate the diffraction envelope of the array and correct the aberrations of the system from measured atom light shifts, and their text describes no model of the pixel crosstalk or the quantizer. The proposal combines two published pieces, the weights of Di Leonardo et al. [12] and the device model of Schroff et al., and answers the problem differently from phase-fixed weighted GS, because the solver is aware of the device, so it is a method and not a validation protocol. The full derivation, the experiments on the factor per geometry and the ablation are in the companion manuscript [45]; this section gives the method as implemented.

Method. The weights of the weighted GS are kept, $w_m \leftarrow w_m \langle r \rangle / r_m$, with $r_m = |V_m|/a_m$ the trap amplitude relative to its design amplitude, measured on the focal field of the device. The projection of the phase update is replaced by K conjugate-gradient steps on the device-aware loss

$$L(\phi) = \frac{\sum_m \left(|V_m(\phi)|^2 - T_m\right)^2}{\sum_m T_m^2}, \qquad T_m = \eta \, \frac{(w_m a_m)^2}{\sum_k (w_k a_k)^2}, \qquad \eta = \sum_m |V_m|^2, \tag{50}$$

Algorithm 14 Device-aware weighted GS: the weight update with conjugate-gradient steps on the device-aware loss, and the safeguard that returns the best admissible iterate.

Require: traps on bins, device, warm iterations W, steps per outer iteration K_c, outer budget K, option `weights`

$\phi \leftarrow W$ iterations of weighted GS from the random superposition (the ideal-model start); $w_m \leftarrow 1$; $a_m \leftarrow \sqrt{w_m^{\text{target}} / \langle w^{\text{target}} \rangle}$

$V \leftarrow$ trap fields of ϕ on the device; floor $\leftarrow 0.9\times$ efficiency of V; best $\leftarrow (\phi, \sigma(V))$

for $i = 1, \ldots, K$ **do**

  $V \leftarrow$ trap fields of ϕ on the device; **if** `weights` **then** $r_m \leftarrow |V_m|/a_m$; $w_m \leftarrow w_m \langle r \rangle / r_m$ **end if**

  $T_m \leftarrow \eta\,(w_m a_m)^2 / \sum_k (w_k a_k)^2$ with $\eta = \sum_m |V_m|^2$

  $\phi \leftarrow K_c$ conjugate-gradient steps on $L(\cdot; w)$ of Eq. (50) from ϕ (restarted at every outer iteration)

  record the history entry; **if** efficiency $\geq$ floor **and** $\sigma < \sigma_{\text{best}}$ **then** best $\leftarrow (\phi, \sigma)$ **end if**

end for

return the phase of best (the warm start included)

where $V = \mathrm{Op}(\phi)$ runs through the straight-through quantizer, the crosstalk operator (the 3×3 upsampling and the circular convolution with the kernel of Eq. (34)), the Fourier transform of the upsampled grid and the pixel-aperture envelope. The gradient is the closed form of the adjoint of Eq. (35), so there is no automatic differentiation. The target level is the current efficiency η, so the steps flatten the traps without trading efficiency away. The weights are meant to supply the integral feedback that the few steps per iteration lack, but the ablation of Table 30 does not show that they are needed: with every weight kept at one the spread is lower on the acceptance task and on the square lattice of the C08 device, and higher on the ring, so that neither form dominates and the method keeps its weights as defined. Each outer iteration therefore reads

$$w_m \leftarrow w_m \frac{\langle r \rangle}{r_m}, \qquad \phi \leftarrow \mathrm{CG}_K\big[L(\cdot; w); \phi\big], \tag{51}$$

restarting the conjugate gradient at every outer iteration because the target changes with the weights.

Safeguard. The weight update is an integrator, and three conjugate-gradient steps per iteration may not follow it on a device whose trap response is quantized and blurred. On the square lattice of the benchmark on the device of case C08 one seed in two ran away inside the model of the device-aware weighted GS itself: the loss rose by an order of magnitude between outer iterations two and four, the weights spread to a ratio of about a thousand and the efficiency collapsed. The method now returns its best iterate, the warm start included: the iterate of lowest relative standard deviation on its device model among those that keep a fraction 0.9 of the warm start's efficiency, the ten percent of the criterion. A run that converges returns its last iterate or one of its tail, and a run that winds up returns an admissible iterate no worse than its start; both are tested.

Pseudocode. Algorithm 14 lists the method with its safeguard.

Parameters. The number of conjugate-gradient steps per outer iteration is 3, DinHot's choice; the warm start is 12 iterations of ideal-model weighted GS (without the phase freeze; the count equals the freeze index of Kim); the outer budget of the acceptance test is four iterations. The device must have its traps on bins with no depth offset or vortex, and a shaped target is refused.

Complexity. Each conjugate-gradient step costs one forward and one adjoint evaluation of the upsampled model, about nine times the pixels of the plain transform; the time per iteration relative to the baseline was not measured. The method is baked.

Acceptance criterion and gate. On the crosstalk device the relative standard deviation is at least two times below that of phase-fixed weighted GS at an efficiency at most ten percent lower (Table 19). The gate takes the 10×10 task on the device with the published kernel, scores the weighted-GS baseline at two iteration counts, and compares the device-aware weighted GS with four outer iterations on one seed against the strongest of the two. The measurement of this report, taken from the measurements of the companion manuscript [45], is: crosstalk device, 10 x 10 task: relative standard deviation 0.20 [0.16, 0.30] percent against 5.21 [4.75, 5.46] for phase-fixed weighted GS at 100 iterations (a factor 26 [16, 33]) at 0.995 [0.991, 0.997] of its efficiency; 4-bit device with fill 0.85: 0.72 [0.60, 1.16] against 5.73 [5.13, 5.88] (a factor 7.9 [5.0, 8.6]) at 0.990 [0.983, 0.996] of its efficiency; taken from the companion manuscript's beyond.json, 5 and 3 seeds. The report also reads the committed bake and benchmark. On the small device-limited case C08 the baked factor is 1.9 on the bake's own seed, at an efficiency ratio of 0.966, and 2.0 on the ratio of the means of its 3 seeds. In the benchmark on the device of C08, the device-aware weighted GS meets both

Algorithm 15 Flicker-bounded phase-continuous sequence. Each frame is a warm-started phase-fixed weighted GS whose trap phases are locked to the smooth trajectory of Eq. (54).

Require: initial traps, destination traps with the same identities, frame count $S+1$, frame iterations K_f, phase-step bound $\Delta\theta_{\max}$
compute $x_m(s)$ and $\theta_m(s)$ for all m and s by Eqs. (53) and (54)
if $\max_{m,s} |\theta_m(s+1) - \theta_m(s)| > \Delta\theta_{\max}$ **then**
    raise an error
end if
$\phi_0 \leftarrow$ cold phase-fixed weighted GS on $x(0)$; $w_m \leftarrow 1$
for $s = 1, \ldots, S$ **do**
    $\phi_s, w \leftarrow K_f$ warm iterations of Eq. (52) at $x(s)$ and $\theta(s)$, started from ϕ_{s-1}, w
end for
return frames $\phi_0, \ldots, \phi_S$ and the transient ratios of Eq. (55)

thresholds on 2 of 5 geometries (G2 and G3), lowers the spread without the factor of two on the large set, and keeps its start on the lattices, where every iterate past the start gives up more than a tenth of the light; on the ideal device the spreads of the device-aware weighted GS and phase-fixed weighted GS are both below the resolution of the file on most geometries, which is a tie and not a gain. The hypothesis is therefore confirmed on the criterion of the ladder and narrowed by the benchmark.

Limits. The pixel of the kernel width is DinHot's reading, the SLM pixel, because the source does not settle it, and the other reading changes the width by a factor of three, on which the size of the gain depends (Section 6.13). Flicker is random and is dropped from the optimization model, so it is scored but not modelled. The crosstalk parameters are device specific, because the calibration of the source takes hours; the size of the effect on a typical device of smaller pitch is unknown. The module is `b1_device_wgs`.

6.21 Flicker-bounded phase-continuous sequence

The second proposed method concerns moving traps. The linear phase interpolation of Section 6.16 interpolates the position, amplitude and optical phase of every tweezer between two endpoint holograms and inverts the transform at each step, so the intermediate holograms are never individually optimized. The hypothesis is that optimizing every intermediate frame, with the trap phases locked to a smooth trajectory, keeps the minimum transient intensity of a moving trap at least 0.9 of its mean and above that of the interpolation, with a per-frame spread no worse. The gap is that the source of the interpolation observes with a high-speed camera that the weighted-GS spot nearly vanished several times, and with atoms in a separate test that phase slips near π per step lost atoms and that slips of several tenths of π barely mattered [28]; no source optimizes the frames of a trap trajectory. The experiments and the scope measurements are in the companion manuscript [45].

Method. For a trajectory $x_m(s)$, $s = 0, \ldots, S$, and trap phases $\theta_m(s)$ smooth in s, every frame is the phase-fixed weighted GS of phase-fixed weighted GS with the trap phases locked to the trajectory,

$$c_m = w_m a_m e^{i\theta_m(s)}, \qquad w_m \leftarrow w_m \frac{\langle r \rangle}{r_m}, \qquad \phi_j = \arg \sum_m c_m \, e^{+i\Delta_j^m(x_m(s))}, \tag{52}$$

started from the previous frame's hologram and weights with a few warm iterations per frame (at the first frame after the cold start the weights are one, because the cold stage does not return its own). The positions follow the minimal-jerk profile and the phases the same blend,

$$x_m(s) = x_m(0) + \big(x_m(S) - x_m(0)\big) h(\tau), \quad h(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \quad \tau = \frac{s}{S}, \tag{53}$$

$$\theta_m(s) = \theta_m(0) + \operatorname{wrap}\big(\theta_m^{\text{end}} - \theta_m(0)\big)\, h(\tau), \tag{54}$$

with the end phases held at the initial ones by default. Frame 0 is a cold phase-fixed weighted GS. The optimization replaces the penalty form of the loss, a weighted sum of squared deviations of the field phase from θ, by the hard lock, its infinite-weight limit.

Pseudocode. Algorithm 15 lists the sequence.

Transient intensity (DinHot's model). Between frames s and $s+1$ the displayed SLM field is modelled as the blend $f_a = A[(1-a)e^{i\phi_s} + a\, e^{i\phi_{s+1}}]$ with $a \in [0,1]$, the liquid crystal relaxing between the two

holograms, and trap m is read at its trajectory point $x_m(s+a)$, so that $I_m(s,a) = |V_m(f_a; x_m(s+a))|^2$. The quantity of the claim is

$$\rho_m = \min_{s,a} \frac{I_m(s,a)}{\operatorname{mean}_s I_m(s)}, \tag{55}$$

evaluated over a number of sub-steps per frame interval, and both the flicker-bounded sequence and the interpolation baseline of the test are scored by this same function. The source gives the measurement of the interpolation, not its transient equation, so the comparison is internally fair and not a statement about a real SLM. The refresh rate and the temporal response of the device are not modelled.

Parameters. The default frame count is 13, the length of the sort of the design study; the iterations per frame default to 10 (a mutation with none fails the acceptance test); the phase step per frame is bounded by 0.3 rad, DinHot's bound based on the observation that slips of several tenths of π barely mattered.

Acceptance criterion and gate. The minimum transient intensity of a moving trap is at least 0.9 of its mean against the interpolation (Table 19); the gate moves two columns of a 4×4 lattice by four bins in 13 frames, over three seeds, with the interpolation baseline written in the test from the text of the source, and requires, per seed, a ratio at least 0.9 and above that of the interpolation and a largest per-frame spread no worse. The measurement of this report, taken from the companion manuscript [45], is: 4 x 4 lattice, two columns moved by four bins in 13 frames: lowest transient intensity of a moving trap 0.920 [0.915, 0.932] of its mean against 0.654 [0.551, 0.709] for linear phase interpolation; largest per-frame spread 0.34 [0.14, 0.39] against 22.8 [19.2, 27.0] percent; taken from the companion manuscript's beyond.json, 5 seeds. The measured scope is narrower than the criterion, as the companion manuscript reports: with the fewest frames that keep the peak step per frame below a bound set where case C10's lowest moving trap first reached 0.9, the claim holds on most seeds and not on all; on the long sort of C10 at an equal number of frames the interpolation leads; and on the seeds below the bound the limit is the phase path that the end phases prescribe, not the frame count.

Complexity. The sequence is S warm-started runs, $O(S\,K_f\,N_{\mathrm{pix}}M)$ in the point-trap form, and the method is baked; the source of the interpolation reports a very short time per hologram for its own method on a modern graphics processor, which this product does not rely on.

Limits. The transient model is DinHot's: it compares sequences under one model of the display and gives no absolute intensity of a measured device; the baseline is DinHot's implementation of the published description and not the authors' code. The module is `b2_flicker_bounded`.

6.22 Learned warm start for phase-fixed weighted GS

The third proposed method is a negative result, kept as a result. The hypothesis is that a small permutation-equivariant network over the trap list, predicting the initial trap phases θ_m of phase-fixed weighted GS, lets phase-fixed weighted GS reach a relative standard deviation of one percent in at least two times fewer iterations (median) than from a random start on held-out geometries, with the final efficiency and uniformity equal within one percent and a model of at most one megabyte. The gap is that a U-Net generates a whole phase for about twenty traps in a fraction of a second [30], DeepCGH shows learned speedups for volumes [29], and a hologram network of below 620 kilobytes is possible [31] (abstract only read); no source that was read uses a network to initialize weighted GS, and none compares with it. The risk named before the work was that the random start is already reliable for the weighted GS, so that the gain may be small.

Method. The start is the superposition hologram with the learned trap phases,

$$\phi_j^{(0)} = \arg \sum_m \sqrt{w_m}\, e^{\,i(\Delta_j^m + \theta_m)}, \tag{56}$$

the form of random superposition with learned instead of random phases, and phase-fixed weighted GS runs from it. The network is a permutation-equivariant set transformer: Fourier features of the trap coordinates, an embedding, two attention blocks, a head for the phase and a set-dependent chirp coefficient. Training is unsupervised: the loss is the weighted mean of $\log(\sigma^2 + 0.01)$ of the trap-intensity relative standard deviation σ (in percent) over the start and ten unrolled iterations of phase-fixed weighted GS with an early freeze, the start weighing 0.3 and each iteration 1,

$$\mathcal{L} = \frac{1}{B} \sum_{b=1}^{B} \frac{\sum_{u=0}^{U} \omega_u \log\left(\sigma_{b,u}^2 + \epsilon\right)}{\sum_{u=0}^{U} \omega_u}, \qquad \epsilon = 0.01, \quad \omega_0 = 0.3,\ \omega_{u\geq 1} = 1, \tag{57}$$

through a differentiable copy of the rule that was checked against phase-fixed weighted GS, so that the network learns a start from which phase-fixed weighted GS converges sooner. It is not a regression onto

Algorithm 16 Learned warm start for phase-fixed weighted GS: the network predicts one start phase per trap, and phase-fixed weighted GS runs from the superposition hologram built with them.

Require: traps (at most 1024, no vortex), released network (or none), freeze index N, iteration budget K

if a model is released **then**
    $\theta \leftarrow$ network output for the trap list (coordinates in bins, depths in Rayleigh ranges, weights)
    $\phi^0 \leftarrow \arg \sum_m \sqrt{w_m}\, e^{i(\Delta^m + \theta_m)}$ by Eq. (56)
else
    $\phi^0 \leftarrow$ none (the answer is phase-fixed weighted GS from its own random start, the provenance `baseline`)
end if
return phase-fixed weighted GS of Algorithm 2 from ϕ^0 with freeze index N and budget K (point form)

converged phases, which are not unique (the global phase and the many fixed points of the weighted GS would make the labels ambiguous). The invariance to the global phase is automatic, since the loss reads intensities only. The trap-set generators are lattices, hexagonal sets, rings, random sets with a minimum spacing and three-dimensional stacks; there is no quasi-periodic family, so that geometry of the benchmark is unseen.

Pseudocode. Algorithm 16 lists the inference; training minimizes the loss of Eq. (57) offline.

Evaluation and result. The held-out set was written before training: 66 geometries, generated demanding ones plus the benchmark geometries, with seeds recorded in the training record. The model, with its early freeze, needs a median of 8 iterations to reach one percent against 14 for a random start with the original freeze, a ratio of 1.75 against the required factor of two, and the final efficiency differs by 4.2 percent between the arms against the required one. A control with no network and the early freeze needs a median of 9 iterations on the control subset, and 40 percent of its geometries never reach one percent (47 percent with the network). The early freeze, not the network, shortens the run, so the hypothesis is refuted. An oracle bound explains why: starting from the trap phases of a converged random-start phase-fixed weighted GS solution, the original freeze still needs many iterations on dense lattices, and only a short freeze shortens the run.

Parameters. The module default for the freeze is 12, the value of phase-fixed weighted GS; the model arm in training and in the evaluation uses the early freeze; the form defaults to `point` because three-dimensional traps need it. With no released model the answer is phase-fixed weighted GS from its own random start with the provenance `baseline`.

Acceptance criterion and gate. The ladder row records a measured negative. The gate asserts that the ratio of the medians matches the stored one and lies in the interval from 1.5 to 2.0, that the final efficiency differs by more than one percent, that the control supports the explanation, and that no released warm-start model file exists; a second test holds a released model, if one is ever released, to the original criterion. The model is not released, and the committed record of the refusal is the evidence. The remaining open question, a label-based model or a change of the freeze rule, is a new design question, not tuning, and it is not claimed. The module is `b3_warm_start`.

6.23 Aberration-robust hologram

After a Zernike correction (Section 6.14) residual aberrations remain and vary across the field. The fourth proposed method keeps the trap spread low over the whole distribution of residuals instead of at the nominal point. The hypothesis as narrowed after a measurement is that minimizing the expected trap-intensity loss over a distribution of residual Zernike aberrations gives a hologram whose trap spread over an ensemble of residuals is lower than that of the nominal phase-fixed weighted GS: the 95th percentile of the relative standard deviation over a 200-sample ensemble is at least 1.25 times below that of phase-fixed weighted GS, with the mean efficiency at most ten percent lower. Robustness to pointing errors is an option that is measured and reported and not claimed. The original claim was robustness to aberrations and to trap displacement together, with a factor of two; with the real implementation it failed, and the claim was narrowed. The gap is that aberrations vary across the field and are corrected with one global set of low-order Zernike modes [17]; the nearest work on holograms robust to such errors is [34], and no source that was read optimizes the hologram for a distribution of residual aberration.

Method. The expected loss over residual Zernike coefficients $a_k \sim \mathcal{N}(0, s_a^2)$ for Noll modes 4 to 11, in waves,

Algorithm 17 Aberration-robust hologram: the expected loss over residual Zernike aberrations by the sigma-point design (conjugate gradient) or by random draws (Adam).

Require: traps, prior s_a, level fraction ℓ, warm iterations W, iteration budget K, design, samples q, rate lr, pointing prior

$\phi \leftarrow W$ iterations of phase-fixed weighted GS from the random superposition; level $\leftarrow \ell\times$ efficiency of ϕ
if design is sigma points **then**
  define $F(\phi) = \frac{1}{16}\sum_{k=1}^{8}\sum_{\pm} L\big(\phi \pm 2\pi s_a\sqrt{8}\,Z_k\big)$ and its gradient at the target level level
  return ϕ after K conjugate-gradient iterations on F (Algorithm 5)
else
  for $i = 1, \ldots, K$ **do**
    draw q perturbations (Noll 4 to 11 coefficients $\sim \mathcal{N}(0, s_a^2)$ and independent trap displacements); average the loss and gradient at $\phi + 2\pi W(a)$ over the displaced traps
    Adam step with rate lr (as in Algorithm 6)
  end for
  return ϕ
end if

with piston and tilts excluded, is minimized,

$$\min_{\phi}\ \mathbb{E}_a\, L\big(\phi + 2\pi W(a)\big), \qquad L = \frac{\sum_m (I_m - T_m)^2}{\sum_m T_m^2}, \qquad T_m = \ell\, e_{\mathrm{nom}} \frac{w_m}{\sum w}, \tag{58}$$

with W the wavefront added to the SLM phase as in the total-phase sum of Nogrette et al. [22], e_{nom} the efficiency of the starting hologram and ℓ the target level fraction. The aberration is a phase offset, so the gradient is the unperturbed closed form at the shifted phase. The default design replaces the expectation by a deterministic sigma-point set, plus and minus $s_a\sqrt{8}$ on one mode at a time,

$$\mathbb{E}_a\, L \approx \frac{1}{16}\sum_{k=1}^{8}\sum_{\pm} L\big(\phi \pm 2\pi s_a\sqrt{8}\,Z_k\big), \tag{59}$$

sixteen points of mean zero and covariance exactly $s_a^2 I$ over the eight modes, so that the loss has no sampling noise and a Fletcher–Reeves conjugate gradient converges on it. A second design draws random samples that are resampled at every iteration, with Adam, and adds independent trap displacements; it is selected automatically when pointing errors are requested. The warm start is phase-fixed weighted GS itself.

Pseudocode. Algorithm 17 lists the two designs.

Parameters. The prior s_a is 0.02 waves per mode, a stated choice that is not sourced; the level fraction is 0.9, a measured trade-off (a higher level gives a lower factor, because the level trades efficiency for robustness); the warm start has 30 iterations; the samples per iteration are 8.

Acceptance criterion and gate. Over a 200-sample ensemble the 95th percentile of the spread is at least 1.25 times below that of phase-fixed weighted GS with an efficiency at most ten percent lower (Table 19). The gate runs the 10×10 task with the nominal hologram from the in-test weighted GS and the aberration-robust hologram with 40 iterations, evaluates both on an ensemble of a different seed than the design, and checks the two bounds; a second test only asserts that the pointing case runs and returns finite ratios. The measurement of this report is: 95th percentile of the relative standard deviation a median 1.435 times below phase-fixed weighted GS (1.42 to 1.57) over a 200-sample aberration ensemble, at a median 0.982 of its efficiency; pointing errors 1.04 times (seed 1). The aberration claim holds on every seed. The pointing claim is not obtained in the acceptance configuration (a ratio of 1.04 over three seeds, 0.85 to 1.05); the spread under displacement has a first-order slope term and a second-order curvature floor, and with a displacement prior alone and 100 steps the slope falls about fivefold and the ratio reaches 1.48 at 0.10 bins (1.44 to 1.50) at 0.91 of the efficiency, what remains being close to the curvature floor that the spot width fixes. The ensemble is drawn from the same prior as the design, so robustness to a different distribution or to a measured instrument is not established. In the aberrated case C09 the mean spread of the aberration-robust hologram is below that of phase-fixed weighted GS by a smaller factor than the ensemble claim, from a single realization per seed.

Complexity. Each step costs the number of samples (or sigma points) times one forward and adjoint evaluation of the point-trap model, that is, that number of weighted-GS iterations, and the ensemble evaluation is 200 forward evaluations; the method is baked, and the acceptance test takes about a minute on the development workstation.

Algorithm 18 Efficiency–uniformity front by continuation over λ, warm-started from the neighbouring value.

Require: traps, number of values n_λ, warm iterations W, conjugate-gradient iterations K per value

$\phi \leftarrow W$ iterations of phase-fixed weighted GS from the random superposition; $e_0 \leftarrow$ efficiency of ϕ; $T_m \leftarrow e_0 w_m / \sum_k w_k$
for $\lambda \in \{0, \frac{1}{n_\lambda - 1}, \dots, 1\}$ **do**
    $\phi \leftarrow K$ Fletcher–Reeves conjugate-gradient iterations on L_λ of Eq. (60) from ϕ (Algorithm 5)
    store ϕ and the point (e, u, σ) of its trap fields
end for
non-dominated points $\leftarrow$ the stored points that no other point beats in both e and u; HV $\leftarrow$ Eq. (61)
return the stored hologram of largest $e \cdot u$ (the knee), the front and HV

Limits. The prior is a stated parameter. The benefit depends on the target level and the budget. The model is aberrations and nothing else on the ideal point-trap model. On ideal cases the aberration-robust hologram's spread is a small nominal spread, the price of robustness. The module is `b4_robust`.

6.24 Efficiency–uniformity Pareto front

Every method trades efficiency against uniformity and reports single points of that trade-off: Di Leonardo et al. report the direct search and the adaptive form at stated weights [12], Polin et al. use one scalar weight [15], and MRAF has one mixing parameter. The fifth proposed method computes the whole front on the same task. The hypothesis is that a continuation over a trade-off parameter $\lambda \in [0, 1]$ of one loss, warm-started from the neighbouring value, gives a front whose hypervolume in the efficiency–uniformity plane is at least that of the union of the single points of GS, GAA, direct search and weighted GS, with at least one front point dominating each of GS and GAA, in under a minute on a processor for the 10×10 task. A risk anticipated in the design of the method is that the weighted-GS point is already near the corner, so that the front may be flat and its useful region the large or device-affected cases.

Method. The loss over the continuation is

$$L_\lambda(\phi) = (1 - \lambda) \frac{\sum_m (I_m - T_m)^2}{\sum_m T_m^2} - \lambda \frac{\sum_m I_m}{e_0}, \qquad T_m = e_0 \frac{w_m}{\sum_k w_k}, \tag{60}$$

with $I_m = |V_m|^2$ and e_0 the efficiency of the weighted-GS hologram at the uniform end. DinHot divides the two terms by $\sum T^2$ and e_0 so that both are dimensionless and of order one, which makes the weight λ comparable across tasks; the scaling is part of the method's definition. The start at $\lambda = 0$ is phase-fixed weighted GS refined by the same conjugate gradient, and each next λ is warm-started from its neighbour and solved with Fletcher–Reeves conjugate gradient. The gradient is the closed form $G_m = [2(1-\lambda)(I_m - T_m)/\sum T^2 - \lambda/e_0] V_m$ pulled back by the point-trap adjoint. The metric of the front is the hypervolume, the area of the union of the rectangles $[0, p]$ over the points $p = (e, u)$, with the reference point at the origin,

$$\mathrm{HV} = \sum_i \left(e_{(i)} - e_{(i-1)}\right) \max_{k \geq i} u_{(k)}, \qquad e_{(0)} = 0, \tag{61}$$

over the non-dominated points sorted by efficiency; the dominance relation keeps the rows for which both columns are maximized. The knee returned as the hologram is the point of largest $e \cdot u$.

Pseudocode. Algorithm 18 lists the continuation.

Parameters. The number of values of λ is 20, within the range of ten to twenty set in the design of the method; the warm start is 30 phase-fixed iterations; the conjugate-gradient iterations per λ are the budget (15 in the acceptance test). The ideal device is assumed, so the front does not apply to a device-limited case.

Acceptance criterion and gate. The hypervolume is at least that of the plain GS, the adaptive form, direct search and weighted GS points on the 10×10 task (Table 19). The gate evaluates the baseline points by the metrics, runs the Pareto front with 20 values of λ, and requires the hypervolume bound, at least one front point dominating each of plain GS and the adaptive form, and a run time under a minute. The measurement of this report is: hypervolume 0.8190 against 0.8018 for the union of the GS, generalized adaptive additive GS, weighted GS and direct search points; 7 front points dominate GS and 8 dominate generalized adaptive additive GS. The efficiency changes by only a few percent across the whole front while the uniformity falls from one to a small value, so on this task the trade-off is steep and the useful part of the front is the high-uniformity end, which is the flat-front risk anticipated in the design. In the benchmark the

stored front, the converged run of the first seed, reaches at least the hypervolume of the published points on 0 of the 6 geometries with a front against their medians at 100 iterations on the ideal device, and on 6 against their medians at 30; at 100 iterations it falls short on G1 (0.838 against 0.839), G2 (0.699 against 0.702), G3 (0.708 against 0.711), G4 (0.766 against 0.772), G5 (0.893 against 0.899), G6 (0.791 against 0.798), by at most 0.007. The count at 100 iterations moves between runs, because the medians of a cell depend on the seeds its wall-clock bound admits.

Complexity. The method is n_λ warm-started conjugate-gradient runs of the budget's iterations each, with one forward and one adjoint evaluation per line-search step; the front of the acceptance task takes a fraction of a minute on the development workstation, under the limit of one minute, and the time depends on the machine.

Limits. The criterion is one-sided and the margin is small: the test cannot distinguish a front that is the same as a single good point. The hypervolume depends on the reference point and is dominated by the high-uniformity region. The front is for one task and one setup. The module is `b5_pareto`, whose hypervolume and dominance functions are public for the benchmark.

7 Experimental protocol

7.1 The cases

A case is an optical setup, a device, a target (a list of traps or a shaped potential), an aberration, a method set, a seed and, for dynamic cases, a particle and a rearrangement. There are 14 cases in four categories (Table 17): arrays (a 3×3 grid, a 10×10 grid with camera feedback, a 32×32 array of 1024 traps, the calibration task of the analytic and Gerchberg–Saxton baselines, and a disordered array of 819 traps on integer bins at least five bins apart), shaped and three-dimensional targets (a disc potential with the crosstalk model, a three-plane array with vortices, a vortex ring with a line, and a three-plane volume at the geometry of the released learned model), device and optics (high numerical aperture, a device-limited array, an aberrated array with and without correction) and dynamics and controls (an atom-rearrangement sequence and a trap outside the field, which is refused by the first contract and never computed). The baked cases span 9 to 1024 traps. The optical setups are listed in Table 18; the bead setup is in water, and the setup of C14 is the volume of the released learned model.

Every case states numeric expected ranges for its metrics, measured on three seeds, and the bake fails outside them. All cases are synthetic: the parameters are stated and no measured sample is used. The cases C01, C02, C03, C12 and C13 sit on integer-bin lattices so that the Fourier form applies and the live lane keeps its time budget; the cases C05 to C10 keep off-bin traps because they carry depth, vortices, a high numerical aperture, a device, an aberration or a rearrangement, which the Fourier form cannot represent. A device case excludes traps off the focal plane, vortex charge and an aberration, because the device model is read at one focal plane. The reference method of a case is the one whose maps the replay shows, and the method the reader picks among those valid for the case is a variant, as is the device (ideal or the case's own).

7.2 Metrics, baselines and acceptance criteria

The metrics are those of Eq. (5): the efficiency, the uniformity and the relative standard deviation of the trap intensities, plus the zero-order and ghost fractions of the power budget (reported per case in Table 26), all computed by the scorer from the final phase through the case's device. For a shaped target the two required slots carry the signal-region metrics: the relative-standard-deviation slot holds the root-mean-square fractional error of Eq. (25) in percent after the scale fit, and the uniformity slot holds one minus its fraction, clipped at zero. For the learned volume network the metric is the accuracy of Eq. (42), and the comparison is the median per-volume ratio of the accuracies at equal wall-clock.

The baselines are run by the product on the same task, never only quoted: the analytic and Gerchberg–Saxton methods (from the superposition of prisms and lenses to weighted GS) are the baselines of the weighted methods; GS is the baseline of the optimization methods at equal iterations; the volumetric Gaussian GS is the baseline of the learned network at equal wall-clock; the phase-fixed weighted GS is the baseline of every proposed method; and the random start is the baseline of the warm start. The published values that a method's acceptance criterion quotes are used as anchors, and the report states where the engine reproduces them and where it does not. The acceptance criterion of each method is one measurable statement with one test (Table 19); the measured value of each, with its seeds, is Table 20. A criterion is confirmed when the

test passes, and a method that fails its criterion on DinHot's grids is reported as measured, not hidden: the criterion of the warm start is refuted, the criterion of the aberration-robust hologram was narrowed after a measurement, and the criterion of the freeze index of the phase-fixed method is met only with a larger value than the source's.

Table 19: The methods of the studio and the acceptance criterion the design document fixes for each: the published methods and the methods proposed here.

Method	Acceptance criterion
Published methods	
Superposition of prisms and lenses	on the 10x10 task with a flat beam, efficiency within 0.10 of 0.29 and uniformity below 0.05
Random superposition	on the 10x10 task with a flat beam, efficiency above superposition of prisms and lenses and at least 0.65; uniformity below 0.35 (DinHot measured 0.25 at 256 squared against 0.01 at 768 squared in the source; the ordering, not the value, transfers)
Random mask	efficiency about 1/M; at the source's 768 squared grid, uniformity within the spread of 0.58
Gerchberg-Saxton, point-trap and FFT forms	the two forms agree to 10^{-9} rad; 30 iterations on the 10x10 task with a flat beam reach efficiency at least 0.9 and uniformity near 0.6
Generalized adaptive additive	uniformity above GS at equal iterations on the 10x10 task
Weighted GS	efficiency and uniformity at least 0.9 within a median of 15 iterations on the 10x10 task with a flat beam (the source reports about 10); the 3x3x3 lattice above 0.9 and 0.98
Phase-fixed weighted GS	fewer iterations than weighted GS to a relative standard deviation of 0.5 percent on a 50x30 array, with the phase fixed after 60 iterations in the acceptance run (the code default 'n_fix' is 12, as in Kim et al. 2019; at 12 it stalls at 3.58 percent on DinHot's grid, re-measured 2026-10-30); from the same start phase-fixed weighted GS matches slmsuite 0.5.0's WGS-Kim to 1e-4 percent (the first field is not reweighted, g_0 = 1)
MRAF	on disc, ring and bar targets, signal-region error at least five times below GS at the cost of efficiency
Direct search	uniformity at least 0.98 on the 10x10 task; under 5 percent rms variation on a quasi-periodic set
Conjugate-gradient minimization	the analytic gradient matches finite differences; a square lattice below 1 percent error
Wirtinger gradient descent with Adam	lower trap-intensity error than GS at equal iterations on the 10x10 task
Closed-loop WGS on an emulated camera	from a 19 percent rms open-loop spread, at most 2.1 percent in 20 iterations at gain 0.7; the acceptance runs on the emulated camera ('core/camera.py'), which reproduces the published feedback curves (Nogrette 2014: 19 to 1.4 percent in 20 iterations; Kim 2019: 22 percent to 1.59 plus or minus 0.18 percent in three phase-fixed steps over 8 ensembles, 1.4 in the histogram of one run; the bounds 2.1 and 1.8 are set above them)
Device-aware CG with pixel crosstalk	on the crosstalk device, lower error than conjugate-gradient minimization without the model on a large disc; a staircase line search lets it run its budget on a quantizing device, where its scored spread does not improve systematically (a measured negative)
Zernike correction and sensorless modal recovery	recovers an injected set of Noll modes 4 to 11 within 5 percent of their amplitude
Laguerre-Gaussian vortex traps	the ring phase winds by 2 pi l with an error under 0.1 rad
Hungarian assignment and linear phase interpolation	no crossing moves for alpha 1.5 on a 7x7 to 3x3 sort; a moving trap above 70 percent of its mean intensity on the frames themselves (the minimum over the frames of the sequence over their mean; between frames, under the blend model of flicker-bounded phase-continuous sequence's transient, it is not: notebook 07-04)
Multi-plane CNN hologram with an optional volumetric GS refinement (DeepCGH-style)	on 162 held-out three-plane volumes, the hybrid (the network's phase, refined by volumetric 3D GS inside the same wall-clock budget only when that raises the volume accuracy) reaches a higher accuracy than volumetric 3D GS alone at that budget (measured median ratio 1.140); the refinement was kept on none of the volumes, so the measured method is the network alone
Optimal-transport initialized MRAF	lower error than MRAF from a random start on the shaped targets
Multi-plane GS	on a three-plane target, every plane mean within 25 percent of the overall mean (volumetric 3D GS of Whyte and Courtial)

Method	Acceptance criterion
Methods proposed here	
Device-aware weighted GS	on the crosstalk device, relative standard deviation at least two times below phase-fixed weighted GS, efficiency at most 10 percent lower; measured against the corrected phase-fixed weighted GS and against slmsuite's WGS-Kim alike: a median of 25.8 times (15.9 to 33.3 over five seeds) at 0.995 of the efficiency, and 2.0 times on the full C08 device (the bake, three seeds; 2.2 in notebook 03-04) with 4-bit phase and flicker; in the benchmark, on the C08 device at 100 iterations with the default parameters, device-aware weighted GS meets both thresholds on 2 of 5 geometries (G2 ring of 36 traps, 2.28 times at 0.965 of the efficiency, G3 random 100 traps, 4.69 times at 0.938 of the efficiency); lowers the spread without the factor of two on G6 (1.73 times at 1.06); keeps its start on G1 (0.97), G4 (1.05); device-aware weighted GS's cells hold 2 seeds
Flicker-bounded phase-continuous sequence	minimum transient intensity of a moving trap at least 0.9 of its mean, against LPI on the same assignment: with the frames that keep the peak step per frame at most 0.2 Fourier units (a bound set on C10's sort) it holds on most seeds (acceptance task, 13 frames at 0.31 Fourier units per frame, seeds 1 to 3: flicker-bounded phase-continuous sequence 0.916 to 0.926, LPI 0.551 to 0.709; long lattice moves: median 0.933 and 0.936, 6 of 8 seeds, and flicker-bounded phase-continuous sequence above LPI on 7 of 8 seeds of each); on C10's long sort at 25 frames LPI leads in every seed, and flicker-bounded phase-continuous sequence needs the end phases and 137 frames to reach 0.902 to 0.917 on 5 of 6 seeds of the plan of the Hungarian assignment with phase interpolation (median 0.911; LPI 0.360 to 0.690), with one seed collapsing on its phase path
Learned warm start for phase-fixed WGS (refuted)	measured negative: on held-out geometries the network start needs a median 8 iterations to 1 percent against 14 from a random start (1.75 times, below the 2 times claimed), and a control with the same early phase fix and no network gains as much; the model is not released and the claim learned warm start is reported as refuted
Aberration-robust hologram	over a 200-sample ensemble of residual aberrations, the 95th percentile of the relative standard deviation at least 1.25 times below phase-fixed weighted GS, efficiency at most 10 percent lower (measured: a median of 1.435 times, 1.42 to 1.57 over three seeds, at 0.982 of the efficiency); robustness to pointing errors is measured and reported, not claimed (1.04 times at the acceptance budget; 1.48 times at 0.10 bins and 0.912 of the efficiency with a displacement prior alone and five times the budget)
Efficiency-uniformity Pareto front	hypervolume at least that of the GS, generalized adaptive additive GS, direct search and weighted GS points on the 10x10 task (measured: 0.8190 against 0.8018); in the benchmark the stored front (efficiency-uniformity Pareto front's converged run of the first seed) reaches that hypervolume on none of the 6 geometries with a front against the medians at 100 iterations on the ideal device, below them by at most 0.007, and on all 6 against those at 30 (the count at 100 iterations moves between runs of the benchmark, 3, 4 and 0, because a cell's medians depend on the seeds its wall-clock bound admits), 2026-10-31 run

Table 20: Measured acceptance of the methods on DinHot's own grids (unit tests; seeds as stated). Intervals are [min, max].

Method	Measured	Seeds
Superposition of prisms and lenses	efficiency 0.281, uniformity 0.0003 (deterministic)	1
Random superposition	efficiency 0.750 [0.738, 0.752], uniformity 0.252 [0.106, 0.272]	6
Random mask	256 squared: efficiency 0.010, uniformity 0.057 [0.031, 0.072]; 768 squared: efficiency 0.008, uniformity 0.566 [0.543, 0.590]	5 and 2
Gerchberg-Saxton	flat beam: efficiency 0.939 [0.933, 0.939], uniformity 0.604 [0.557, 0.626]; Gaussian beam: efficiency 0.815 against its ceiling 0.869	3
Generalized adaptive additive GS	uniformity 0.688 [0.638, 0.709] against 0.615 [0.550, 0.637] for GS at equal iterations; efficiency 0.811 against 0.816	5
Weighted GS	flat beam: efficiency and uniformity both at 0.9 after 12, 15 and 12 iterations (median uniformity 0.857 at 10); Gaussian beam: 12, 15 and 12; 3 x 3 x 3 lattice (0.9 and 0.98): 35, 20 and 27; budgets 15, 20 and 100	3
Phase-fixed weighted GS	50 x 30 array to 0.5 percent in 64 and 64 iterations against 136 and 106 for weighted GS (phase fixed after 60; efficiency 0.83 and 0.83); with the source value of 12 it stalls at 3.58 percent	2

Method	Measured	Seeds
Mixed-region amplitude freedom	signal-region error 12.4 (disc), 7.8 (ring) and 10.4 (bar) times below GS at the worse of two starts; efficiency 0.181 to 0.184 against 0.963 to 0.992	2 starts
Direct search	uniformity 0.9994 and 0.9992 (efficiency 0.66 and 0.65); 119 quasi-periodic traps at 0.023 percent rms	2 and 1
Conjugate-gradient minimization	analytic gradient within 2.5e-06 of finite differences; square lattice error 0.55 and 0.51 percent	2
Wirtinger gradient descent with Adam	relative standard deviation 2.53, 2.46 and 2.26 percent against 17.78, 17.98 and 18.82 for GS at 30 iterations	3
Closed-loop weighted GS	emulated camera loop: open-loop 20.9 and 20.4 percent to 0.84 and 1.07 percent in 20 iterations; 24.3 and 23.6 percent to 1.05 and 0.74 percent in three phase-fixed steps	2
Device-aware CG with pixel crosstalk	error 0.241 (efficiency 0.535) against 1.382 (0.507) for conjugate-gradient minimization without the device model, large disc on the crosstalk device	1
Zernike correction and sensorless modal recovery	worst per-mode relative error 0.002 [0.001, 0.002] over Noll modes 4 to 11	3
Laguerre-Gaussian vortex traps	ring phase winds by 2 pi l (winding error below the 1e-9 rad of the arithmetic) with a maximum error of 0.033 [0.028, 0.060] rad (isolated vortices of charge 1, 2, 3 and -2)	2 per charge
Hungarian assignment and linear phase interpolation	0 path crossings in 60 loadings; moving traps keep 0.767 [0.747, 0.792] of their mean intensity (worst moving trap of each of 5 loadings)	60 loadings
Multi-plane CNN hologram	three-plane network refined by volumetric 3D GS: median accuracy ratio 1.140 (quartiles 1.103 and 1.163) on 162 held-out volumes; taken from the committed metrics.json, not remeasured here	162 volumes
Optimal-transport initialized MRAF	signal-region error 0.0204 (disc), 0.0179 (ring) and 0.0221 (line) against 0.250, 0.188 and 0.223 for MRAF from a random start (the better of two); baked size: 0.0259 (disc) and 0.0171 (ring) against 0.266 and 0.228	2 and 1
Volumetric 3D GS	plane means over overall mean 0.938, 0.987 and 1.075 and 0.926, 1.034 and 1.039 on three planes	2
Device-aware weighted GS	crosstalk device, 10 x 10 task: relative standard deviation 0.20 [0.16, 0.30] percent against 5.21 [4.75, 5.46] for phase-fixed weighted GS at 100 iterations (a factor 26 [16, 33]) at 0.995 [0.991, 0.997] of its efficiency; 4-bit device with fill 0.85: 0.72 [0.60, 1.16] against 5.73 [5.13, 5.88] (a factor 7.9 [5.0, 8.6]) at 0.990 [0.983, 0.996] of its efficiency; taken from the companion manuscript's beyond.json, 5 and 3 seeds	5 and 3
Flicker-bounded phase-continuous sequence	4 x 4 lattice, two columns moved by four bins in 13 frames: lowest transient intensity of a moving trap 0.920 [0.915, 0.932] of its mean against 0.654 [0.551, 0.709] for linear phase interpolation; largest per-frame spread 0.34 [0.14, 0.39] against 22.8 [19.2, 27.0] percent; taken from the companion manuscript's beyond.json, 5 seeds	5
Aberration- and pointing-robust hologram	95th percentile of the relative standard deviation a median 1.435 times below phase-fixed weighted GS (1.42 to 1.57) over a 200-sample aberration ensemble, at a median 0.982 of its efficiency; pointing errors 1.04 times (seed 1)	3 (pointing 1)
Efficiency-uniformity Pareto front	hypervolume 0.8190 against 0.8018 for the union of the GS, generalized adaptive additive GS, weighted GS and direct search points; 7 front points dominate GS and 8 dominate generalized adaptive additive GS	1

7.3 Seeds, repetitions, hardware and run times

Every method of a case receives the same seed, so that all of them start from the same random numbers. The acceptance tests state their seeds in Table 20: between one and six, the number a test needs to bound a measurement within the time of the test suite. The expected ranges of the cases are measured on three seeds. The benchmark repeats the methods on its geometries over up to 20 seeds; a group runs seeds until a bound on its wall clock, and at least two, so a cell carries between 2 and 20 seeds, which the file records per row. The uncertainty reported is the observed range over the seeds for the acceptance tests and the bake, and the median with the inter-quartile range for the benchmark; no parametric interval is claimed, because the seeds are few and the distributions are not Gaussian.

The measuring machine is the development workstation recorded in the product's file of measured resources: a laptop processor with twelve cores, forty-eight gigabytes of memory and a laptop graphics processor, Python 3.13 and Node 24, with the browser measurements taken in Chromium driven by Playwright. The canonical bake of the eleven cases that run in parallel took 515 s on an idle machine and 761 s while the machine ran other jobs, at a peak of 322 MB for one case process; the benchmark and the training are

bounded by the design at hours and run locally, and the times in the benchmark are wall-clock seconds of the machine that ran it, under shared load, so they differ between runs and are used only to compare methods inside one run. The browser timings of Section 5.5 are quoted from the same record.

7.4 The benchmark protocol

The benchmark compares the methods on one protocol. Its geometries are (7 in all) a 10×10 square, a ring of 36 traps, 100 random traps with a minimum spacing, a quasi-periodic set of 119 traps, a $5 \times 5 \times 3$ lattice, a large set (the 32×32 lattice and the disordered 819-trap set, pooled by seed parity) and four shaped fields (a ring, a disc, a line and a Laguerre–Gaussian ring, pooled by seed). The grids are 128, 256 pixels, with the benchmark optics keeping the Fourier unit and the focal bin fixed across grids; the iteration budgets are 10, 30, 100 and the time budgets are 0.05, 1 s, with a converged row run to a cap of iterations. For every method, geometry, device and parameter set and for every seed, six runs are made; every cell reports the median and the inter-quartile range of efficiency, uniformity, relative standard deviation, time and iterations, on the ideal device and on the C08 device (every hologram scored against its device). The device rows run with the default parameters only and not on the multi-plane geometries.

A method that declares a tuning grid of four candidates, or has a fallback entry whose names its run function accepts, is tuned: each candidate runs on three other seeds at 30 iterations and the one with the lowest median relative standard deviation among those with at least half the best median efficiency gives the tuned rows. The number of tuning runs is the same for every tuned method, and no row mixes one method's tuned parameters with another's defaults, as the parameter field of each row states. The committed results file holds 1326 rows of 21 of the 24 engine methods; 61 method–geometry cells are recorded as skipped with the reason the file states (a method that does not accept the target, such as a two-dimensional method on a three-dimensional lattice, or one that refuses the geometry), and the 3 methods without a row (Laguerre-Gaussian vortex traps, Hungarian assignment with phase interpolation, multi-plane CNN hologram) refuse every geometry of the protocol by their specification (a vortex charge, a rearrangement block, the geometry the learned model was trained for); their tests and their cases measure them.

7.5 Evaluation of the learned models

The two learned methods are evaluated only on held-out geometries that the training never saw. The held-out set is written before training and its hash is recorded in the model card. The metrics document compares each model with its baseline: the volumetric GS at the same wall-clock budget for the network (the budget is the median time of the baseline at 30 iterations on those volumes), and the random start for the warm start, with the same early freeze as a control without the network. A model that does not meet its criterion is not released: the baseline answers, readiness says training, and the measured values are reported as they are. The rules that keep the evaluation sound are stated before the results: held-out means disjoint from calibration and training; the acceptance values of each method state seeds, grid and date; accurate means that the result agrees with the stated engine and device model, not that it matches a real instrument; and a published number is labelled with its source and conditions while a DinHot measurement is labelled as one.

8 Results

8.1 Baked cases

Each case is computed offline with every method valid for it (the seed is stated in each manifest), checked against its expected ranges, and committed as a manifest and a trace of at most 256 KiB. Table 21 compares the calibration case with the published rows of Di Leonardo et al. [12]: the uniformity order of the Gerchberg–Saxton family (plain GS, the adaptive form, weighted GS) and the weighted-GS uniformity are reproduced, as are the lowest uniformity of the superposition and the lowest efficiency of the random mask, while the order of the two random baselines is reversed (random superposition has the higher uniformity here, 0.183 against 0.063, and the lower in the source, 0.01 against 0.58); the efficiencies of the GS family are lower than published because the engine's Gaussian input beam lowers the efficiency below the flat-beam value by the scale of the single-trap reference level, whereas the source uses a flat beam (the flat-beam test of plain GS gives 0.939 efficiency and 0.604 uniformity; Table 20). The uniformity of random superposition and of the random mask differs from the published values on the engine's smaller grid, as the acceptance test of those

methods states. The table also gives the phase-fixed weighted GS, which has no published row on this task and leaves a mean spread of 0.51 percent over the three seeds, with a seed range from 0.01 to 1.48 percent.

Table 22 summarizes the baked cases. Where both ran (9 cases of the bake, among them the device-limited case C08 and the aberrated case C09), phase-fixed weighted GS leaves a median relative standard deviation of 0.38 percent against 17 percent for plain GS, each the median over the cases of the mean over the three seeds of the bake; the medians of the single seeds range from 0.014 to 0.29 percent and from 16 to 19 percent, so that the first seed of the bake, the one the manifests hold, gives a lower value for the phase-fixed form (0.01 percent) than the mean of the three. The mean efficiency of the phase-fixed form lies between 0.447 (the aberrated case) and 0.794. The full matrix of the bake is Table 23 for the spread and Table 24 for the efficiency, each cell the mean over the three seeds of the canonical bake with its range. Over the 12 point-trap cases, the weighted GS has a lower mean spread than GS in 11 of the 11 cases where both ran; over the 9 cases where plain GS, weighted GS and the phase-fixed form all ran (one common set, so that the medians are comparable), the median over the cases of the mean spread is 17 percent for GS (efficiency 0.80), 1.1 percent for weighted GS (0.80) and 0.38 percent for the phase-fixed form (0.79), so that the integral feedback of the weights removes more than an order of magnitude of the spread and the freeze of the phase a further factor of a few, at an efficiency that stays within a few hundredths. The analytic baselines show the reverse: the superposition and the random superposition leave the largest spreads, the random mask trades its efficiency (about the inverse of the trap count, Table 24) for a moderate spread that depends on the pixels per trap, and the gradient method conjugate-gradient minimization at the 30-iteration budget of the bake is far from converged, as its method section states. Direct search reaches the smallest spreads on the ideal-device lattices it runs on, at a lower efficiency than the weighted forms, while the knee of the proposed front reaches a spread of the order of the phase-fixed form at the efficiency of the weighted form; the flicker-bounded sequence is scored on its final hologram on the rearrangement case only.

Table 23: Relative standard deviation of the trap intensities (percent), mean over the three seeds of the canonical bake with the minimum and maximum over the seeds in brackets, every method on every point-trap case it ran on (three blocks of cases); blank cells are methods the case does not run.

Method	C01	C02	C03	C05	C06
Superposition of prisms and lenses	64 [64, 64]	274 [274, 274]	925 [925, 925]	79 [79, 79]	27 [27, 27]
Random superposition	35 [26, 48]	40 [36, 42]	39 [38, 39]	43 [42, 44]	9.5 [8.2, 12]
Random mask	4.4 [3.2, 5.7]	54 [52, 55]			
Gerchberg-Saxton	17 [7.3, 24]	17 [16, 19]	18 [17, 19]	19 [17, 22]	8.2 [7.3, 9.3]
Generalized adaptive additive GS	8.6 [3.9, 17]	13 [12, 15]			
Weighted GS	0.009 [0.0023, 0.019]	1.2 [0.86, 1.5]	1.1 [0.99, 1.3]	1.2 [0.5, 2.3]	0.13 [0.083, 0.17]
Phase-fixed weighted GS	0.0035 [<0.001, 0.0096]	0.38 [0.014, 0.85]	0.42 [0.2, 0.66]		<0.001 [<0.001, <0.001]
Direct search	<0.001 [<0.001, <0.001]	0.014 [0.013, 0.014]		0.0064 [0.0061, 0.0068]	0.0024 [0.0023, 0.0026]
Conjugate-gradient minimization	52 [44, 63]	41 [38, 44]		53 [47, 62]	45 [36, 51]
Wirtinger gradient descent with Adam	1.3 [0.49, 1.9]	2.5 [2.4, 2.6]		1.8 [1.8, 1.8]	
Closed-loop weighted GS	18 [18, 19]	20 [19, 21]	20 [19, 20]		
Device-aware CG with pixel crosstalk					
Zernike correction and sensorless modal recovery					
Laguerre-Gaussian vortex traps				1.2 [0.5, 2.3]	0.13 [0.083, 0.17]
Multi-plane CNN hologram					
Volumetric 3D GS				62 [62, 63]	
Device-aware weighted GS					
Flicker-bounded phase-continuous sequence					
Learned warm start for phase-fixed WGS	0.0033 [<0.001, 0.0098]	0.38 [0.014, 0.85]	0.42 [0.2, 0.67]		
Aberration- and pointing-robust hologram	0.59 [0.42, 0.77]	0.85 [0.83, 0.87]			
Efficiency-uniformity Pareto front	0.031 [<0.001, 0.093]	0.53 [<0.001, 1.1]		0.66 [0.51, 0.82]	

Method	C07	C08	C09	C10

Superposition of prisms and lenses	116 [116, 116]		78 [78, 78]	
Random superposition	38 [33, 42]			
Random mask	15 [13, 17]			
Gerchberg-Saxton	18 [13, 22]	24 [22, 28]	15 [15, 16]	
Generalized adaptive additive GS	14 [13, 17]			
Weighted GS	0.76 [0.39, 1.3]	14 [12, 14]	6 [4.8, 7.3]	0.98 [0.92, 1.1]
Phase-fixed weighted GS	0.004 [<0.001, 0.0089]	13 [13, 13]	6.2 [4.7, 7.1]	0.022 [0.012, 0.043]
Direct search	0.0029 [0.0025, 0.0036]			
Conjugate-gradient minimization	63 [56, 71]	32 [30, 35]		
Wirtinger gradient descent with Adam	2.5 [2, 3.2]			
Closed-loop weighted GS				
Device-aware CG with pixel crosstalk		25 [11, 41]		
Zernike correction and sensorless modal recovery			0.33 [0.077, 0.68]	
Laguerre-Gaussian vortex traps				
Multi-plane CNN hologram				
Volumetric 3D GS				
Device-aware weighted GS		6.5 [5.9, 7]		
Flicker-bounded phase-continuous sequence				0.02 [<0.001, 0.061]
Learned warm start for phase-fixed WGS	0.004 [<0.001, 0.0089]			
Aberration- and pointing-robust hologram	0.98 [0.94, 1]	13 [13, 14]	5.3 [4.6, 5.8]	
Efficiency-uniformity Pareto front	0.54 [0.25, 0.99]			

Method	C12	C13	C14
Superposition of prisms and lenses	269 [269, 269]	25 [25, 25]	69 [69, 69]
Random superposition	41 [37, 44]	14 [13, 14]	26 [24, 28]
Random mask	51 [47, 54]		
Gerchberg-Saxton	18 [16, 20]	11 [11, 11]	17 [15, 18]
Generalized adaptive additive GS	13 [12, 14]		
Weighted GS	1.1 [1, 1.4]	0.66 [0.59, 0.74]	0.49 [0.32, 0.59]
Phase-fixed weighted GS	0.51 [0.013, 1.5]	0.26 [0.18, 0.3]	
Direct search			
Conjugate-gradient minimization			30 [26, 32]
Wirtinger gradient descent with Adam			2 [1.9, 2.1]
Closed-loop weighted GS		19 [19, 20]	
Device-aware CG with pixel crosstalk			
Zernike correction and sensorless modal recovery			
Laguerre-Gaussian vortex traps			
Multi-plane CNN hologram			36 [36, 36]
Volumetric 3D GS			16 [15, 18]
Device-aware weighted GS			
Flicker-bounded phase-continuous sequence			
Learned warm start for phase-fixed WGS		0.26 [0.18, 0.3]	
Aberration- and pointing-robust hologram			
Efficiency-uniformity Pareto front			

Table 24: Efficiency of the same runs as Table 23: mean over the three seeds of the canonical bake with the range over the seeds in brackets, the fraction of the input power in the traps, scored on the case's own device.

Method	C01	C02	C03	C05	C06
Superposition of prisms and lenses	0.45 [0.45, 0.45]	0.28 [0.28, 0.28]	0.19 [0.19, 0.19]	0.45 [0.45, 0.45]	0.59 [0.59, 0.59]
Random superposition	0.72 [0.71, 0.74]	0.74 [0.72, 0.75]	0.74 [0.74, 0.74]	0.77 [0.73, 0.78]	0.69 [0.69, 0.70]
Random mask	0.09 [0.09, 0.09]	0.01 [0.01, 0.01]			
Gerchberg-Saxton	0.80 [0.78, 0.82]	0.82 [0.81, 0.82]	0.82 [0.82, 0.82]	0.91 [0.90, 0.92]	0.71 [0.70, 0.71]
Generalized adaptive additive GS	0.81 [0.78, 0.82]	0.81 [0.81, 0.82]			
Weighted GS	0.80 [0.77, 0.82]	0.80 [0.80, 0.81]	0.81 [0.80, 0.81]	0.89 [0.89, 0.90]	0.70 [0.70, 0.71]
Phase-fixed weighted GS	0.79 [0.77, 0.82]	0.79 [0.79, 0.80]	0.79 [0.79, 0.79]		0.70 [0.70, 0.70]
Direct search	0.65 [0.63, 0.69]	0.65 [0.64, 0.66]		0.68 [0.66, 0.70]	0.64 [0.64, 0.64]
Conjugate-gradient minimization	0.17 [0.15, 0.18]	0.26 [0.25, 0.27]		0.21 [0.21, 0.21]	0.16 [0.15, 0.17]
Wirtinger gradient descent with Adam	0.79 [0.78, 0.79]	0.76 [0.76, 0.76]		0.82 [0.81, 0.83]	
Closed-loop weighted GS	0.81 [0.81, 0.81]	0.81 [0.80, 0.81]	0.81 [0.81, 0.81]		
Device-aware CG with pixel crosstalk					
Zernike correction and sensorless modal recovery					
Laguerre-Gaussian vortex traps				0.89 [0.89, 0.90]	0.70 [0.70, 0.71]
Multi-plane CNN hologram					
Volumetric 3D GS				0.53 [0.51, 0.54]	
Device-aware weighted GS					
Flicker-bounded phase-continuous sequence					
Learned warm start for phase-fixed WGS	0.79 [0.77, 0.82]	0.79 [0.79, 0.80]	0.79 [0.79, 0.79]		
Aberration- and pointing-robust hologram	0.78 [0.76, 0.81]	0.78 [0.77, 0.78]			
Efficiency-uniformity Pareto front	0.80 [0.77, 0.82]	0.80 [0.80, 0.81]		0.89 [0.89, 0.90]	

Method	C07	C08	C09	C10
Superposition of prisms and lenses	0.35 [0.35, 0.35]		0.21 [0.21, 0.21]	
Random superposition	0.73 [0.71, 0.75]			
Random mask	0.04 [0.04, 0.04]			
Gerchberg-Saxton	0.81 [0.81, 0.81]	0.70 [0.69, 0.70]	0.46 [0.46, 0.46]	
Generalized adaptive additive GS	0.80 [0.80, 0.80]			
Weighted GS	0.80 [0.79, 0.80]	0.69 [0.68, 0.70]	0.45 [0.44, 0.45]	0.80 [0.80, 0.81]
Phase-fixed weighted GS	0.79 [0.78, 0.79]	0.68 [0.67, 0.68]	0.45 [0.44, 0.45]	0.79 [0.79, 0.80]
Direct search	0.64 [0.63, 0.65]			
Conjugate-gradient minimization	0.19 [0.19, 0.20]	0.31 [0.30, 0.32]		
Wirtinger gradient descent with Adam	0.77 [0.76, 0.77]			
Closed-loop weighted GS				
Device-aware CG with pixel crosstalk		0.42 [0.25, 0.55]		
Zernike correction and sensorless modal recovery			0.80 [0.79, 0.81]	
Laguerre-Gaussian vortex traps				
Multi-plane CNN hologram				
Volumetric 3D GS				
Device-aware weighted GS		0.65 [0.65, 0.66]		
Flicker-bounded phase-continuous sequence				0.79 [0.79, 0.80]
Learned warm start for phase-fixed WGS	0.79 [0.78, 0.79]			

Aberration- and pointing-robust hologram	0.77 [0.76, 0.77]	0.67 [0.67, 0.67]	0.44 [0.43, 0.45]
Efficiency-uniformity Pareto front	0.80 [0.79, 0.80]		

Method	C12	C13	C14
Superposition of prisms and lenses	0.33 [0.33, 0.33]	0.60 [0.60, 0.60]	0.50 [0.50, 0.50]
Random superposition	0.74 [0.72, 0.75]	0.69 [0.69, 0.69]	0.70 [0.69, 0.70]
Random mask	0.01 [0.01, 0.01]		
Gerchberg-Saxton	0.82 [0.81, 0.82]	0.74 [0.74, 0.74]	0.80 [0.80, 0.80]
Generalized adaptive additive GS	0.81 [0.81, 0.82]		
Weighted GS	0.80 [0.80, 0.80]	0.74 [0.74, 0.74]	0.79 [0.78, 0.79]
Phase-fixed weighted GS	0.79 [0.79, 0.80]	0.72 [0.72, 0.73]	
Direct search			
Conjugate-gradient minimization			0.31 [0.30, 0.34]
Wirtinger gradient descent with Adam			0.75 [0.75, 0.76]
Closed-loop weighted GS		0.74 [0.74, 0.74]	
Device-aware CG with pixel crosstalk			
Zernike correction and sensorless modal recovery			
Laguerre-Gaussian vortex traps			
Multi-plane CNN hologram			0.40 [0.40, 0.40]
Volumetric 3D GS			0.75 [0.75, 0.75]
Device-aware weighted GS			
Flicker-bounded phase-continuous sequence			
Learned warm start for phase-fixed WGS		0.72 [0.72, 0.73]	
Aberration- and pointing-robust hologram			
Efficiency-uniformity Pareto front			

The seed-to-seed uncertainty of the bake is read in Table 25. On the ideal-device grid C02 the weighted methods differ from seed to seed in a spread that is already near or below one percent (the range of the phase-fixed form spans more than an order of magnitude), so a one-seed ranking among them would be fragile; on the device-limited case C08 the ranges are narrow and the methods fall into groups: weighted GS, phase-fixed weighted GS and the aberration-robust hologram keep a spread of the order of ten percent, and the device-aware the device-aware weighted GS lowers it. On C08 phase-fixed weighted GS (phase-fixed weighted GS), which ignores the device, keeps a spread of 13.4 percent, and the device-aware weighted GS (the device-aware weighted GS) lowers it to 7.0 percent, a factor of 1.9 on the bake's own seed (2.0 on the ratio of the means of its 3 seeds), on that small grid (the larger task of the acceptance gate gives a larger factor; both are reported in the companion manuscript [45]). On the aberrated case C09, correction of the Zernike modes (the Zernike correction) restores efficiency and uniformity that the uncorrected methods lose; the proposed aberration-robust hologram scores a lower mean spread than the nominal method on one realization per seed, a smaller factor than its ensemble criterion.

Figure 4 shows the same matrix of three-seed means as a heat map, which makes the structure visible: the rows of the weighted and optimization methods are dark (low spread) on every ideal-device case where they run, the rows of the analytic baselines are light, and the device-limited and aberrated columns are lighter for every method, which is the cost of the device and of the aberration that the device-aware and robust methods target. Figure 5 follows the spread along the iterations from the baked traces. Each curve is the history that the method reports for itself, which is the quantity the method optimizes. On the ideal-device grid the curves of the weighted and phase-fixed forms fall by orders of magnitude over the budget, GS plateaus at a spread of the order of ten percent, the adaptive form plateaus a little lower, and the Wirtinger descent starts at a large spread and falls steadily. On the device-limited case the picture differs by what each method sees. The weighted and phase-fixed forms keep falling, because their history is measured on the ideal model that

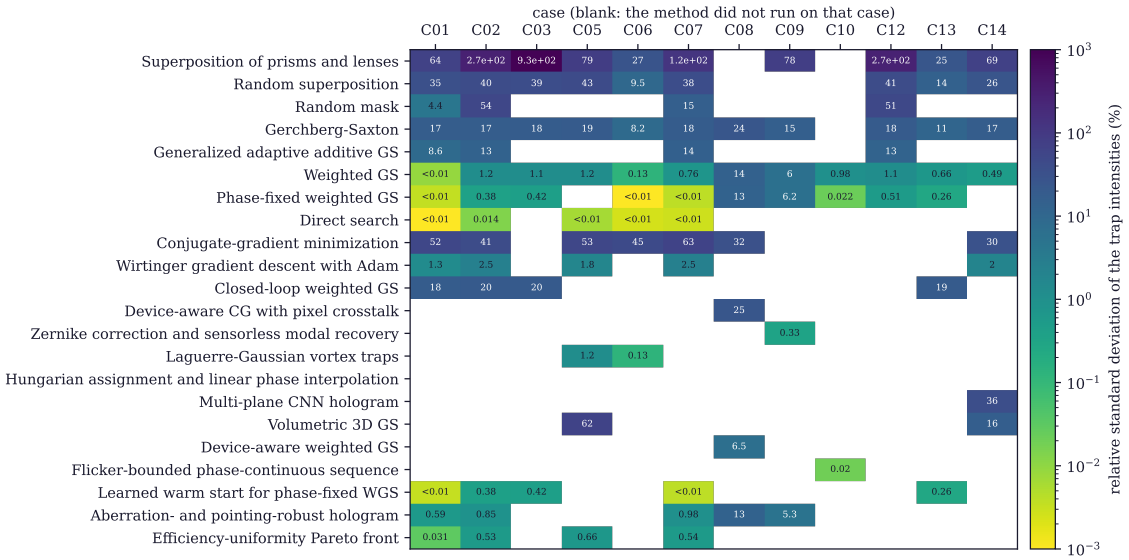


Figure 4: Relative standard deviation of the trap intensities, every method on every point-trap case of the bake (the shaped-potential case C04 is scored on its signal region and is omitted). Each cell is the mean over the three seeds of the canonical bake of one method at the case budget (the matrix of Table 23, not the single bake seed of the manifests). Plain Gerchberg–Saxton leaves the largest spreads among the iterative methods; the weighted and phase-fixed forms, the direct search, the Pareto front and the device-aware methods remove most of it where they apply.

they optimize, while the scored spread on the device is the value of Table 23, of the order of ten percent: this is the gap between equalizing the model and equalizing the device that the device-aware method addresses. The device-aware weighted GS reports its history on the device model and settles at a floor set by that model, and the conjugate-gradient methods are still far from converged at the budget of the bake.

Figure 6 shows what the numbers describe in the focal plane: the hologram of the reference method on C02 is a quantized phase mask whose focal intensity is a regular lattice with a faint structure between the traps, and the device-aware hologram on the device-limited case forms traps that are visible but not equal, because the quantizer and the flicker are not modelled by the optimizer. The figure shows one hologram per panel and does not show how the maps change with the method, because the trace carries the maps of the reference method only.

The power budget of the point-trap cases (Table 26) separates the light that reaches the traps, the zero order and the ghost traps. On the lattices of cases C02, C03, C07, C08, C12 and C13 the ghost traps hold about an eighth to a fifth of the input power (0.12 to 0.22) for both plain GS and the phase-fixed form, and on the ring-and-line case C06 and the aberrated case C09 they hold 0.42 to 0.46; the zero order is nil except on the nine-trap case C01 (0.09 to 0.11) and on the cases with a high numerical aperture and a device (0.03 and 0.01). The phase-fixed form, which equalizes the traps, sends 0.002 to 0.023 more of the power to ghost traps than plain GS on every case, so that its higher uniformity is paid for by a small loss of efficiency and not by a redistribution to the zero order. The table is read at the bake seed of each case, one seed, and is a description and not an estimate with an interval.

8.2 Feedback and particles

The emulated camera loop reduces the spread as in the published curves. In the baked traces the relative standard deviation falls from 9.1 to 0.02 percent in 5 steps on C01, from 20.1 to 1.34 percent in 20 steps on the 10×10 grid C02, from 22.3 to 1.44 percent in 3 steps on the large array C03, and from 22.0 to 0.44 percent in 5 steps on the disordered array C13 (Figure 7). The published closed-loop result of Nogrette et al. [22] is a trap-depth spread of 19 percent rms reduced to 1.4 percent in 20 iterations; the unit test of the closed-loop method reproduces it within the factor set in the design document (Table 20). The loop of the cases differs from the closed-loop method itself: it applies the same factor to the case's weight-aware method and reports the measured spread of the camera readings, which is why the curves of Figure 7 reach a fraction of a percent while the true intensities keep the size of the camera's own gain error. The figure shows the measured spread and not the true one, so it shows that the loop equalizes what the camera sees, which is

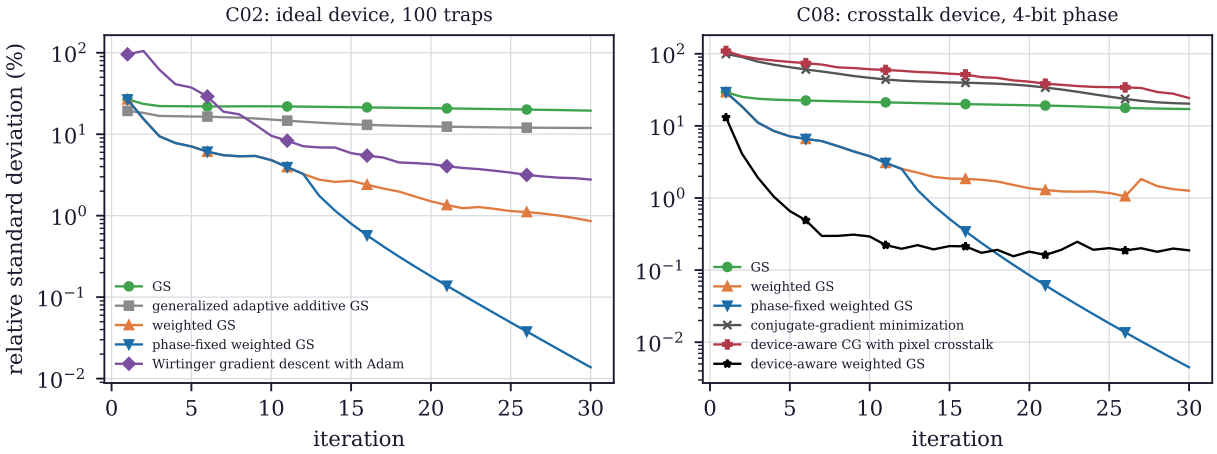


Figure 5: Relative standard deviation of the trap intensities against the iteration, read from the baked traces: (a) the ideal-device grid C02 for GS, the adaptive form, weighted GS, phase-fixed weighted GS and Wirtinger descent; (b) the device-limited array C08 for the same family, the conjugate gradient, its device-aware form and the device-aware weighted GS. Each curve is the history the method reports for itself: the ideal model for GS, the adaptive form, weighted GS, phase-fixed weighted GS and the conjugate gradient, the device model for the device-aware conjugate gradient and the device-aware weighted GS. The vertical axis is logarithmic.

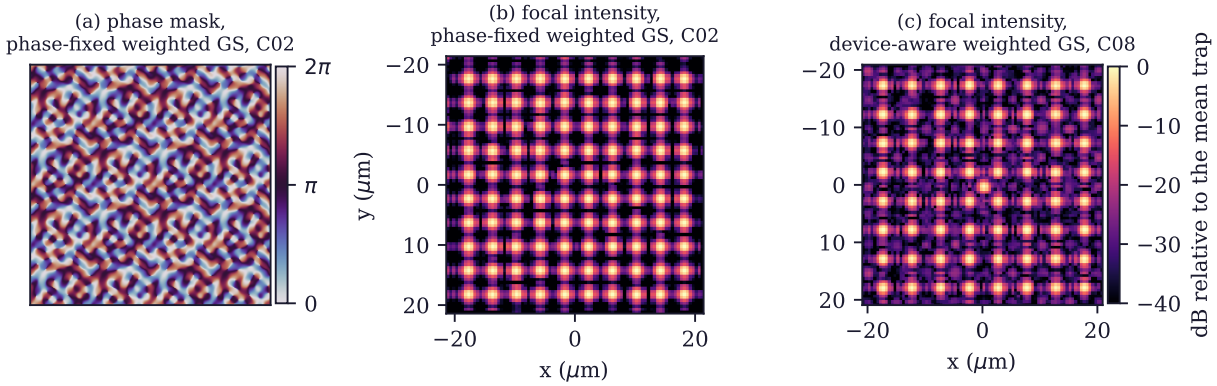


Figure 6: (a) Phase mask of the reference method on C02. (b) Its focal intensity, in dB relative to the mean trap, on the ideal device: a regular lattice. (c) The focal intensity of the device-aware weighted GS on C08, scored on the device with 4-bit phase, crosstalk and a fill factor of 0.85: the traps are visible and not equal.

the property of the published procedure.

The particle simulations act on the computed traps: C01 carries a bead in the central trap (stiffness 4.01 pN/μm and a corner frequency of 70.0 Hz in the trace) and C03 an atom (a corner frequency of 107200 Hz and a radial trap frequency of 109092 Hz in the trace); the trap depth follows the intensity [35], and the bead's spectrum is a Lorentzian whose corner frequency is the stiffness over $2\pi\gamma$ [40], which the particle oracle checks. The fit of a Lorentzian to an average of n power spectra carries a bias of relative size between $-2/n$ and $+1/n$ [41]; the bounds on the fitted corner frequency, twelve percent in the oracle test and ten percent in the engine's dynamics test, are DinHot's own tolerances for the segment counts those runs use.

8.3 The benchmark

The protocol of Section 7.4 gives the benchmark. Table 27 names, per geometry and device, the published method and the proposed method with the lowest median spread at 100 iterations among those that reach at least half of the cell's best efficiency, and the best method at a time budget of 1 s; Table 28 gives the medians of every method over the 6 point-trap geometries, and Figure 8 every cell. On the ideal device the best method of a geometry is a published method on 1 and a proposed method on 4 of the 7 geometries, and on 2 the best published and proposed methods tie below the resolution of the file (0.001 percent); on the C08 device the counts are 4 and 2 of 6 (the multi-plane geometry has no device rows), with 0 ties. The

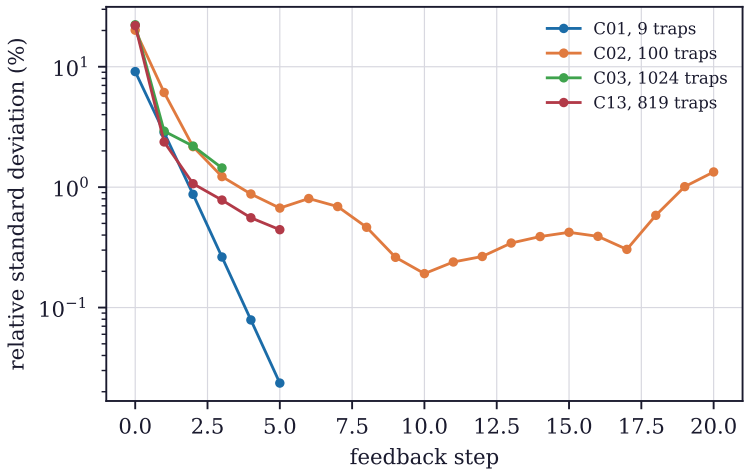


Figure 7: Emulated camera feedback: relative standard deviation of the trap intensities against the feedback step, on four cases (baked traces). The measured spread is that of the camera readings divided by the target weights, the quantity the experimenter sees.

flicker-bounded sequence enters these cells as its static frame, a phase-locked weighted GS, and not as a sequence, which is the quantity of its claim (Section 6.21). Over the point-trap geometries the lowest median spread on the ideal device is that of flicker-bounded phase-continuous sequence (<0.001 percent at a median efficiency of 0.756), and on the C08 device that of Wirtinger gradient descent with Adam (14 percent at 0.510). The medians of the 17 methods that keep at least half of the best efficiency span <0.001 to 25 percent on the ideal device and 14 to 34 percent (16 methods) on the C08 device, where every method runs with its default parameters and only the device-aware ones model the device. The times are wall-clock seconds of the machine that ran the benchmark, under shared load.

Figure 9 reads the same cells by geometry. On the ideal device the medians of the methods spread over several orders of magnitude within one geometry, from the analytic baselines to the optimization methods, and the proposed methods sit at the low end together with the phase-fixed and direct-search methods; on the C08 device the medians are compressed into a band of about one decade, with the geometry that has no device rows blank and no method below a few percent, a floor set by the quantizer and the flicker, which no method models completely. The figure does not show the efficiency of each point, which Table 28 gives, and it does not show the seed-to-seed interval, which is in the results file.

8.4 The learned models

Two networks were trained offline with PyTorch, exported to ONNX with a list of operators the browser runtime supports, and evaluated on held-out sets written before the final training with other seeds. A model is released only if it meets its criterion; otherwise the baseline answers. The first network (Section 6.17) maps three plane amplitudes at -8, 0 and $+8$ SLM-limited Rayleigh ranges to one phase for the 128×128 SLM, in the setting of DeepCGH [29]. It is trained without labels through the differentiable Fourier model with the loss $1 - \mathrm{AC}$ of Eq. (42) for 16000 steps. The exported graph is 4.67 MB. The baseline is the volumetric three-dimensional GS on the same traps with a wall-clock budget of 2.40 s, the median time of the volumetric GS at 30 iterations. On 162 held-out volumes the network answers in about 44 ms with a median accuracy ratio of 1.140 (quartiles 1.103 and 1.163), above the baseline on 100 percent of the volumes; the median accuracy is 0.687 against 0.599. By kind the ratio is 1.166 (lattices), 1.149 (random), 1.119 (mixed), 1.100 (rings), 1.067 (the $5 \times 5 \times 3$ benchmark lattice) and 1.187 (the three-plane layout). The graph agrees with its training framework on the reconstructed plane intensities to 8.4×10^{-6} relative to the maximum. The accuracy measure scores full plane images, ghosts and blur included, which is what the network optimises; the volumetric GS optimises trap uniformity, so the ratio is a statement about this measure. The two-dimensional counterpart, a single-plane network followed by GS refinement, ties: on 219 held-out targets the ratio at equal wall-clock is 0.9994, below the criterion, and that model is not released.

The second model is a permutation-equivariant set network that predicts initial trap phases for phase-fixed weighted GS (Section 6.22). On 66 held-out geometries it needs a median of 8 iterations to reach one percent against 14 from a random start (a factor of 1.75, below the factor of two set as the criterion), but only with

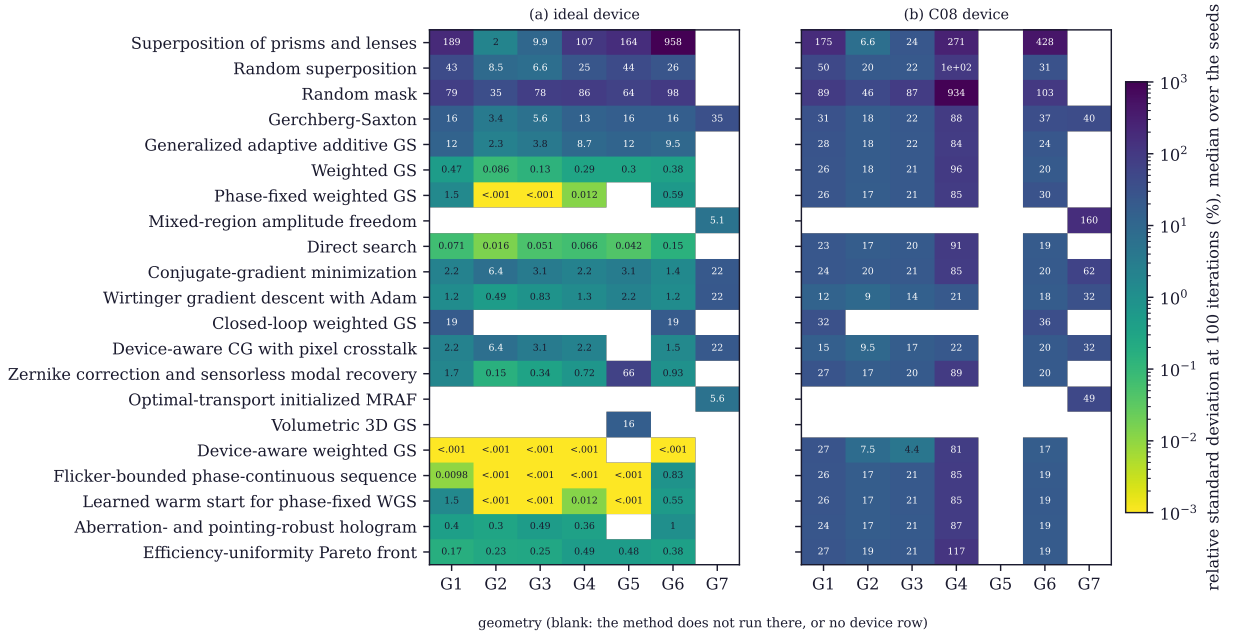


Figure 8: Benchmark: median relative standard deviation at 100 iterations with the default parameters, every method on every geometry, on the ideal device (a) and on the C08 device (b). Blank: the method does not accept the geometry or refuses it, or the geometry has no device row.

the early phase freeze (after 3 iterations, against 12 for the random arm). A control with no network, a random start and the same early freeze, needs a median of 9 iterations on the control subset, and 40 percent of its geometries never reach one percent (47 percent with the network); the final efficiency differs by 4.2 percent between the arms. The gain comes from the freeze, not from the network, so the proposal is refuted and the model is not released (Figure 10). The panel (b) of the figure shows the three arms, and does not show the share that never converges, which the text gives.

8.5 Ablations, sensitivities and the Pareto fronts

Three sensitivities were measured, one per method whose result depends on a parameter that the source sets differently. The first is the freeze index of the phase-fixed weighted GS on a crowded array (Table 29, Figure 11): the minimum spread reached in 150 iterations falls as the index rises, from a stall at the source value to a value below the target at the larger values, and with $N = 60$ the phase-fixed form reaches 0.5 percent in fewer iterations than weighted GS on both seeds (50 x 30 array to 0.5 percent in 64 and 64 iterations against 136 and 106 for weighted GS (phase fixed after 60; efficiency 0.83 and 0.83); with the source value of 12 it stalls at 3.58 percent). The reading is that the phase frozen after a few iterations belongs to an unconverged hologram on this grid, and the weight update then cannot repair it; the table shows that the dependence is not monotone at the largest value, so the optimum depends on the grid and the budget. The second is the scale convention of the adaptive form (Section 6.5): with the printed bracket the uniformity falls below GS, and with the scale convention it rises above it on every seed. The third is the target level of the aberration-robust hologram (Section 6.23), which trades efficiency for robustness: a higher level gives a smaller improvement factor. Among the components of the device-aware weighted GS (Section 6.20) the safeguard that returns the best admissible iterate is what makes the method never worse than its start, and the quantizer treated straight-through is the component that costs more than it gives on case C08. The fourth component, the weight update, was ablated (Table 30, measured for the companion manuscript): with every weight kept at one the relative standard deviation is lower on the acceptance task and on the square lattice of the C08 device, higher on the ring, and the same on the random and quasi-periodic sets, so that the weight update is not shown to be needed and the method is reported with the weights it is defined with.

The fronts of the efficiency–uniformity method are the last measured outcome (Table 31, Figure 12). On the ideal device the stored fronts are short: they hold between one and seven non-dominated points, and from the first to the last stored point the efficiency changes by at most a few hundredths while the uniformity changes by a few hundredths at most on the lattice geometries and not at all where a single point is stored. The weighted-GS corner is already near the optimum of the trade-off, so the front is nearly flat in this region,

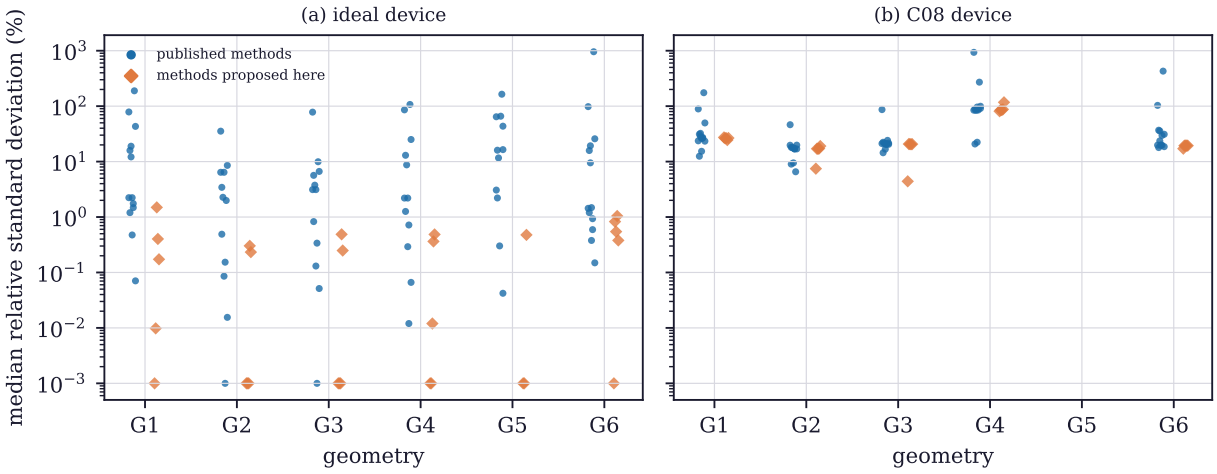


Figure 9: Median relative standard deviation at 100 iterations of every method on each point-trap geometry of the benchmark, on the ideal device (a) and on the C08 device (b); blue circles are the published methods and orange diamonds the methods proposed here. The vertical axis is logarithmic and the horizontal position within a geometry only separates the two groups.

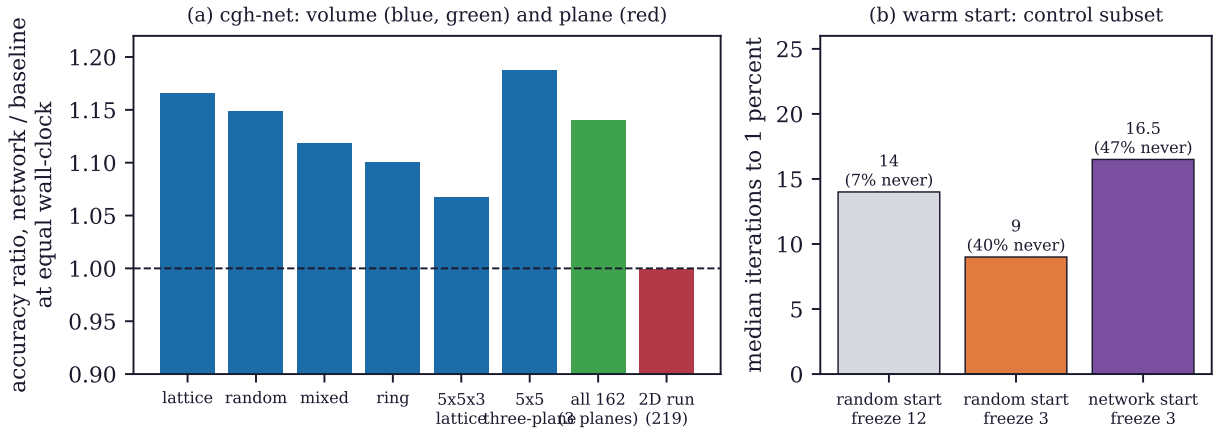


Figure 10: (a) The cgh-net accuracy ratio over its baseline at equal wall-clock, by target kind on the held-out volumes (G5: the $5 \times 5 \times 3$ lattice; 3-plane: the 5×5 three-plane layout), for the volume network (blue, green) and the two-dimensional single-plane network (red); the dashed line is the baseline. (b) The warm-start control: median iterations to 1 percent for a random start with the original and the early freeze and for the network start with the early freeze.

which is the risk that the design had named. The figure shows the front against the single points of the published methods at 100 iterations, so that a reader sees which points the front dominates; it does not show the hypervolume, which the table of the benchmark counts per geometry in Section 8.3.

8.6 Summary of the proposed methods

Of the five proposed methods, three meet their acceptance criteria on their acceptance tasks, each within a scope that the benchmark or the scope measurements narrow (a device-aware weighted GS update, a flicker-bounded phase-continuous sequence and an efficiency–uniformity front), one meets a narrowed criterion (an aberration-robust hologram, whose pointing-error clause is not obtained in the acceptance configuration and is reached only with a displacement prior optimized alone at five times the budget) and one is refuted by measurement (a learned warm start for phase-fixed weighted GS). The device-aware method is measured on the device of C08 over 2 of 5 geometries (G2 and G3), where it meets both thresholds, and the front's stored result reaches the hypervolume of the published points on 0 of the 6 geometries at 100 iterations and on 6 at 30. The details of the device-aware weighted GS and of the flicker-bounded sequence, with the factor on each geometry and the long moves of the sequence, are in the companion manuscript [45]; the outcome of each of

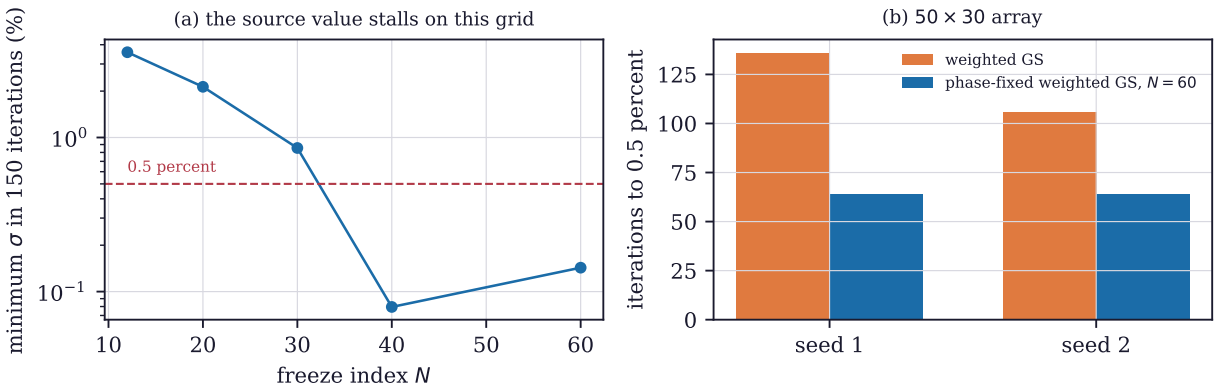


Figure 11: Phase-fixed weighted GS on the crowded 50×30 array: (a) the minimum relative standard deviation in 150 iterations against the freeze index N, with the target of 0.5 percent dashed; (b) iterations to reach 0.5 percent for weighted GS and for the phase-fixed form ($N = 60$) on the two seeds of the acceptance test.

the five is the same one that its method section states, and none is reported as better than it measured. The summary does not rank the five against each other, because their criteria are different quantities on different tasks, and it does not extend any of them beyond the tasks and the device on which it was measured. The measured acceptance of the aberration-robust hologram and the Pareto front is in Table 20 (hypervolume 0.8190 against 0.8018 for the union of the GS, generalized adaptive additive GS, weighted GS and direct search points; 7 front points dominate GS and 8 dominate generalized adaptive additive GS).

9 Validation

9.1 Analytic checks and oracles

A result is believed only when something independent of the engine agrees with it. The first group of checks compares the engine with analytic or manufactured solutions. Total power is conserved by the propagation (Parseval's relation holds for the centred orthonormal transform), and the Fourier form and the point-trap form give the same field at a trap on a bin, on grids of four sizes. The point-trap form matches an explicit evaluation of Eq. (1) with a depth offset and a vortex; the back-projection is the adjoint of the projection, which every weighted update relies on; and the angular spectrum matches an analytic Gaussian beam at one Rayleigh range. For the device, the adjoint of the crosstalk operator passes the inner-product identity on three devices, the kernel matches its definition in Eq. (34), the blur is the circular convolution with the kernel, the pixel envelope matches a brute-force transform of a pixel-sampled hologram for two fill factors, the crosstalk path reduces to the ideal path when the kernel is a delta, and the efficiency falls monotonically with fewer phase bits, with the first order of a binary grating at the value $4/\pi^2$ within a stated fraction.

The second group are physical limits that no implementation may violate. A random mask reaches an efficiency about the inverse of the trap count (an oracle written separately from the method) and a vortex winds its phase by $2\pi l$. The single-trap reference level of Eq. (6) is not among these limits: it is not a bound for many traps (Section 2.2, Appendix B) and no test treats it as one. The particle oracle test checks that the simulated bead's power spectrum fits a Lorentzian whose corner frequency is the stiffness of the computed trap divided by $2\pi\gamma$ (within twelve percent) and that the position variance follows equipartition; the dynamics test of the engine checks the same corner frequency within ten percent and that the position histogram follows the Boltzmann law of the computed potential (total variation below 0.06). The oracles live in a module that shares no code with the engine, so that a disagreement is a finding about one of the two sides and no model judges a model. For the gradient methods the closed-form gradient is checked against central finite differences for the powered costs, on shaped targets and on traps, and through the crosstalk operator. The Hungarian solver is checked against brute force on random rectangular matrices, and the incremental field update of the direct search against a full evaluation.

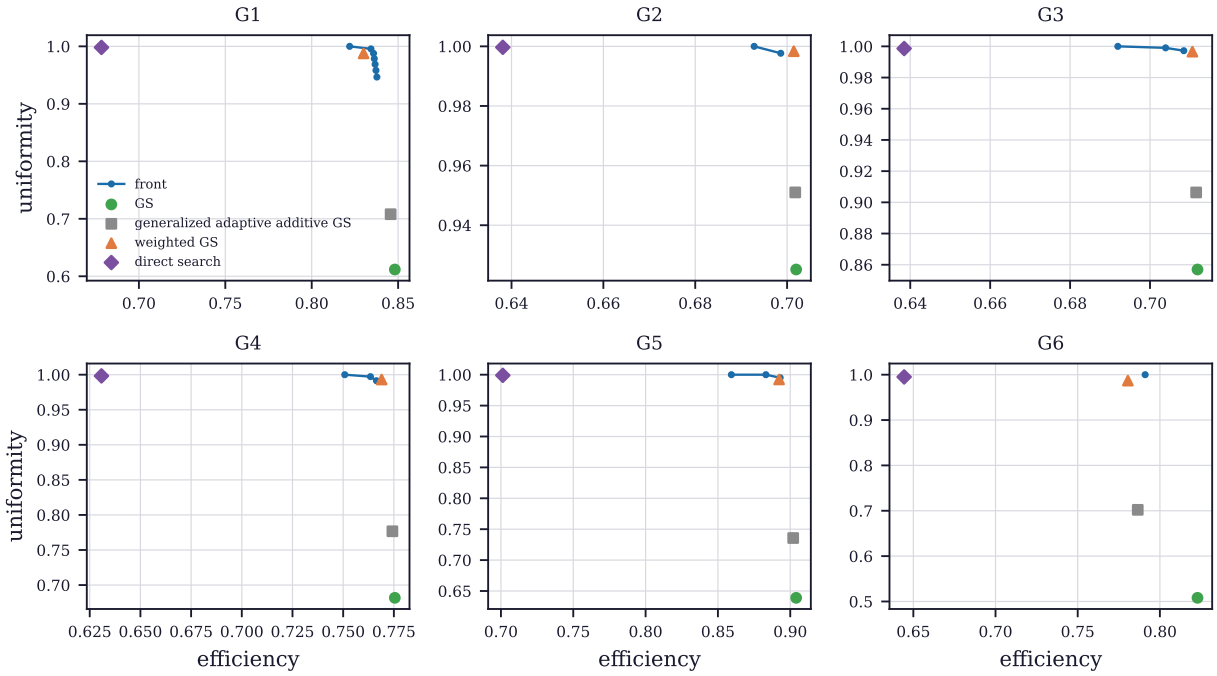


Figure 12: Stored efficiency–uniformity fronts of the benchmark on the ideal device, one panel per geometry with a front, against the median efficiency and uniformity at 100 iterations of the published methods GS, the adaptive form, weighted GS and direct search on the same geometry.

9.2 Published values

The second kind of validation reconciles the engine with the published values that the methods replicate (Table 32 lists the checks, and Table 20 the measurements). On the calibration task (Table 21) part of the ordering of the six baselines of Di Leonardo et al. carries over and part does not. The uniformity of the Gerchberg–Saxton family orders as in the source, plain GS (0.575) below the adaptive form (0.696) below weighted GS (0.972, against the published 0.99); the superposition has the lowest uniformity and the random mask the lowest efficiency; and the efficiencies of the three family members are non-increasing in both. The ordering of the two random baselines does not carry over: the published uniformity of random superposition (0.01) is below that of the random mask (0.58), whereas the engine's smaller grid gives the reverse (0.183 against 0.063), which the methods state as the dependence on the pixels per trap. The efficiencies of the Gaussian-beam runs fall below the flat-beam values of the source, and the flat-beam test closes the difference (the single-trap reference level of Eq. (6) is the scale of the gap, not a bound). The closed-loop methods are reconciled with the end points of two published curves: the 19 percent to 1.4 percent trap-depth spread of Nogrette et al. [22] and the 22 percent to 1.59 percent of Kim et al. [16], within the factor set in the design document. The device-aware conjugate gradient reproduces the ordering of Schroff et al. [23] and not the published error level; the optimal-transport initialization reproduces the ordering of Swan et al. [32] and has an efficiency in the MRAF class and not in the published class; and the random-superposition uniformity and the phase-fixed freeze index are not reproduced, which the corresponding methods state. These differences are kept as results. The case C13 is context for the disordered array of Kim et al. and not a band, because the build measures a lower spread with a Gaussian beam and a camera emulator without read noise, so that the two are not comparable. Where a published number was not re-run, as for the direct search on the quasi-periodic set, it is labelled as quoted.

9.3 The tests that guard each result

The repository holds 1358 test functions in the Python test tree, in folders for the engine (124), the methods (213), the contracts (79), the bake (44), the learned models (44), the adversarial plants (325), the notebooks (45), the manuscript checks (5) and the top-level guards (479), plus the browser-side tests of the live engine. Every acceptance criterion of Table 19 has a test that measures it on the live grid, and the test is the gate named in the criterion; a method whose test fails is not a method of the studio. The adversarial tests plant the failure each gate exists to catch: a fraction written where a percent belongs, a position in metres, a

trace over the byte budget, a manifest that differs between two bakes, a committed artifact that changes under a test, a method labelled live that does not fit the budget. Mutation runs confirm that a test can fail: replacing the tuned learning rate of the Wirtinger descent by the source value fails its acceptance test, and replacing the transport seed of the optimal-transport method by a random phase fails its disc and ring tests.

The live engine is held to the Python engine by parity tests over the reference documents that the Python engine writes: 46 documents, each quantity within the tolerance its document states, at most 10^{-9} absolute, with 92 browser-side tests in all. A one-iteration shift of the phase freeze of the phase-fixed method, a rule that is easy to get wrong, turns the parity test red, which is how the choice is pinned. The pipeline tests run in a sandbox that fails the test run if a committed artifact changes, and the scripts that regenerate every table and figure of this report read only committed files, so that the numbers in the report are the numbers of the artifacts. The units gate runs in the bake and again over the shipped artifacts; the lane gate and the expected ranges fail the bake on a value outside them. The validation does not establish that the model describes an instrument: that is the limit of Section 11.

10 Discussion

What the results mean for the problem depends on the target. For point arrays on an ideal device, the integral feedback of the weights and the freeze of the focal phase are the two ingredients that remove the spread: the weighted GS reaches a median spread of 1.1 percent within the budget of the bake, and the phase-fixed form reaches a fraction of a percent when the freeze index is matched to the grid (Table 23, Figure 11). The freeze index of the source does not transfer to a smaller SLM, a result the replication produced and the report keeps. The direct search and the Pareto knee reach comparable spreads at a lower efficiency, so the efficiency that the user is willing to pay decides between them, and the front of Section 8.5 shows that on these tasks the price is small because the weighted-GS corner is already near the optimum.

On a real device the picture changes, and it is the main finding of the device cases. The weighted forms equalize the model and not the device, so their spread on the device-limited case stays of the order of ten percent even though their own history falls to a small value; the device-aware weighted GS closes part of that gap on the case and on two geometries of the benchmark, and keeps its start on the lattices of the device because every iterate past the start gives up more than a tenth of the light. The measured support for the method is therefore on a ring and a random set on a device with crosstalk, and not as a replacement for the phase-fixed form on lattices, where the safeguard returns the start. The quantizer and the flicker are the unmodelled part: with them the floor of every method is a few percent, and the straight-through quantizer in the model costs more than it gives on the case that has it.

For shaped potentials, mixed-region freedom trades efficiency for accuracy by design, and the optimal-transport seed gives an error about an order of magnitude lower than the random start at the same efficiency class. For aberrations, the sensorless search recovers an injected set of modes within a small fraction of their amplitude on an ideal camera, and the aberration-robust hologram lowers the 95th percentile of the spread over an ensemble of residuals by a median factor of 1.435 at an efficiency cost of 1.8 percent; against random displacements of the traps its acceptance configuration gives no gain (a ratio of 1.04), while a displacement prior optimized alone for 100 steps reaches 1.48 at 0.10 focal bins at 0.91 of the efficiency, so the budget decides and the curvature of the spot sets only the floor that remains (the companion manuscript of the method gives the measurement). For moving traps, the flicker-bounded sequence keeps the transient intensity higher than linear interpolation in the model of the transient and on most seeds of the longer moves, and not on all; the limit of the seeds that fail is the phase path that the end phases prescribe and not the frame count, so the method needs a phase-path plan before it is a drop-in replacement.

The learned methods are measured on their own held-out sets. The volume network answers in tens of milliseconds and beats the volumetric GS at equal wall-clock on every held-out volume, which makes it useful when the target changes at interaction rate; its advantage is on the accuracy measure of plane images and not on trap uniformity, so a user who needs the spread of Table 23 still runs a weighted method. The warm start is refuted by its control: the early freeze does the work, not the network. That a learned component can be refuted by a control is part of the argument for keeping controls in the protocol. Where each method fails is stated in its section; the user-facing summary is that the studio recommends the phase-fixed form (with its index checked on the user's grid) for point arrays on an ideal device, the device-aware method when the device couples pixels and the geometry is a ring or a random set of the kind that was measured (on the non-lattice quasi-periodic set of the benchmark it is the weakest proposed entry on the C08 device, 81 percent against 21 percent for the best published method of that cell, and its support is 2 of 5 geometries of the benchmark (G2 and G3), one simulated device and 2 seeds in that cell), the optimal-transport seed for

smooth potentials, the sensorless Zernike search before any of them when an aberration is suspected, and the network when the time budget is a few tens of milliseconds.

11 Limitations

DinHot computes and evaluates holograms in simulation: it does not drive an SLM or a camera, and its camera-in-the-loop method runs on an emulated camera whose errors are stated parameters. Device and set-up parameters come from published experiments, so absolute numbers belong to the stated model and not to a measured instrument. The engine is scalar (Fraunhofer and angular spectrum); the high-numerical-aperture case carries a validity flag and the vectorial lane is not built. The fill-factor, four-bit and flicker effects of the device have no published source in this build and are computed only. The source of the crosstalk kernel does not settle whether its width in inverse pixels refers to the SLM pixel or to the computational pixel, one third of it; the engine reads it per SLM pixel, a factor of three away from the other reading, so the kernel is a model of the device and not of a measured one. The transient-intensity model of the flicker-bounded sequence is DinHot's own model of the display between two frames; it ranks methods under one model and is not calibrated against a measured display, and the linear-phase-interpolation baseline is DinHot's implementation of a published description, not the authors' code.

The statistics are limited by the seeds. The unit measurements of Table 20 use between one and six seeds, the bake three, and the benchmark 2 to 20 seeds per cell, with a wall-clock bound that lets the count move between runs, so that a count such as the number of geometries on which the front reaches the hypervolume of the published points at 100 iterations is not stable. The uncertainty reported is the observed range or the inter-quartile range, not a confidence interval. The ensemble of the aberration-robust hologram is drawn from the prior it was designed on, so robustness to another distribution is not established. The cgh-net advantage is on a plane-image accuracy measure over generated volumes and the benchmark's volumetric layouts, not on a hardware experiment, and the model is specific to its crop and planes. The learned warm start is refuted and the two-dimensional network ties with GS.

The data are synthetic and the cases few: 14 cases, one of them refused by design, cover the categories but not the diversity of laboratory set-ups. Several of the methods carry reconstructions that are marked in their sections and not confirmed against the original text: the scale convention of the adaptive form, the multiplicative reading of the closed-loop update, the pixel of the kernel width, the step unit of the linear interpolation, the scaling of the front's loss and the accuracy formula of the learned network. The executive figures of the introduction are taken from the cited publications and not re-measured. Finally, the devices and the tweezers of the introduction are neutral-atom arrays and microscopic beads; the optical-trapping literature for cells, neurons and machining is cited for context and no result here is a claim about those uses.

12 Reproducibility

The source code, the baked artifacts (manifests, traces, benchmark results, model cards, the ONNX graph and its held-out metrics) and the scripts that regenerate every table and figure of this report from them are in the repository `https://github.com/fsantibanezleal/CEFOP_DinHot` (Apache-2.0), under `manuscripts/holographic-trap-studio/`; the studio runs at `https://dinhot.fasl-work.com`. From a clone, with the pipeline environment of the repository, the commands are: the script `data/acceptance.py` measures the acceptance quantity of each method as its gate computes it; `data/params.py` reads the keyword defaults of every method; `data/extract.py` and `data/extract_methods.py` write the macro files and the table files from the committed artifacts, the schemas and the catalog; `figures/make_figs.py` (with the notebook environment, which has the plotting library) writes the twelve figures as vector PDF; and three passes of `pdflatex` over `tex/main.tex` build the report. The canonical bake and the benchmark are explicit release operations (`python data-pipeline/run.py` and its `benchmark` command) and take about the times of Section 7.3; training is local and writes into the data root, which the repository does not hold. The data sources are the case library of the repository and the committed artifacts; there is no third-party dataset. The version of the product that this report describes is 0.02.002. The released graph and its card are committed, and the held-out sets are identified by their hash in the model cards.

13 Conclusions

DinHot compares twenty-four hologram methods on one scalar Fourier model with a device model, 14 cases, a benchmark of 1326 rows and two learned models, and keeps every measured difference from the published sources as a result. The answer to the problem of choosing a method is conditional on the target and the device. Over 9 cases of the bake, among them a device-limited and an aberrated one, the phase-fixed weighted GS leaves a median relative standard deviation of 0.38 percent (the medians of the single seeds range from 0.014 to 0.29) against 17 percent (16 to 19) for plain GS, means over three seeds, but only with a freeze index matched to the grid; on a device with crosstalk the weighted forms equalize the model and not the device, and the device-aware weighted GS lowers the spread by a factor of 2.0 on the ratio of the means of the device-limited case. Of the five proposed methods, three meet their criteria within a measured scope, one meets a narrowed criterion and one is refuted; the volume network beats its baseline at equal wall-clock on all 162 held-out volumes with a median ratio of 1.140. The next steps are the measured hardware loop with a real SLM and camera, the vectorial focus, a calibrated device model, and a label-based or rule-changing design for the warm start, none of which is in this build.

References

[1] H. J. Manetsch, G. Nomura, E. Bataille, X. Lv, K. H. Leung and M. Endres, "A tweezer array with 6,100 highly coherent atomic qubits," *Nature* 647, 60–67, 2025. https://doi.org/10.1038/s41586-025-09641-4.

[2] Y. T. Chew *et al.*, "Ultraprecise holographic optical tweezer array," *Phys. Rev. A* 110(5), 053518, 2024. https://doi.org/10.1103/PhysRevA.110.053518.

[3] N.-C. Chiu *et al.*, "Continuous operation of a coherent 3,000-qubit system," *Nature* 646(8087), 1075–1080, 2025. https://doi.org/10.1038/s41586-025-09596-6.

[4] D. Bluvstein *et al.*, "Logical quantum processor based on reconfigurable atom arrays," *Nature* 626, 58–65, 2024. https://doi.org/10.1038/s41586-023-06927-3.

[5] R. Lin *et al.*, "AI-enabled parallel assembly of thousands of defect-free neutral atom arrays," *Phys. Rev. Lett.* 135(6), 060602, 2025. https://doi.org/10.1103/2ym8-vs82.

[6] S. Wang, Y. Hu, Y. Zhou, P. Lan and Z. Xu, "Feedback intensity equalization algorithm for multi-spots holographic tweezer," arXiv:2407.17049, 2024 (preprint; no journal version found). https://arxiv.org/abs/2407.17049.

[7] P. Schroff, E. Haller, S. Kuhr and A. La Rooij, "Rapid stochastic spatial light modulator calibration and pixel crosstalk optimization," *Opt. Express* 32(27), 48957, 2024. https://doi.org/10.1364/OE.539548.

[8] C. J. Bustamante, Y. R. Chemla, S. Liu and M. D. Wang, "Optical tweezers in single-molecule biophysics," *Nat. Rev. Methods Primers* 1, 25, 2021. https://doi.org/10.1038/s43586-021-00021-6.

[9] A. R. Mardinly *et al.*, "Precise multimodal optical control of neural ensemble activity," *Nat. Neurosci.* 21(6), 881–893, 2018. https://doi.org/10.1038/s41593-018-0139-8.

[10] Z. Kuang *et al.*, "High throughput diffractive multi-beam femtosecond laser processing using a spatial light modulator," *Appl. Surf. Sci.* 255(5), 2284–2289, 2008. https://doi.org/10.1016/j.apsusc.2008.07.091.

[11] Y. Peng, S. Choi, N. Padmanaban and G. Wetzstein, "Neural holography with camera-in-the-loop training," *ACM Trans. Graph.* 39(6), 1–14, 2020. https://doi.org/10.1145/3414685.3417802.

[12] R. Di Leonardo, F. Ianni and G. Ruocco, "Computer generation of optimal holograms for optical trap arrays," *Opt. Express* 15(4), 1913, 2007. https://doi.org/10.1364/OE.15.001913.

[13] S. Bianchi and R. Di Leonardo, "Real-time optical micro-manipulation using optimized holograms generated on the GPU," *Comput. Phys. Commun.* 181(8), 1444–1448, 2010. https://doi.org/10.1016/j.cpc.2010.04.012.

[14] M. Montes-Usategui, E. Pleguezuelos, J. Andilla and E. Martín-Badosa, "Fast generation of holographic optical tweezers by random mask encoding of Fourier components," *Opt. Express* 14(6), 2101, 2006. https://doi.org/10.1364/OE.14.002101.

[15] M. Polin, K. Ladavac, S.-H. Lee, Y. Roichman and D. G. Grier, "Optimized holographic optical traps," *Opt. Express* 13(15), 5831, 2005. https://doi.org/10.1364/OPEX.13.005831.

[16] D. Kim *et al.*, "Large-scale uniform optical focus array generation with a phase spatial light modulator," *Opt. Lett.* 44(12), 3178, 2019. https://doi.org/10.1364/OL.44.003178.

[17] S. Ebadi *et al.*, "Quantum phases of matter on a 256-atom programmable quantum simulator," *Nature* 595, 227–232, 2021. https://doi.org/10.1038/s41586-021-03582-4.

[18] M. Pasienski and B. DeMarco, "A high-accuracy algorithm for designing arbitrary holographic atom traps," *Opt. Express* 16(3), 2176, 2008. https://doi.org/10.1364/OE.16.002176.

[19] T. Harte, G. D. Bruce, J. Keeling and D. Cassettari, "Conjugate gradient minimisation approach to generating holographic traps for ultracold atoms," *Opt. Express* 22(22), 26548, 2014. https://doi.org/10.1364/OE.22.026548.

[20] P. Chakravarthula, Y. Peng, J. Kollin, H. Fuchs and F. Heide, "Wirtinger holography for near-eye displays," *ACM Trans. Graph.* 38(6), 1–13, 2019. https://doi.org/10.1145/3355089.3356539.

[21] D. P. Kingma and J. Ba, "Adam: a method for stochastic optimization," 3rd International Conference on Learning Representations, 2015; arXiv:1412.6980. https://arxiv.org/abs/1412.6980.

[22] F. Nogrette *et al.*, "Single-atom trapping in holographic 2D arrays of microtraps with arbitrary geometries," *Phys. Rev. X* 4(2), 021034, 2014. https://doi.org/10.1103/PhysRevX.4.021034.

[23] P. Schroff, A. La Rooij, E. Haller and S. Kuhr, "Accurate holographic light potentials using pixel crosstalk modelling," *Sci. Rep.* 13, 3252, 2023. https://doi.org/10.1038/s41598-023-30296-6.

[24] M. Persson, D. Engström and M. Goksör, "Reducing the effect of pixel crosstalk in phase only spatial light modulators," *Opt. Express* 20(20), 22334, 2012. https://doi.org/10.1364/OE.20.022334.

[25] R. J. Noll, "Zernike polynomials and atmospheric turbulence," *J. Opt. Soc. Am.* 66(3), 207, 1976. https://doi.org/10.1364/JOSA.66.000207.

[26] G. Whyte and J. Courtial, "Experimental demonstration of holographic three-dimensional light shaping using a Gerchberg–Saxton algorithm," *New J. Phys.* 7, 117, 2005. https://doi.org/10.1088/1367-2630/7/1/117.

[27] W. Lee, H. Kim and J. Ahn, "Defect-free atomic array formation using the Hungarian matching algorithm," *Phys. Rev. A* 95(5), 053424, 2017. https://doi.org/10.1103/PhysRevA.95.053424.

[28] I. H. A. Knottnerus, Y. C. Tseng, A. Urech, R. J. C. Spreeuw and F. Schreck, "Parallel assembly of neutral atom arrays with an SLM using linear phase interpolation," *SciPost Phys.* 19(4), 118, 2025. https://doi.org/10.21468/SciPostPhys.19.4.118.

[29] M. H. Eybposh, N. W. Caira, M. Atisa, P. Chakravarthula and N. C. Pégard, "DeepCGH: 3D computer-generated holography using deep learning," *Opt. Express* 28(18), 26636, 2020. https://doi.org/10.1364/OE.399624.

[30] Z. Ren *et al.*, "Creation of a tweezer array for cold atoms utilizing a generative neural network," *APL Quantum* 1(4), 046111, 2024. https://doi.org/10.1063/5.0219170.

[31] L. Shi, B. Li, C. Kim, P. Kellnhofer and W. Matusik, "Towards real-time photorealistic 3D holography with deep neural networks," *Nature* 591, 234–239, 2021. https://doi.org/10.1038/s41586-020-03152-0.

[32] H. Swan, A. Torchylo, M. J. Van de Graaff, J. Rudolph and J. M. Hogan, "High-fidelity holographic beam shaping with optimal transport and phase diversity," *Opt. Express* 33(3), 6290–6303, 2025. https://doi.org/10.1364/OE.540901.

[33] A. Torchylo, H. Swan, L. Tellez and J. M. Hogan, "A fast, large-scale optimal transport algorithm for holographic beam shaping," *Opt. Express* 34(4), 6796–6803, 2026. https://doi.org/10.1364/OE.588337.

[34] Y. Machu, G. Creutzer, C. Sayrin and M. Brune, "Full-field-of-view aberration correction for large arrays of focused beams," *Opt. Express*, 2026. https://doi.org/10.1364/OE.595662; preprint arXiv:2512.15967.

[35] R. Grimm, M. Weidemüller and Y. B. Ovchinnikov, "Optical dipole traps for neutral atoms," *Adv. At. Mol. Opt. Phys.* 42, 95–170, 2000. https://doi.org/10.1016/S1049-250X(08)60186-X.

[36] N. Holland, D. Stuart, O. Barter and A. Kuhn, "Benchmarking modern algorithms to holographically create optical tweezers for laser-cooled atoms," *J. Mod. Opt.* 65(18), 2133–2141, 2018. https://doi.org/10.1080/09500340.2018.1499978.

[37] V. N. Mahajan, "Strehl ratio for primary aberrations in terms of their aberration variance," *J. Opt. Soc. Am.* 73(6), 860–861, 1983. https://doi.org/10.1364/JOSA.73.000860.

[38] Y. Harada and T. Asakura, "Radiation forces on a dielectric sphere in the Rayleigh scattering regime," *Opt. Commun.* 124(5–6), 529–541, 1996. https://doi.org/10.1016/0030-4018(95)00753-9.

[39] T. D. Bouloumis and S. Nic Chormaic, "From far-field to near-field micro- and nanoparticle optical trapping," *Appl. Sci.* 10(4), 1375, 2020. https://doi.org/10.3390/app10041375.

[40] S. F. Tolić-Nørrelykke, E. Schäffer, J. Howard, F. S. Pavone, F. Jülicher and H. Flyvbjerg, "Calibration of optical tweezers with positional detection in the back focal plane," *Rev. Sci. Instrum.* 77(10), 103101, 2006. https://doi.org/10.1063/1.2356852.

[41] S. F. Nørrelykke and H. Flyvbjerg, "Power spectrum analysis with least-squares fitting: amplitude bias and its elimination, with application to optical tweezers and atomic force microscope cantilevers," *Rev. Sci. Instrum.* 81(7), 075103, 2010. https://doi.org/10.1063/1.3455217.

[42] D. Blackman and S. Vigna, "Scrambled linear pseudorandom number generators," *ACM Trans. Math. Softw.* 47(4), 1–32, 2021. https://doi.org/10.1145/3460772.

[43] A. Lizana *et al.*, "Time fluctuations of the phase modulation in a liquid crystal on silicon display: characterization and effects in diffractive optics," *Opt. Express* 16(21), 16711, 2008. https://doi.org/10.1364/OE.16.016711.

[44] I. Moreno, B. K. Gutierrez, M. M. Sánchez-López, J. A. Davis, H. P. Khanal and D. M. Cottrell, "Diffraction efficiency of stepped gratings using high phase-modulation spatial light modulators," *Opt. Lasers Eng.* 126, 105910, 2020. https://doi.org/10.1016/j.optlaseng.2019.105910.

[45] F. Santibáñez-Leal, "Device-aware weighted Gerchberg–Saxton and flicker-bounded phase-continuous sequences for holographic optical trap arrays," Zenodo, concept DOI https://doi.org/10.5281/zenodo.23788733.

A Contract fields

The tables of Section 4 list the top-level fields of each contract. This appendix lists the nested fields of the larger contracts, generated from the schema files by the same script: the path of the field (a dot descends into an object, brackets into an array item, and braces into a map keyed by an identifier such as a method id), its type, whether the schema requires it, its unit where the name carries one, and the range or constant that the schema states. The schema files are the single source of the contracts: the Python producers and consumers validate against them in their tests, the TypeScript types are generated from them, and a change to a field bumps the version of its contract by the semantic-version rule. Fields that the schema leaves unconstrained show a dash in the range column; a dash in the unit column marks an identifier, a count or a dimensionless quantity.

Table 33 lists the parameter record, one case as the ingestion accepts it, and Table 34 the case manifest, the authoritative record of one baked case. Table 35 gives the trace, the replay artifact of one case; Table 36 the benchmark results, the protocol's table by method, geometry and budget; Table 37 the held-out metrics of the model slots; and Table 38 the readiness record, which states what is real in the build and which the app reads.

Table 33: Nested fields of the contract `params-record`, generated from its schema file: the path of each field, its type, whether it is required, its unit and the range or constant the schema states.

Field	Type	Req.	Unit	Range
`case_id`	string	yes	-	pattern
`setup`	object	yes	-	-
`setup.wavelength_nm`	number	yes	nm	300 to 2000
`setup.focal_mm`	number	yes	mm	0.5 to 1000
`setup.na`	number	yes	-	0.05 to 1.5
`setup.n_imm`	number	yes	-	1.0 to 1.6
`setup.n_pix`	enum	yes	-	{64, 128, 256, 512, 1024}
`setup.pitch_um`	number	yes	μm	1 to 50
`setup.pad`	integer	yes	-	1 to 4
`setup.beam`	object	yes	-	-
`device`	object or null	yes	-	-
`target`	object	yes	-	-
`aberration`	object or null	yes	-	-
`methods`	array of string	yes	-	1 to 24 items
`reference_method`	string	yes	-	pattern
`budget`	object	yes	-	-
`budget.iterations`	integer	yes	-	1 to 5000
`budget.time_s`	number or null	yes	s	0.01 to 3600
`seed`	integer	yes	-	0 to 2147483647
`feedback`	object or null	yes	-	-

Field	Type	Req.	Unit	Range
dynamics	object or null	yes	-	-
rearrangement	object or null	yes	-	-
expect	object	yes	-	-

Table 34: Nested fields of the contract `case-manifest`, generated from its schema file: the path of each field, its type, whether it is required, its unit and the range or constant the schema states.

Field	Type	Req.	Unit	Range
schema	const	yes	-	= dinhot.manifest/v1
case_id	string	yes	-	length >= 1
title	object	yes	-	-
title.en	string	yes	-	length >= 1
title.es	string	yes	-	length >= 1
category	object	yes	-	-
category.en	string	yes	-	length >= 1
category.es	string	yes	-	length >= 1
real_or_synthetic	enum	yes	-	{real, synthetic}
expected_band	object	yes	-	-
expected_band.en	string	yes	-	length >= 1
expected_band.es	string	yes	-	length >= 1
engine	object	yes	-	-
engine.package	const	yes	-	= dinhot-holo
engine.version	string	yes	-	pattern
engine.model	const	yes	-	= scalar-fourier
params	object	yes	-	-
params.setup	object	yes	-	-
params.setup.wavelength_nm	number	yes	nm	300 to 2000
params.setup.focal_mm	number	yes	mm	0.5 to 1000
params.setup.na	number	yes	-	0.05 to 1.5
params.setup.n_imm	number	yes	-	1.0 to 1.6
params.setup.n_pix	enum	yes	-	{64, 128, 256, 512, 1024}
params.setup.pitch_um	number	yes	μm	1 to 50
params.setup.pad	integer	yes	-	1 to 4
params.setup.beam	object	yes	-	-
params.device	object or null	yes	-	-
params.target	object	yes	-	-
params.aberration	object or null	yes	-	-
params.methods	array of string	yes	-	1 to 24 items
params.reference_method	string	yes	-	pattern
params.budget	object	yes	-	-
params.budget.iterations	integer	yes	-	1 to 5000
params.budget.time_s	number or null	yes	s	0.01 to 3600
params.seed	integer	yes	-	0 to 2147483647
params.feedback	object or null	yes	-	-
params.dynamics	object or null	yes	-	-
params.rearrangement	object or null	yes	-	-
params.expect	object	yes	-	-
seed	integer	yes	-	-
artifact	object	yes	-	-
artifact.path	string	yes	-	pattern
artifact.format	const	yes	-	= json
artifact.trace_schema	const	yes	-	= dinhot.trace/v1
artifact.bytes	integer	yes	-	1 to
lane	enum	yes	-	{live, precompute}
gate	object	yes	-	-
gate.lane	enum	yes	-	{live, precompute}
gate.pure_python	boolean	yes	-	-
gate.wheels	array of string	yes	-	-
gate.trace_bytes	integer	yes	bytes	1 to
gate.run_ms_budget	number	yes	-	>0 to
gate.trace_bytes_budget	integer	yes	-	1 to
gate.reasons	array of string	yes	-	-
flags	array of object	yes	-	-
methods	object	yes	-	-
methods{id}.lane	enum	yes	-	{live, baked}
methods{id}.efficiency	number	yes	fraction	-
methods{id}.uniformity	number	yes	fraction	-
methods{id}.rel_std_pct	number	yes	percent	-
methods{id}.zero_order	number	yes	fraction	-
methods{id}.ghost	number	yes	fraction	-
methods{id}.iterations	integer	yes	-	-
methods{id}.converged	boolean	yes	-	-
models	object	yes	-	-
expect	object	yes	-	-
anchor	object	yes	-	-
anchor.source	object	yes	-	-
anchor.source.en	string	yes	-	length >= 1
anchor.source.es	string	yes	-	length >= 1
anchor.url	string or null	yes	-	pattern

Table 35: Nested fields of the contract `trace`, generated from its schema file: the path of each field, its type, whether it is required, its unit and the range or constant the schema states.

Field	Type	Req.	Unit	Range
`schema`	const	yes	-	= dinhot.trace/v1
`case_id`	string	yes	-	length >= 1
`grid`	object	yes	-	-
`grid.n_pix`	integer	yes	-	-
`grid.pad`	integer	yes	-	-
`grid.bin_um`	number	yes	μm	-
`grid.half_field_um`	number	yes	μm	-
`traps`	array of object	yes	-	0 to 4096 items
`traps[].id`	integer	yes	-	0 to 4095
`traps[].x_um`	number	yes	μm	-100000.0 to 100000.0
`traps[].y_um`	number	yes	μm	-100000.0 to 100000.0
`traps[].z_um`	number	yes	μm	-100000.0 to 100000.0
`traps[].weight`	number	yes	fraction	0.01 to 100
`traps[].charge`	integer	yes	-	-20 to 20
`shape`	object or null	yes	-	-
`methods`	object	yes	-	-
`methods{id}.history`	array of object	yes	-	0 to 200 items
`methods{id}.history[].iteration`	integer	yes	-	-
`methods{id}.history[].efficiency`	number	yes	fraction	-
`methods{id}.history[].uniformity`	number	yes	fraction	-
`methods{id}.history[].rel_std_pct`	number	yes	percent	-
`methods{id}.final`	object	yes	-	-
`methods{id}.final.efficiency`	number	yes	fraction	-
`methods{id}.final.uniformity`	number	yes	fraction	-
`methods{id}.final.rel_std_pct`	number	yes	percent	-
`methods{id}.final.zero_order`	number	yes	fraction	-
`methods{id}.final.ghost`	number	yes	fraction	-
`methods{id}.trap_intensity`	array of number	yes	-	-
`reference`	object	yes	-	-
`reference.method`	string	yes	-	pattern
`reference.phase_u8`	string	yes	-	pattern
`reference.focal`	object	yes	-	-
`reference.focal.size`	integer	yes	-	1 to 192
`reference.focal.bin_um`	number	yes	μm	-
`reference.focal.db_floor`	number	yes	-	-
`reference.focal.intensity_db_u8`	string	yes	-	pattern
`reference.axial`	object or null	yes	-	-
`feedback`	object or null	yes	-	-
`dynamics`	object or null	yes	-	-
`rearrangement`	object or null	yes	-	-
`summary`	object	yes	-	-
`summary.best_method`	string	yes	-	pattern
`summary.notes`	object	yes	-	-
`summary.notes.en`	string	yes	-	length >= 1
`summary.notes.es`	string	yes	-	length >= 1

Table 36: Nested fields of the contract `benchmark-results`, generated from its schema file: the path of each field, its type, whether it is required, its unit and the range or constant the schema states.

Field	Type	Req.	Unit	Range
`schema`	const	yes	-	= dinhot.benchmark/v1
`engine_version`	string	yes	-	pattern
`protocol`	object	yes	-	-
`protocol.grids`	array of integer	yes	-	-
`protocol.seeds`	integer	yes	-	-
`protocol.budgets`	object	yes	-	-
`protocol.budgets.iterations`	array of integer	yes	-	-
`protocol.budgets.time_s`	array of number	yes	s	-
`protocol.device`	object	yes	-	-
`protocol.device.en`	string	yes	-	length >= 1
`protocol.device.es`	string	yes	-	length >= 1
`geometries`	array of object	yes	-	-
`geometries[].id`	string	yes	-	pattern
`geometries[].name`	object	yes	-	-
`geometries[].name.en`	string	yes	-	length >= 1
`geometries[].name.es`	string	yes	-	length >= 1
`geometries[].n_traps`	integer	yes	-	-
`geometries[].dims`	enum	yes	-	{2, 3}
`rows`	array of object	yes	-	-
`rows[].method`	string	yes	-	pattern
`rows[].geometry`	string	yes	-	pattern
`rows[].device`	enum	yes	-	{ideal, device}
`rows[].budget_kind`	enum	yes	-	{iterations, time, converged}
`rows[].budget`	number	yes	-	-
`rows[].params`	enum	yes	-	{default, tuned}
`rows[].n_seeds`	integer	yes	-	-
`rows[].efficiency`	object	yes	fraction	-
`rows[].efficiency.median`	number	yes	-	-
`rows[].efficiency.q1`	number	yes	-	-

Field	Type	Req.	Unit	Range
rows[].efficiency.q3	number	yes	-	-
rows[].uniformity	object	yes	fraction	-
rows[].uniformity.median	number	yes	-	-
rows[].uniformity.q1	number	yes	-	-
rows[].uniformity.q3	number	yes	-	-
rows[].rel_std_pct	object	yes	percent	-
rows[].rel_std_pct.median	number	yes	-	-
rows[].rel_std_pct.q1	number	yes	-	-
rows[].rel_std_pct.q3	number	yes	-	-
rows[].time_s	object	yes	s	-
rows[].time_s.median	number	yes	-	-
rows[].time_s.q1	number	yes	-	-
rows[].time_s.q3	number	yes	-	-
rows[].iterations	object	yes	-	-
rows[].iterations.median	number	yes	-	-
rows[].iterations.q1	number	yes	-	-
rows[].iterations.q3	number	yes	-	-
pareto	array of object	yes	-	-
pareto[].geometry	string	yes	-	pattern
pareto[].method	const	yes	-	= efficiency-uniformity Pareto front
pareto[].points	array of object	yes	-	-
pareto[].points[].lambda	number	yes	-	-
pareto[].points[].efficiency	number	yes	fraction	-
pareto[].points[].uniformity	number	yes	fraction	-
skipped	array of object	no	-	-
skipped[].method	string	yes	-	pattern
skipped[].geometry	string	yes	-	pattern
skipped[].reason	object	yes	-	-
skipped[].reason.en	string	yes	-	length >= 1
skipped[].reason.es	string	yes	-	length >= 1

Table 37: Nested fields of the contract `model-metrics`, generated from its schema file: the path of each field, its type, whether it is required, its unit and the range or constant the schema states.

Field	Type	Req.	Unit	Range
schema	const	yes	-	= dinhot.model-metrics/v1
engine_version	string	yes	-	pattern
slots	object	yes	-	-
slots.cgh-net	object	no	-	-
slots.cgh-net.held_out_n	integer	yes	-	-
slots.cgh-net.accuracy_vs_gs	object	yes	-	-
slots.cgh-net.time_ms	object	yes	ms	-
slots.cgh-net.baseline	const	yes	-	= volumetric 3D GS
slots.warmstart-model	object	no	-	-
slots.warmstart-model.held_out_n	integer	yes	-	-
slots.warmstart-model.iterations_to_1pct	object	yes	-	-
slots.warmstart-model.final_rel_std_pct	object	yes	percent	-

Table 38: Nested fields of the contract `readiness`, generated from its schema file: the path of each field, its type, whether it is required, its unit and the range or constant the schema states.

Field	Type	Req.	Unit	Range
schema	const	yes	-	= example.readiness/v1
product_version	string	yes	-	pattern
catalog_sha256	string	yes	-	pattern
contracts	array of object	yes	-	1 to items
contracts[].id	string	yes	-	pattern
contracts[].version	string	yes	-	pattern
contracts[].status	enum	yes	-	{real, active}
contracts[].provenance	enum	yes	-	{real, stub, simulator, fixture, baseline, ...}
contracts[].state	enum	yes	-	{available, partial, stale, provider, training, ...}
contracts[].reason	object or null	yes	-	-
contracts[].missing	array of string	yes	-	-

B The window, the envelope and the efficiency ceiling

Three relations of Section 2 are used without a full derivation in the body. The first is the unaliased window, derived for this product and not quoted from a paper. The SLM samples the phase on a grid of pitch p, so that its field repeats in the focal plane with the period $\lambda f/p$ of the grating equation; a centred grid of that period spans from $-X_{1/2}$ to $X_{1/2}$ with $X_{1/2} = \lambda f/(2p)$, and a trap at or beyond the bound folds back onto the grid and cannot be told from a trap inside it, which is why the first contract refuses it. One bin of the transform padded by pad is the period divided by the padded size, $\Delta x = \lambda f/(\mathrm{pad}\, n_{\mathrm{pix}}\, p)$.

The second is the pixel-aperture envelope. A square pixel of width $a = \sqrt{\text{fill}}\, p$ is a rectangular aperture, whose Fourier transform is a cardinal sine of argument $ax/(\lambda f)$ along each axis. At the edge of the window the argument is $\sqrt{\text{fill}}/2$, so that the intensity is

$$\mathcal{E}(X_{1/2}) = \operatorname{sinc}^2\Big(\frac{\sqrt{\text{fill}}}{2}\Big), \tag{62}$$

about 0.4 for a fill factor of one, the value that the pixel-aperture test confirms against a brute-force transform of a pixel-sampled hologram. The envelope has unit value at the centre, and the power lost to dead area is not applied, so the efficiencies of the report are fractions of the power that reaches the focal plane.

The third is the efficiency ceiling, which is a bound for one trap only. The efficiency of Eq. (5) is the sum of the trap intensities, each a fraction of the input power. A hologram with the best possible phase at a single trap gives the field $V = \frac{1}{N_{\text{pix}}} \sum_j A_j$, because all the terms of Eq. (2) are then in phase; its intensity is the square of the mean amplitude, and no phase pattern can do better at that trap. For a flat beam $A_j = 1$ and the bound is one; for a Gaussian beam it is the square of the mean amplitude of Eq. (6), below one. For many traps the only bound for every beam and every phase is $e \le 1$, and $\overline{A}^2$ is exceeded: a numerical maximization for a Gaussian beam with $\overline{A}^2 = 0.81$ reaches 0.995 at ten traps. The tests use $\overline{A}^2$ as a reference level (the Gaussian tests state 0.9 of it), so those thresholds are relative to the single-trap limit and not to a proven ceiling of the array.

Table 1: Notation: the symbols used in more than one section of the report, with their meaning and unit; a symbol that belongs to one derivation only (the sigma points of one method, the statistics of one test) is defined where it appears. A unit of 1 marks a dimensionless quantity or a count, and the intensities are fractions of the input power.

Symbol	Meaning	Unit
λ	vacuum wavelength of the laser	nm
f	focal length of the lens in the pupil of which the SLM sits	mm
NA	numerical aperture of the objective	1
n_{imm}	refractive index of the immersion medium	1
n_{pix}	SLM pixels per side	1
N_{pix}	number of SLM pixels, n_{pix}^2	1
p	SLM pixel pitch	μm
pad	zero-padding factor of the focal grid	1
n_{fft}	focal grid size, $\mathrm{pad}\, n_{\mathrm{pix}}$	1
j	index of an SLM pixel, at position (x_j, y_j) from the optical axis	m
$m,\ M$	index and number of traps	1
(x_m, y_m, z_m)	position of trap m in the focal volume	μm
l_m	vortex (topological) charge of trap m	1
$w_m,\ a_m$	target weight of trap m and its target amplitude $\sqrt{w_m}$	1
A_j	input beam amplitude, normalised so that $\sum_j A_j^2 = N_{\mathrm{pix}}$	1
ϕ_j	phase displayed at pixel j	rad
Δ_j^m	phase delay from pixel j to trap m (Eq. (1))	rad
$V_m,\ I_m$	complex field at trap m and its intensity $\lvert V_m\rvert^2$ (fraction of the input power)	1
e	efficiency, the sum of the trap intensities	1
u	uniformity, $1 - (\max I - \min I)/(\max I + \min I)$	1
σ	relative standard deviation of the trap intensities	percent
η	root-mean-square fractional error of a shaped potential in its measure region	1
$\Delta x,\ X_{1/2}$	width of one focal bin and half of the unaliased field	μm
$K,\ \sigma_K,\ q$	crosstalk kernel of the device and its fitted width and exponent	1
$Z_j,\ c_j$	Zernike mode j (Noll order) and its coefficient	waves
N (or n_{fix})	iteration after which the focal phase is frozen	1
α	exponent of the assignment cost of the Hungarian rearrangement or weight of the generalized adaptive additive GS	1
m (mixing)	mixing parameter of the mixed-region rule	1
$G,\ \rho_m$	feedback gain and measured intensity relative to its target	1
λ_{EU}	trade-off parameter of the efficiency-uniformity Pareto front	1
AC	accuracy of a volume hologram, mean over planes of the overlap of unit-sum intensities	1
$z_{R,\mathrm{SLM}}$	SLM-limited Rayleigh range of the volume network (Eq. (43)), the depth unit of its planes	mm
$\kappa,\ \gamma,\ f_c$	trap stiffness, drag coefficient and corner frequency of a bead	pN/μm, kg/s, Hz
$U,\ \hat{U},\ \omega_r,\ \omega_z$	dipole potential, trap depth of an atom and its radial and axial trap angular frequencies	J, J, rad/s, rad/s
w_{b}	waist of the input Gaussian beam at the SLM, $w_{\mathrm{frac}}\, n_{\mathrm{pix}}\, p/2$ (not the trap weight w_m)	μm
$w_0,\ z_R$	waist and Rayleigh range $\pi w_0^2/\lambda$ of the focus of an atom trap (Eq. (8))	μm
$\omega_0,\ \Gamma,\ \Delta$	resonance angular frequency, natural linewidth and detuning of the atomic transition (Eq. (8)); the detuning Δ carries no indices, unlike the delay Δ_j^m	rad/s
$c,\ \epsilon_0$	speed of light and vacuum permittivity	m/s, F/m
$n_m,\ n_p,\ r$	index of the medium, index and radius of a bead; $m = n_p/n_m$ in the polarizability is an index ratio and not the trap index	1, 1, μm
α (polarizability)	polarizability of the bead of Eq. (9), the third meaning of α	F m^2
$D,\ P(f)$	diffusion coefficient k_BT/γ and power spectral density of the bead position at frequency f	m^2/s, m^2/Hz
$H,\ \boldsymbol{\xi},\ \lambda_m$	angular-spectrum transfer function, spatial frequency and wavelength in the medium, λ/n_{imm} (Eq. (4))	1, 1/μm, nm
$T_m,\ t$	target intensity of trap m in the cost of the optimizers, and the power of that cost (Eq. (28))	1
$L,\ \epsilon$	loss of an optimizer (a quadratic intensity loss unless stated) and a small positive constant that keeps a logarithm or a quotient finite	1
$g_i,\ B_i,\ \psi$	weights, focal amplitude relative to its target and focal phase of the phase-fixed iteration i (Eq. (22))	1, 1, rad
$\theta_m,\ h(\tau)$	phase given to trap m (random in the random superposition, learned in the learned warm start, locked in the flicker-bounded sequence) and the minimal-jerk profile of $\tau = s/S$ (Eq. (53))	rad, 1

Table 2: The data contracts of the build, from the catalog: identifier, version, owner (the module that produces the document or checks the boundary) and the committed documents the schema governs.

Contract	Version	Owner	Documents governed
params-record	1.0.0	data-pipeline/pipeline/io/contract.py	run-timeexchange,nodocument
case-index	1.0.0	data-pipeline/pipeline/core/manifest.py	data/derived/manifests/index.json
case-manifest	1.0.0	data-pipeline/pipeline/core/manifest.py	data/derived/manifests/*.json;!data/derived/manifests/index.json
trace	1.0.0	data-pipeline/pipeline/core/trace.py	data/derived/*/trace.json
benchmark-results	1.0.0	data-pipeline/pipeline/stages/benchmark.py	data/derived/benchmark/results.json
cgh-net	1.0.1	data-pipeline/pipeline/stages/train.py	models/cgh-net.json;models/cgh-net.onnx
warmstart-model	1.0.1	data-pipeline/pipeline/stages/train.py	models/warmstart.json;models/warmstart.onnx
model-metrics	1.0.1	data-pipeline/pipeline/stages/evaluate.py	data/derived/models/metrics.json
model-evidence	1.0.0	data-pipeline/pipeline/stages/train.py	data/derived/models/*-training.json;data/derived/models/*-rejected.json;data/derived/models/evaluation-detail.json
engine-reference	1.0.0	data-pipeline/pipeline/stages/engine_reference.py	tests/fixtures/engine/*.json
readiness	1.0.0	data-pipeline/pipeline/core/readiness.py	data/derived/readiness.json
api	1.0.0	app/	docs/contracts/examples/api-unavailable.json#UnavailableBody

Table 3: Top-level fields of the contract `params-record`, read from its schema file: type, requirement, unit (a dash marks an identifier, a count or a dimensionless quantity) and the range or constant the schema states.

Field	Type	Req.	Unit	Range
case_id	string	yes	-	pattern
setup	object	yes	-	-
device	object or null	yes	-	-
target	object	yes	-	-
aberration	object or null	yes	-	-
methods	array of string	yes	-	1 to 24 items
reference_method	string	yes	-	pattern
budget	object	yes	-	-
seed	integer	yes	-	0 to 2147483647
feedback	object or null	yes	-	-
dynamics	object or null	yes	-	-
rearrangement	object or null	yes	-	-
expect	object	yes	-	-

Table 4: Top-level fields of the contract `case-index`, read from its schema file: type, requirement, unit (a dash marks an identifier, a count or a dimensionless quantity) and the range or constant the schema states.

Field	Type	Req.	Unit	Range
schema	const	yes	-	= dinhot.index/v1
engine_version	string	yes	-	pattern
default_case	string	yes	-	length >= 1
cases	array of object	yes	-	1 to items

Table 5: Top-level fields of the contract `case-manifest`, read from its schema file: type, requirement, unit (a dash marks an identifier, a count or a dimensionless quantity) and the range or constant the schema states.

Field	Type	Req.	Unit	Range
`schema`	const	yes	-	= dinhot.manifest/v1
`case_id`	string	yes	-	length >= 1
`title`	object	yes	-	-
`category`	object	yes	-	-
`real_or_synthetic`	enum	yes	-	{real, synthetic}
`expected_band`	object	yes	-	-
`engine`	object	yes	-	-
`params`	object	yes	-	-
`seed`	integer	yes	-	-
`artifact`	object	yes	-	-
`lane`	enum	yes	-	{live, precompute}
`gate`	object	yes	-	-
`flags`	array of object	yes	-	-
`methods`	object	yes	-	-
`models`	object	yes	-	-
`expect`	object	yes	-	-
`anchor`	object	yes	-	-

Table 6: Top-level fields of the contract `trace`, read from its schema file: type, requirement, unit (a dash marks an identifier, a count or a dimensionless quantity) and the range or constant the schema states.

Field	Type	Req.	Unit	Range
`schema`	const	yes	-	= dinhot.trace/v1
`case_id`	string	yes	-	length >= 1
`grid`	object	yes	-	-
`traps`	array of object	yes	-	0 to 4096 items
`shape`	object or null	yes	-	-
`methods`	object	yes	-	-
`reference`	object	yes	-	-
`feedback`	object or null	yes	-	-
`dynamics`	object or null	yes	-	-
`rearrangement`	object or null	yes	-	-
`summary`	object	yes	-	-

Table 7: Top-level fields of the contract `benchmark-results`, read from its schema file: type, requirement, unit (a dash marks an identifier, a count or a dimensionless quantity) and the range or constant the schema states.

Field	Type	Req.	Unit	Range
`schema`	const	yes	-	= dinhot.benchmark/v1
`engine_version`	string	yes	-	pattern
`protocol`	object	yes	-	-
`geometries`	array of object	yes	-	-
`rows`	array of object	yes	-	-
`pareto`	array of object	yes	-	-
`skipped`	array of object	no	-	-

Table 8: Top-level fields of the contract `cgh-net`, read from its schema file: type, requirement, unit (a dash marks an identifier, a count or a dimensionless quantity) and the range or constant the schema states.

Field	Type	Req.	Unit	Range
`schema`	const	yes	-	= dinhot.model-card/v1
`slot`	enum	yes	-	{cgh-net, warmstart-model}
`onnx`	string	yes	-	pattern
`sha256`	string	yes	-	pattern
`bytes`	integer	yes	-	1 to
`opset`	integer	yes	-	1 to
`inputs`	array of object	yes	-	1 to items
`outputs`	array of object	yes	-	1 to items
`grid`	const	no	-	= 128
`trained_on`	object	yes	-	-
`held_out`	object	yes	-	-
`metrics_path`	string	yes	-	length >= 1

Table 9: Top-level fields of the contract `warmstart-model`, read from its schema file: type, requirement, unit (a dash marks an identifier, a count or a dimensionless quantity) and the range or constant the schema states.

Field	Type	Req.	Unit	Range
`schema`	const	yes	-	= dinhot.model-card/v1
`slot`	enum	yes	-	{cgh-net, warmstart-model}
`onnx`	string	yes	-	pattern
`sha256`	string	yes	-	pattern
`bytes`	integer	yes	-	1 to
`opset`	integer	yes	-	1 to
`inputs`	array of object	yes	-	1 to items
`outputs`	array of object	yes	-	1 to items
`grid`	const	no	-	= 128
`trained_on`	object	yes	-	-
`held_out`	object	yes	-	-
`metrics_path`	string	yes	-	length >= 1

Table 10: Top-level fields of the contract `model-metrics`, read from its schema file: type, requirement, unit (a dash marks an identifier, a count or a dimensionless quantity) and the range or constant the schema states.

Field	Type	Req.	Unit	Range
`schema`	const	yes	-	= dinhot.model-metrics/v1
`engine_version`	string	yes	-	pattern
`slots`	object	yes	-	-

Table 11: Top-level fields of the contract `model-evidence`, read from its schema file: type, requirement, unit (a dash marks an identifier, a count or a dimensionless quantity) and the range or constant the schema states.

Field	Type	Req.	Unit	Range

Table 12: Top-level fields of the contract `engine-reference`, read from its schema file: type, requirement, unit (a dash marks an identifier, a count or a dimensionless quantity) and the range or constant the schema states.

Field	Type	Req.	Unit	Range
`schema`	const	yes	-	= dinhot.engine-ref/v1
`engine_version`	string	yes	-	pattern
`function`	string	yes	-	length >= 1
`tolerance`	object	yes	-	-
`cases`	array of object	yes	-	1 to items

Table 13: Top-level fields of the contract `readiness`, read from its schema file: type, requirement, unit (a dash marks an identifier, a count or a dimensionless quantity) and the range or constant the schema states.

Field	Type	Req.	Unit	Range
`schema`	const	yes	-	= example.readiness/v1
`product_version`	string	yes	-	pattern
`catalog_sha256`	string	yes	-	pattern
`contracts`	array of object	yes	-	1 to items

Table 14: Operations of the dormant read-only API contract (the OpenAPI document of the catalog): method and path, the operation and its response codes; every other method is refused.

Operation	Summary	Responses
`GET /health`	Liveness.	200
`GET /healthz`	Liveness (alias of /health).	200
`GET /api/cases`	The case index (catalog row case-index), baked and refused cases.	200, 503
`GET /api/cases/{case_id}/manifest`	One case manifest (catalog row case-manifest).	200, 404, 503
`GET /api/artifacts/{rel_path}`	An artifact a manifest names (catalog row trace), by its path under data/derived.	200, 404, 503
`GET /api/benchmark`	The benchmark results (catalog row benchmark-results).	200, 503
`GET /api/models/metrics`	The held-out metrics of the model slots (catalog row model-metrics).	200, 503
`GET /api/readiness`	What is real in this build (catalog row readiness).	200, 503

Table 15: The stages of the pipeline: the work each does, what it reads and what it writes; the training and evaluation stages run locally on a graphics processor and never in continuous integration or in the browser.

Stage	Work	Reads	Writes
preprocess	contract 1: schema and the rules a schema cannot state; a rejected record carries its reason	case source document	accepted records
feature extraction	derived geometry: trap count, extent, smallest spacing in bins, fraction of the half field used	accepted records	per-record features
train	offline training of the two learned models	generated training sets	model card, graph, training record
infer	every valid method through the registry with the same seed and budget; independent scoring; camera, particle and rearrangement drivers	records, registry	per-method results
evaluate	held-out metrics of the model slots	held-out sets, graphs	model metrics, evidence
export	contract 2: the compact trace and the manifest with the lane and gate outcomes	results, drivers	traces, manifests, index
benchmark	the benchmark protocol over the registry	registry, geometries	benchmark results

Table 16: Keyword defaults of every method, read from the signature of its `run` function by `params.py`, in two blocks; a default of `unset` is chosen at run time from the target, as the method section states, and methods with no keyword take only the interface arguments.

Method	Keyword	Default	Method	Keyword	Default
device-aware weighted GS	`steps`	3	direct search	`f`	0.5
	`warm`	12		`levels`	8
	`weights`	true		`max_trials`	unset
flicker-bounded phase-continuous sequence	`destination`	unset	conjugate-gradient minimization	`t`	unset
	`frames`	13		`efficiency`	unset
	`frame_iterations`	10	Wirtinger gradient descent with Adam	`lr`	1
	`end_phases`	unset		`efficiency`	unset
	`phase_velocity_max`	0.3	closed-loop weighted GS	`variant`	nogrette
	`n_fix`	12		`gain`	0.7
learned warm start	`n_fix`	12		`camera`	unset
	`form`	point		`n_inner`	unset
	`models_dir`	unset		`n_start`	30
aberration-robust hologram	`q`	8		`start`	gsw
	`lr`	0.02	device-aware CG with pixel crosstalk	`t`	unset
	`warm`	30		`efficiency`	unset
	`aberration`	0.02	Zernike correction	`mode`	recover
	`pointing_bins`	0		`injected`	unset
	`design`	unset		`known`	unset
	`level_fraction`	0.9		`camera`	unset
efficiency-uniformity Pareto front	`n_lambda`	20		`modes`	4 to 11
	`warm`	30		`trial_waves`	0.06
superposition of prisms and lenses	(none)			`base_iterations`	30
random superposition	(none)		Laguerre-Gaussian vortex traps	`charge`	unset
random mask	(none)		Hungarian assignment with phase interpolation	`atoms`	unset
GS	`form`	unset		`alpha`	1.5
	`start_waves`	20		`max_step_bins`	unset
generalized adaptive additive GS	`alpha`	0.5		`n_fix`	12
	`form`	auto	multi-plane CNN hologram	`models_dir`	unset
	`scale`	mean		`refine`	true
weighted GS	`form`	auto	optimal-transport initialized MRAF	`mix`	0.4
phase-fixed weighted GS	`n_fix`	12		`eps`	0.001
	`form`	auto		`ot_iterations`	200
mixed-region amplitude freedom	`mixing`	0.3	volumetric 3D GS	`dz_m`	unset
	`start_waves`	20		`nz`	unset

Table 17: The fourteen cases: category, optical setup, target and the methods that ran in the bake. C11 is refused by the input contract and never computed.

Case	Category	Setup	Target	Methods	Reference
C01	Arrays	850 nm, NA 0.5	9 traps	14	weighted GS
C02	Arrays	850 nm, NA 0.5	100 traps	14	phase-fixed weighted GS
C03	Arrays	850 nm, NA 0.5	1024 traps	7	phase-fixed weighted GS
C04	Shaped and 3D	850 nm, NA 0.5	disc potential, 8-bit	6	device-aware CG with pixel crosstalk
C05	Shaped and 3D	1064 nm, NA 1.2	51 traps	10	direct search
C06	Shaped and 3D	1064 nm, NA 1.2	25 traps	8	phase-fixed weighted GS
C07	Device and optics	1064 nm, NA 1.2	25 traps	13	phase-fixed weighted GS
C08	Device and optics	850 nm, NA 0.5	64 traps, 4-bit, fill 0.85	7	device-aware weighted GS
C09	Device and optics	850 nm, NA 0.5	16 traps, aberrated	6	Zernike correction
C10	Dynamics and controls	850 nm, NA 0.5	100 traps	4	Hungarian assignment with phase interpolation
C11	Dynamics and controls		one trap at 200 μm, outside the field	0	refused
C12	Arrays	850 nm, NA 0.5	100 traps	7	weighted GS
C13	Arrays	850 nm, NA 0.5	819 traps	7	phase-fixed weighted GS
C14	Shaped and 3D	850 nm, NA 0.5	48 traps	8	multi-plane CNN hologram

Table 18: The optical setups of the cases, read from the committed manifests and traces. The device row of C08 is the device the benchmark also uses for its device cells.

Setup	λ (nm)	f (mm)	NA	SLM pixels	pitch (μm)	pad	bin (μm)	half field (μm)
Atom (C01–C04, C09–C13)	850	6	0.5	256^2	20	2	0.498	127.5
Bead (C05–C07, water)	1064	2.4	1.2	256^2	10	2	0.499	127.7
C08	850	3	0.5	128^2	20	2	0.498	63.8
C14	850	2.56	0.5	128^2	20	1	0.850	54.4

The bead setup is in water of index 1.33; C14 is the volume of the released learned model. C08 device: 4-bit phase, crosstalk $\sigma_K = 1.24$, $q = 1.80$ on 3×3 sub-pixels, fill factor 0.85, flicker 0.05 rad.

Table 21: Calibration case C12 (10×10 traps at eight Fourier units): efficiency e and uniformity u of the analytic and Gerchberg–Saxton baselines, the mean over the three seeds of the canonical bake (the range over the seeds in brackets for u and σ), against the values published by Di Leonardo et al. for the same task.

Method	e (baked)	u (baked)	σ (%)	e (published)	u (published)
Superposition of prisms and lenses	0.328	0.000 [0.000, 0.000]	268.79 [268.79, 268.79]	0.29	0.01
Random superposition	0.736	0.183 [0.148, 0.202]	40.66 [37.40, 43.84]	0.69	0.01
Random mask	0.010	0.063 [0.050, 0.077]	50.97 [47.50, 54.25]	0.01	0.58
Gerchberg-Saxton	0.817	0.575 [0.506, 0.640]	18.49 [16.37, 19.97]	0.94	0.60
Generalized adaptive additive GS	0.813	0.696 [0.637, 0.738]	12.60 [11.81, 13.76]	0.93	0.79
Weighted GS	0.801	0.972 [0.970, 0.975]	1.15 [1.02, 1.41]	0.93	0.99
Phase-fixed weighted GS	0.792	0.990 [0.970, 1.000]	0.51 [0.01, 1.48]	n/a	n/a

Table 22: Baked results per case at the case budget, the mean over the three seeds of the canonical bake with the range over the seeds in brackets: the method with the lowest mean relative standard deviation among those with at least half the best mean efficiency, and the phase-fixed weighted GS and plain GS rows where they ran. e efficiency, σ relative standard deviation of the trap intensities (%).

				Phase-fixed weighted GS		
Case	Best	e	σ	σ	e	GS σ
C01	direct search	0.650	<0.01 [<0.01, <0.01]	<0.01 [<0.01, 0.01]	0.788	17.20 [7.34, 23.68]
C02	direct search	0.649	0.01 [0.01, 0.01]	0.38 [0.01, 0.85]	0.792	17.28 [15.99, 19.48]
C03	phase-fixed weighted GS	0.794	0.42 [0.20, 0.66]	0.42 [0.20, 0.66]	0.794	17.81 [17.08, 18.68]
C04	shaped potential	n/a	n/a	n/a	n/a	n/a
C05	direct search	0.679	0.01 [0.01, 0.01]	n/a	n/a	19.35 [16.75, 22.37]
C06	phase-fixed weighted GS	0.701	<0.01 [<0.01, <0.01]	<0.01 [<0.01, <0.01]	0.701	8.23 [7.33, 9.29]
C07	direct search	0.638	<0.01 [<0.01, <0.01]	<0.01 [<0.01, 0.01]	0.785	18.19 [13.36, 22.48]
C08	device-aware weighted GS	0.655	6.50 [5.91, 7.04]	13.21 [13.05, 13.42]	0.678	24.27 [21.62, 28.01]
C09	Zernike correction	0.799	0.33 [0.08, 0.68]	6.16 [4.66, 7.09]	0.447	15.35 [14.98, 15.71]
C10	flicker-bounded phase-continuous sequence	0.793	0.02 [<0.01, 0.06]	0.02 [0.01, 0.04]	0.792	n/a
C12	phase-fixed weighted GS	0.792	0.51 [0.01, 1.48]	0.51 [0.01, 1.48]	0.792	18.49 [16.37, 19.97]
C13	phase-fixed weighted GS	0.725	0.26 [0.18, 0.30]	0.26 [0.18, 0.30]	0.725	10.97 [10.62, 11.17]
C14	weighted GS	0.785	0.49 [0.32, 0.59]	n/a	n/a	16.91 [15.26, 18.41]

Table 25: Relative standard deviation (percent) of selected methods on three cases: mean over the three seeds of the bake and the minimum and maximum over the seeds in brackets, the uncertainty the bake supports; C02 is the ideal-device grid, C08 the device-limited array, C09 the aberrated array.

Method	C02	C08	C09
Gerchberg-Saxton	17 [16, 19]	24 [22, 28]	15 [15, 16]
Weighted GS	1.2 [0.86, 1.5]	14 [12, 14]	6 [4.8, 7.3]
Phase-fixed weighted GS	0.38 [0.014, 0.85]	13 [13, 13]	6.2 [4.7, 7.1]
Wirtinger gradient descent with Adam	2.5 [2.4, 2.6]		
Device-aware weighted GS		6.5 [5.9, 7]	
Aberration- and pointing-robust hologram	0.85 [0.83, 0.87]	13 [13, 14]	5.3 [4.6, 5.8]
Efficiency-uniformity Pareto front	0.53 [<0.001, 1.1]		

Table 26: Power budget of the point-trap cases at the bake seed (one seed per case): the fraction of the input power in the traps e, in the zero order z_0 and in the ghost traps g (everything else on the grid, including folded replicas), for plain GS and phase-fixed weighted GS where they ran. The remainder of the power is zero for the plain Fourier form.

	GS			Phase-fixed weighted GS		
Case	e	z_0	g	e	z_0	g
C01	0.819	0.108	0.120	0.817	0.091	0.122
C02	0.818	0.000	0.117	0.798	0.000	0.140
C03	0.817	0.000	0.115	0.794	0.000	0.138
C06	0.710	0.000	0.449	0.704	0.000	0.461
C07	0.807	0.034	0.126	0.791	0.032	0.144
C08	0.692	0.008	0.133	0.675	0.008	0.148
C09	0.458	0.000	0.423	0.453	0.000	0.430
C12	0.817	0.000	0.123	0.797	0.000	0.145
C13	0.740	0.000	0.202	0.723	0.000	0.217

Table 27: Benchmark, per geometry and device: the published method and the proposed method with the lowest median relative standard deviation σ (%) at 100 iterations among those that reach at least half of the best median efficiency of the cell, and the best method of all at a time budget of 1 s. Default parameters, medians over the seeds of each cell.

Geometry	Device	Published	Proposed	Best at 1 s
G1 Square 10 x 10	ideal	direct search 0.071	device-aware weighted GS <0.001	device-aware weighted GS <0.001
G1 Square 10 x 10	C08	Wirtinger gradient descent with Adam 12	aberration-robust hologram 24	Wirtinger gradient descent with Adam 20
G2 Ring of 36 traps	ideal	phase-fixed weighted GS <0.001	device-aware weighted GS <0.001	device-aware weighted GS <0.001
G2 Ring of 36 traps	C08	superposition of prisms and lenses 6.6	device-aware weighted GS 7.5	superposition of prisms and lenses 6.6
G3 Random 100 traps	ideal	phase-fixed weighted GS <0.001	device-aware weighted GS <0.001	device-aware weighted GS <0.001
G3 Random 100 traps	C08	Wirtinger gradient descent with Adam 14	device-aware weighted GS 4.4	Wirtinger gradient descent with Adam 20
G4 Quasi-periodic 119	ideal	phase-fixed weighted GS 0.012	device-aware weighted GS <0.001	device-aware weighted GS <0.001
G4 Quasi-periodic 119	C08	Wirtinger gradient descent with Adam 21	device-aware weighted GS 81	conjugate-gradient minimization 76
G5 3D lattice 5 x 5 x 3	ideal	direct search 0.042	flicker-bounded phase-continuous sequence <0.001	flicker-bounded phase-continuous sequence <0.001
G6 Large: 32 x 32 and disordered 819	ideal	direct search 0.15	device-aware weighted GS <0.001	phase-fixed weighted GS 0.78
G6 Large: 32 x 32 and disordered 819	C08	Wirtinger gradient descent with Adam 18	device-aware weighted GS 17	weighted GS 19
G7 Shaped fields (ring, disc, line, LG ring)	ideal	GS 35	n/a	GS 36
G7 Shaped fields (ring, disc, line, LG ring)	C08	GS 40	n/a	optimal-transport initialized MRAF 39

Table 28: Benchmark, per method: medians over the point-trap geometries the method ran on (of 6) at 100 iterations with the default parameters, on the ideal device and on the C08 device. σ relative standard deviation (%): the median over the geometries of the cell medians and, in brackets, the median over the geometries of the first and third quartiles over the seeds of each cell; e efficiency; time wall-clock seconds under shared load.

Method	Geometries	σ ideal	e ideal	time (s)	σ C08	e C08
Published methods						
Superposition of prisms and lenses	6	136 [66, 136]	0.399	0.0044	175 [27, 175]	0.279
Random superposition	6	25 [19, 33]	0.710	0.0053	31 [21, 49]	0.478
Random mask	6	78 [75, 82]	0.015	0.068	89 [85, 94]	0.013
Gerchberg-Saxton	6	14 [11, 15]	0.799	0.82	31 [22, 33]	0.523
Generalized adaptive additive GS	6	9.1 [7.8, 11]	0.780	1.1	24 [21, 30]	0.514
Weighted GS	6	0.3 [0.19, 0.35]	0.775	0.72	21 [20, 27]	0.518
Phase-fixed weighted GS	5	0.012 [<0.001, 0.17]	0.751	0.55	26 [20, 28]	0.503
Mixed-region amplitude freedom	0	n/a	n/a	n/a	n/a	n/a
Direct search	6	0.059 [0.057, 0.06]	0.641	58	20 [20, 23]	0.431
Conjugate-gradient minimization	6	2.6 [2, 3]	0.751	0.66	21 [20, 25]	0.486
Wirtinger gradient descent with Adam	6	1.2 [0.91, 1.2]	0.776	0.4	14 [11, 15]	0.510
Closed-loop weighted GS	2	19 [19, 20]	0.822	7.2	34 [26, 35]	0.575
Device-aware CG with pixel crosstalk	5	2.2 [2, 2.7]	0.737	0.77	17 [14, 18]	0.486
Zernike correction and sensorless modal recovery	6	0.83 [0.68, 1]	0.742	0.48	20 [20, 28]	0.511
Optimal-transport initialized MRAF	0	n/a	n/a	n/a	n/a	n/a
Volumetric 3D GS	1	16 [15, 17]	0.890	6.9	n/a	n/a
Methods proposed here						
Device-aware weighted GS	5	<0.001 [<0.001, <0.001]	0.751	18	17 [11, 24]	0.501
Flicker-bounded phase-continuous sequence	6	<0.001 [<0.001, 0.097]	0.756	2.5	21 [20, 25]	0.503
Learned warm start for phase-fixed WGS	6	0.006 [<0.001, 0.13]	0.754	1	21 [20, 28]	0.503
Aberration- and pointing-robust hologram	5	0.4 [0.39, 0.41]	0.744	26	21 [20, 25]	0.499
Efficiency-uniformity Pareto front	6	0.31 [0.28, 0.36]	0.774	57	21 [20, 25]	0.529

Table 29: Phase-fixed weighted GS on the crowded 50×30 array: the minimum relative standard deviation reached in 150 iterations against the freeze index N (one seed, Fourier form), measured by `acceptance.py`. The source value does not transfer to this grid.

N	minimum σ (percent)
12	3.575
20	2.133
30	0.855
40	0.080
60	0.143

Table 30: Ablation of the weight update of the device-aware weighted GS, the method run with its weight update and with every weight kept at one. First block: the 10×10 acceptance task, median [min, max] of the relative standard deviation σ (%) over the seeds. Second block: the four benchmark geometries on the C08 device at 10 outer iterations, median over three seeds of σ and of the efficiency e.

Task	σ with weights	σ no weights	e with	e no weights
acceptance task, crosstalk device (5 seeds)	0.205 [0.157, 0.298]	0.0297 [0.0163, 0.0319]		
acceptance task, 4-bit device (3 seeds)	0.724 [0.597, 1.16]	0.242 [0.202, 0.248]		
G1 10 x 10 square (C08 device, 3 seeds)	29	14	0.598	0.563
G2 ring of 36 (C08 device, 3 seeds)	7.8	8.9	0.453	0.464
G3 100 random (C08 device, 3 seeds)	4.4	4.3	0.401	0.408
G4 quasi-periodic 119 (C08 device, 3 seeds)	65	65	0.493	0.493

Table 31: Stored Pareto fronts of the benchmark (the converged run of the first seed, non-dominated points only): the number of stored points, and the efficiency e and uniformity u of the first and of the last stored point ordered by the trade-off parameter, per geometry with a front, on the ideal device.

Geometry	Points	e first	u first	e last	u last
G1	7	0.822	1.000	0.838	0.947
G2	2	0.693	1.000	0.699	0.998
G3	3	0.692	1.000	0.708	0.997
G4	3	0.751	1.000	0.766	0.991
G5	3	0.859	1.000	0.893	0.995
G6	1	0.791	1.000	0.791	1.000

Table 32: Checks that guard each class of result: the quantity checked, the independent reference it is compared with and the test module that holds it; the tolerance of each is stated in the test and in the design record of the method.

Quantity	Independent reference	Test module
Fourier form against point-trap form	agreement at on-bin traps on four grid sizes	`tests/engine/test_forward`
Power conservation	Parseval's relation	`tests/engine/test_forward`
Depth propagation	analytic Gaussian beam at one Rayleigh range	`tests/engine/test_forward`
Crosstalk adjoint	inner-product identity, three devices	`tests/engine/test_device`
Pixel envelope	brute-force transform, two fill factors	`tests/engine/test_device`
Analytic gradients (conjugate-gradient minimization, Wirtinger descent with Adam, device-aware conjugate gradient)	central finite differences	`tests/methods/optim`
Random-mask efficiency	the inverse of the trap count	`tests/test_oracles`
Vortex winding	$2\pi l$ on a ring of sample points	`tests/methods/control`
Bead spectrum and equipartition	Lorentzian corner frequency, equipartition variance	`tests/test_oracles`
Bead histogram	Boltzmann law of the computed potential	`tests/engine/test_dynamics`
Assignment	brute force on random matrices	`tests/methods/control`
Byte-identical artifacts	two bakes of the same inputs	`tests/test_manifest`
TypeScript engine	fixtures written by the Python engine	`frontend/src/engine/holo`